MACHINE LEARNING (23A31401T)

UNIT – I : Introduction to Machine Learning

Easy digital notes with diagrams for students

Topics in this unit

1. Evolution of ML, paradigms for ML
2. Learning by rote, by induction, reinforcement learning
3. Types of data and matching
4. Stages in machine learning (the full pipeline)
5. Data acquisition, feature engineering, data representation
6. Model selection, learning, evaluation, prediction
7. Search and learning, data sets

> * **Study tip:**
> Every topic has a small diagram. Learn the RED words first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Introduction to Machine Learning: Evolution of Machine Learning, Paradigms for ML, Learning by Rote, Learning by Induction, Reinforcement Learning, **Types of Data**, Matching, Stages in Machine Learning, Data Acquisition, Feature Engineering, Data Representation, Model Selection, Model Learning, Model Evaluation, Model Prediction, Search and Learning, Data Sets.

Part I : Basics, Evolution and Paradigms

I.I Evolution of Machine Learning

Machine Learning (ML) = a branch of AI where a computer **learns from data** and improves with experience, without being told every rule.

- **Tom Mitchell (1997):** a program learns from experience **E** for task **T** with performance **P**, if its performance on T improves with E.
- Example: T = spam detection, E = emails marked spam / not spam, P = % of mails classified correctly.
- AI is the big idea, ML is a part of AI, and **Deep Learning (DL)** is a part of ML.

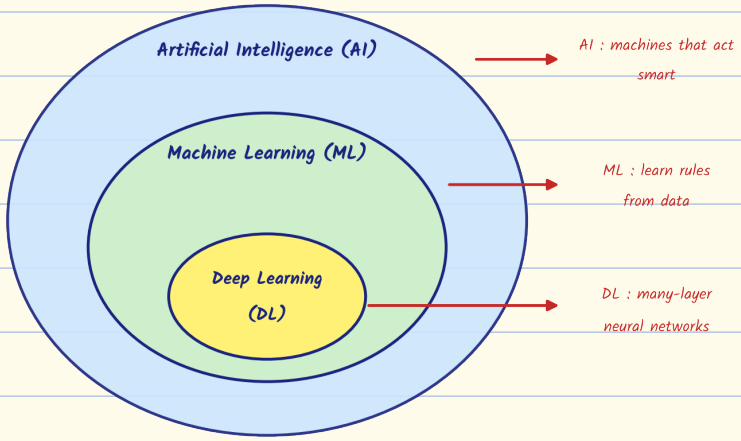


Fig I.I : AI, ML and DL (nested)

Traditional programming: human writes rules, computer applies them to data. **ML:** we give data and answers, computer finds the rules (the **model**).

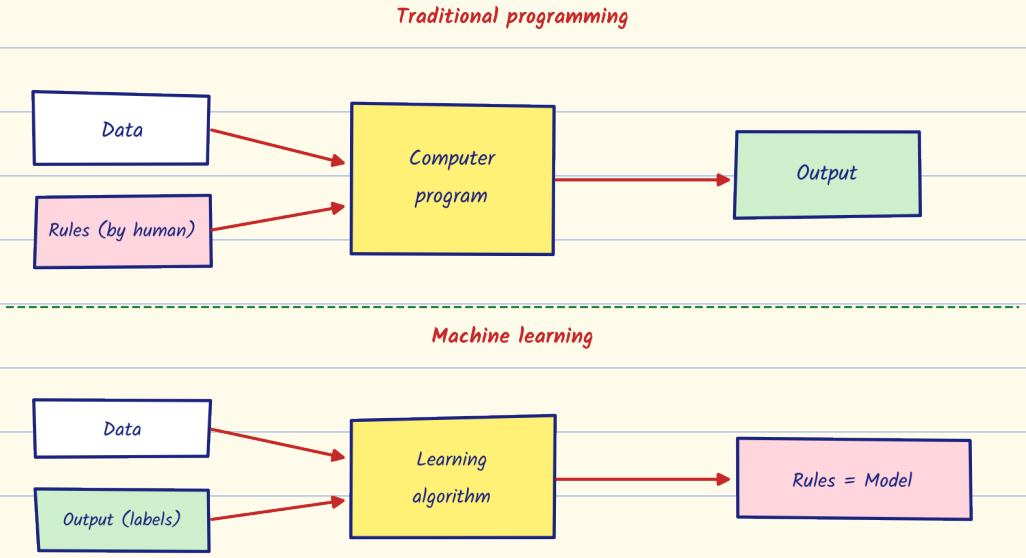


Fig 1.2 : Traditional programming vs machine learning

Short history of ML:

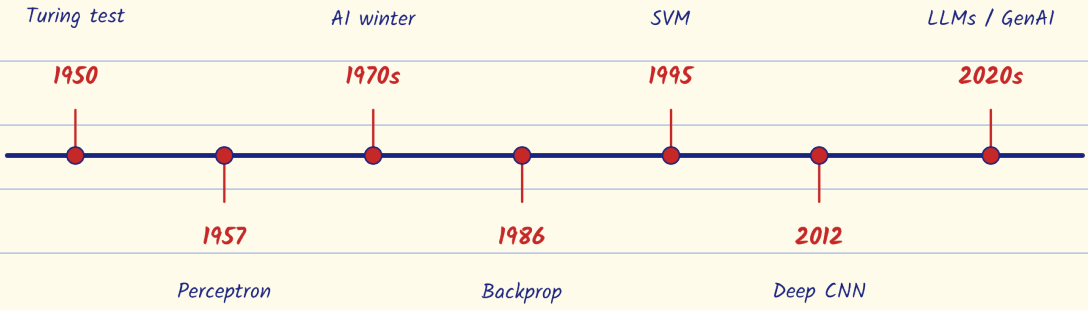


Fig 1.3 : Timeline of machine learning

Period	Key idea
1940s-50s	Neuron model (McCulloch-Pitts), Turing test (1950), Samuel checkers program - term "machine learning" (1959)
1957-60s	Rosenblatt Perceptron - first learning machine; nearest neighbour rule
1970s	Limits of perceptron found -> **AI winter**; rule-based expert systems
1980s-90s	Backpropagation (1986), decision trees, **SVM** (1995), statistical ML, Bayesian methods
2000s	Big data, ensembles (random forest, boosting), Hadoop, better CPUs
2012 onwards	**Deep learning** with GPUs (AlexNet), speech, vision; later Transformers and LLMs / generative AI

* **Remember:**

Three things made ML boom : **more data**, **faster hardware (GPU)** and **better algorithms**.

I.2 Paradigms for Machine Learning

A **paradigm** = a style of learning. It depends on what kind of feedback the learner gets.

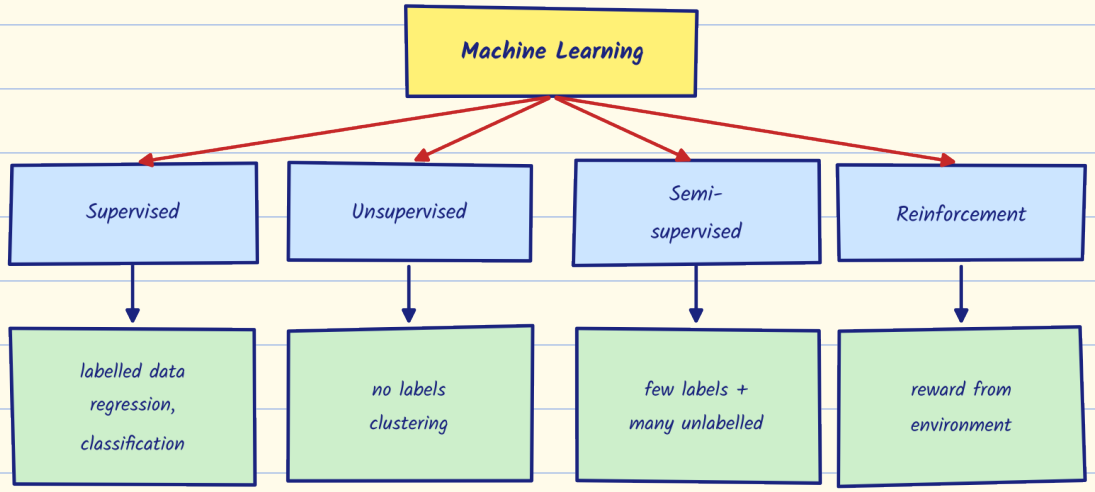


Fig I.4 : Paradigms (types) of machine learning

Paradigm	Data used	Goal	Example
Supervised	Inputs + correct labels	Learn map x -> y (classification, regression)	Spam filter, house price
Unsupervised	Inputs only, no labels	Find hidden structure (clustering, association)	Customer groups
Semi-supervised	Few labelled + many unlabelled	Use cheap unlabelled data to help	Photo tagging
Reinforcement	Reward / penalty signal	Learn best actions	Game playing, robots

- **Classification** - output is a class (spam / not spam). **Regression** - output is a number (price = 52.3 lakh).
- Learning can also be seen as **rote**, **by induction** and **by reinforcement** - explained next.

I.3 Learning by Rote

Rote learning = learning by **memorising**. The system simply stores the examples (a lookup table) and gives the stored answer when the same input comes again.

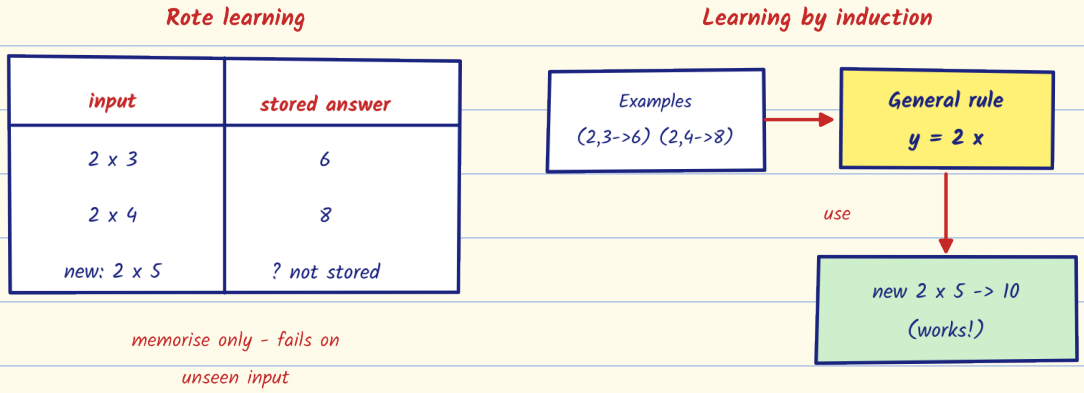


Fig 1.5 : Rote learning (lookup table) vs learning by induction

- No understanding, **no generalisation** - an unseen input has no answer.
- Very fast to recall, but memory grows with every new example.
- Example: Samuel checkers program stored board positions with their values; a student mugging up tables.
- In ML terms, a pure **look-up / caching** system (store input -> output).

1.4 Learning by Induction

Induction = going from **specific examples to a general rule**. The learner sees many examples and builds a hypothesis that also works for **new, unseen** data.

- Seen: sparrow flies, crow flies, eagle flies -> rule: **birds fly** (a general hypothesis).
- This is the **core of supervised learning** : training examples -> model -> predict on new data.
- Uses **inductive bias** = assumptions that help pick one rule among many possible rules.
- Induced rules may be wrong (penguins do not fly), so we must **test** them on fresh data.

Point	Rote learning	Induction
What is stored	Raw examples	General rule / model
Unseen input	Fails	Can predict
Generalisation	None	Yes (main aim)
Memory	Grows with data	Fixed size model

1.5 Reinforcement Learning

In **reinforcement learning (RL)** an **agent** interacts with an **environment**. It takes an **action**, and gets a new **state** and a **reward** (or penalty). It learns a **policy** that gives the maximum total reward.

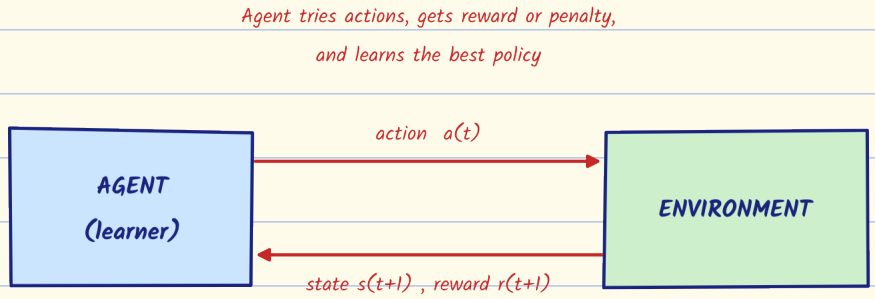


Fig 1.6 : Agent - environment loop in RL

- No labelled answers - only **trial and error** with delayed reward.
- Key terms: **agent, environment, state, action, reward, policy**.
- Balance of **exploration** (try new actions) and **exploitation** (use best known action).
- Examples: chess / Go (AlphaGo), self-driving car, robot walking, ad recommendation.

> * **Example:**
> A robot in a maze: wall hit = -1, reaching exit = +10. After many tries it learns the shortest path - nobody told it the path.

Part 2 : Types of Data and Matching

1.6 Types of Data

ML needs data, and the **type of data** decides which algorithm and pre-processing we use.

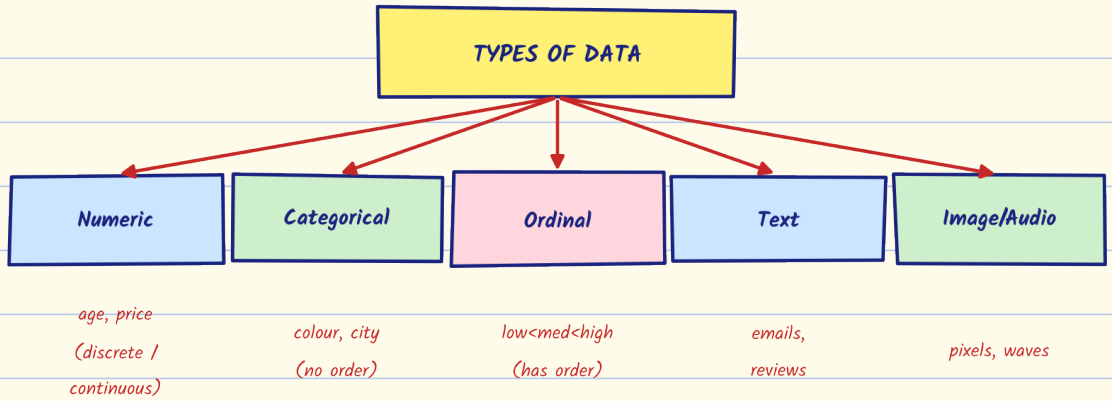


Fig 1.7 : Types of data

Type	Meaning	Example
Numeric (quantitative)	Numbers; **discrete** (counts) or **continuous** (measured)	Age 21, salary 35000, temp 36.6

Type	Meaning	Example
Categorical (nominal)	Labels without order	Gender, city, colour
Ordinal	Categories with a meaningful order	Grade A>B>C, rating 1-5
Binary	Only two values	Yes/No, 0/1
Text	Unstructured words	Reviews, tweets, emails
Image / audio / video	Pixel or signal values	X-ray, MNIST digits, speech
Time-series	Values in time order	Stock price, ECG, sensor log

- **Structured** data = rows and columns (database). **Unstructured** = text, images, audio. **Semi-structured** = JSON, XML.
- **Labelled** data has the correct output; **unlabelled** data does not.
- Categorical data must be **encoded** to numbers (e.g. one-hot) before most models.

I.7 Matching

Matching = comparing a **new pattern** with stored patterns or **prototypes** to find the closest one. The class of the best match is the answer.

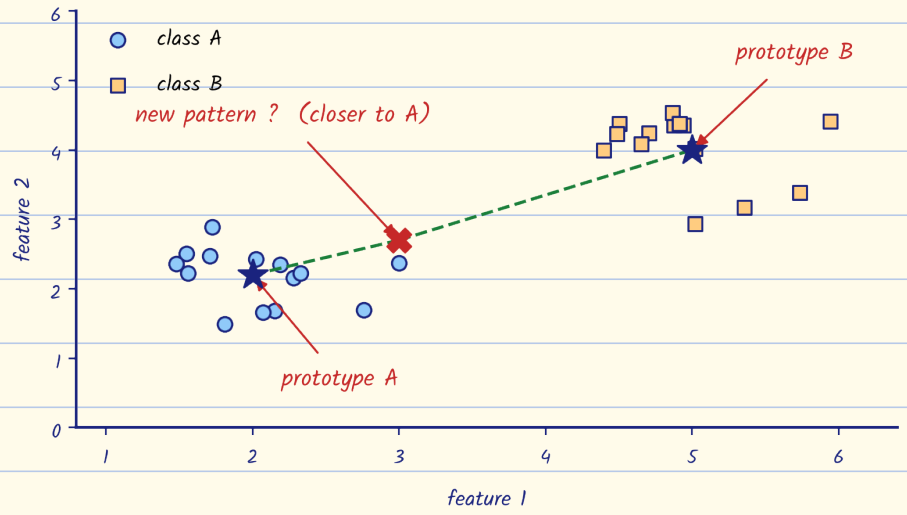


Fig I.8 : Matching a new pattern with class prototypes

- Needs a **similarity / distance measure** : Euclidean, Manhattan, cosine.
- Euclidean distance: d(a,b) = sqrt((a1-b1)^2 + (a2-b2)^2 + ...).
- **Exact matching** : identical (string, key lookup). **Approximate matching** : nearest / most similar.
- Used in **nearest neighbour** classifiers, template matching, face / fingerprint recognition, search engines.

- Matching can be against **every stored example** (k-NN) or only a **representative prototype / mean** of each class (nearest centroid).

* **Example:**
New point (3, 2.7). Distance to prototype A (2, 2.2) = sqrt(1 + 0.25) = 1.12. Distance to B (5, 4) = sqrt(4 + 1.69) = 2.39. Smaller distance -> class **A**.

Part 3 : Stages in Machine Learning

1.8 Stages in Machine Learning

Building an ML system is a **pipeline** of stages. Results of later stages often send us **back** to improve earlier ones.

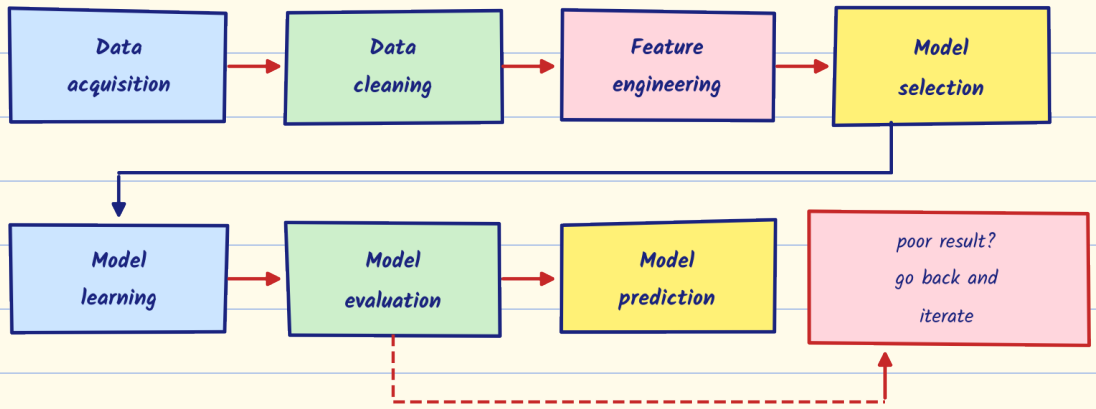


Fig 1.9 : Stages (pipeline) of a machine learning project

1. **Problem definition** - what to predict, what is success?
2. **Data acquisition** - collect data.
3. **Data preparation / cleaning** - missing values, noise, duplicates.
4. **Feature engineering** - build useful features.
5. **Model selection** - pick algorithm.
6. **Model learning (training)**.
7. **Model evaluation** - test on unseen data.
8. **Model prediction** (deployment) - use on new data, monitor.

1.9 Data Acquisition

Data acquisition = collecting the raw data needed to train the model. **Good data matters more than a fancy algorithm** (garbage in, garbage out).

- **Sources:** databases, sensors / IoT, web scraping, APIs, surveys, logs, public repositories (UCI, Kaggle).
- **Quality checks:** enough **quantity**, **relevance**, **accuracy**, balanced classes, no bias, legal / privacy rules.
- **Cleaning:** fill **missing values** (mean / median), remove **duplicates**, handle **outliers** and noise, fix wrong labels.
- **Pre-processing: normalisation** (scale to 0-1) : x_new = (x - min) / (max - min); **standardisation** : z = (x - mean) / std.

1.10 Feature Engineering

A **feature** = one measurable property of the data (age, pixel value, word count). **Feature engineering** = creating, choosing and transforming features so the model learns better.

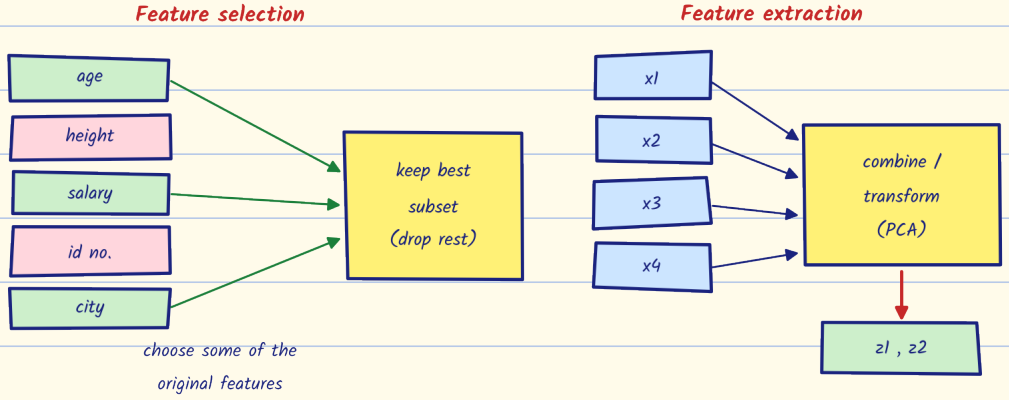


Fig 1.10 : Feature selection vs feature extraction

Technique	Idea	Example
Feature construction	Make new feature from old	BMI = weight / height^2; date -> day of week
Feature selection	Keep the useful subset, drop the rest	Drop id number; filter, wrapper, embedded methods
Feature extraction	Combine features into fewer new ones	PCA, LDA, TF-IDF for text
Encoding / scaling	Make features model-friendly	One-hot, label encoding, normalisation

- Benefits: **less over-fitting**, faster training, better accuracy, easier to understand.
- **Curse of dimensionality** : too many features need exponentially more data.

1.11 Data Representation

Computers need data as **numbers** in a fixed form. Each sample is written as a **feature vector**, and the whole data set as a **matrix**.

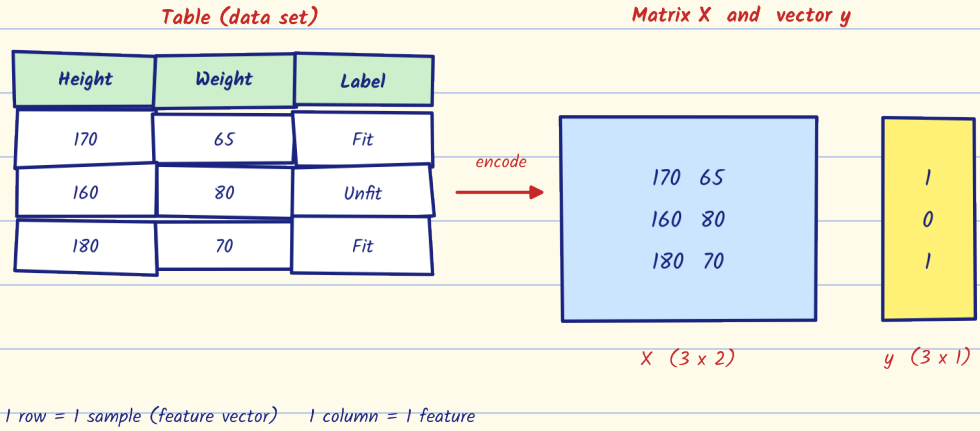


Fig 1.11 : Data set as a table, feature matrix X and label vector y

- Sample i = **feature vector** $x = (x1, x2, ..., xd)$. Data set = matrix X of size $n \times d$ (n samples, d features).
- Labels are stored in vector **y** (size n). Supervised data = pairs (x, y).
- **Images** -> pixel matrix / flattened vector. **Text** -> bag of words, TF-IDF, word embeddings. **Categories** -> one-hot vectors.
- Other representations: **graphs** (social network), **trees**, **sequences** (time-series), **logic rules**.

> * **Remember:**
> One-hot for colour {Red, Green, Blue}: Red = (1,0,0), Green = (0,1,0), Blue = (0,0,1). No false order is created.

1.12 Model Selection

Model selection = choosing the best **type of model** (and its settings) for the problem. A model is a function **h(x)** with **parameters**.

- Depends on: **type of problem** (classification / regression / clustering), size and type of data, need for interpretability, time and memory.
- Candidates: linear / logistic regression, k-NN, decision tree, SVM, naive Bayes, neural network.
- **Parameters** are learnt from data (weights). **Hyper-parameters** are set by us before training (k in k-NN, tree depth, learning rate) and tuned with **validation data / cross-validation**.

- **Occam's razor** : among models with similar accuracy, choose the **simpler** one.
- Model complexity balance: too simple -> **under-fit** (high bias); too complex -> **over-fit** (high variance).

1.13 Model Learning

Model learning (training) = using the training data to find the parameters that give the **least error**.

- Define a **loss (cost) function**, e.g. mean squared error: J = (1/n) x sum of (y - y_hat)^2.
- An **optimisation** method reduces the loss step by step - e.g. **gradient descent** : w = w - alpha x (slope of J).
- Loop: predict -> measure loss -> update parameters -> repeat (each full pass over data = **epoch**).
- Training ends when the loss stops falling or after a set number of epochs.
- Learning type decides the loss: labels (supervised), similarity (clustering), reward (RL).



Fig 1.12 : Under-fitting, good fit and over-fitting

1.14 Model Evaluation

Model evaluation = measuring how well the trained model works on **unseen data**. Training accuracy alone is misleading (over-fitting).

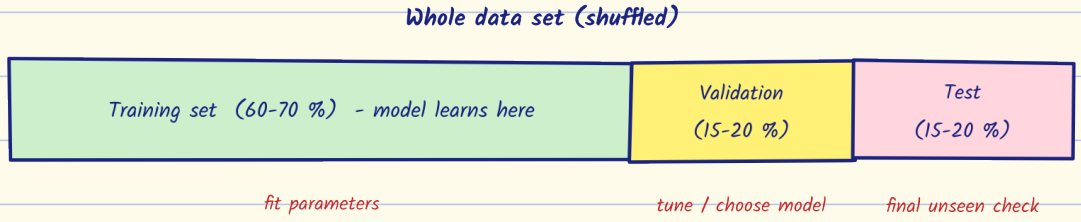


Fig 1.13 : Train / validation / test split

- **Training set** - learn parameters. **Validation set** - tune hyper-parameters, pick model. **Test set** - final honest score, used **once**.

- **k-fold cross-validation** : split into k parts; train on k-1, test on 1; repeat k times; average the score.
- **Over-fitting** : training error low, validation error high. **Under-fitting** : both errors high.

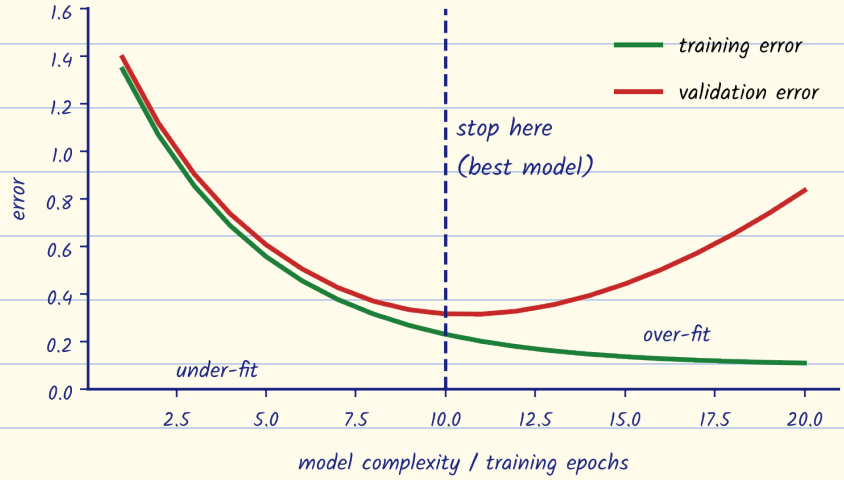


Fig 1.14 : Validation error rises after the best point (over-fitting)

Metric	Formula / meaning
Accuracy	(TP + TN) / total predictions
Precision	TP / (TP + FP) - of predicted positives, how many are right
Recall	TP / (TP + FN) - of real positives, how many found
F1 score	2 x P x R / (P + R)
MSE / RMSE	For regression : mean of (y - y_hat)^2, and its square root

TP, TN, FP, FN come from the **confusion matrix** (true / false positives and negatives).

1.15 Model Prediction

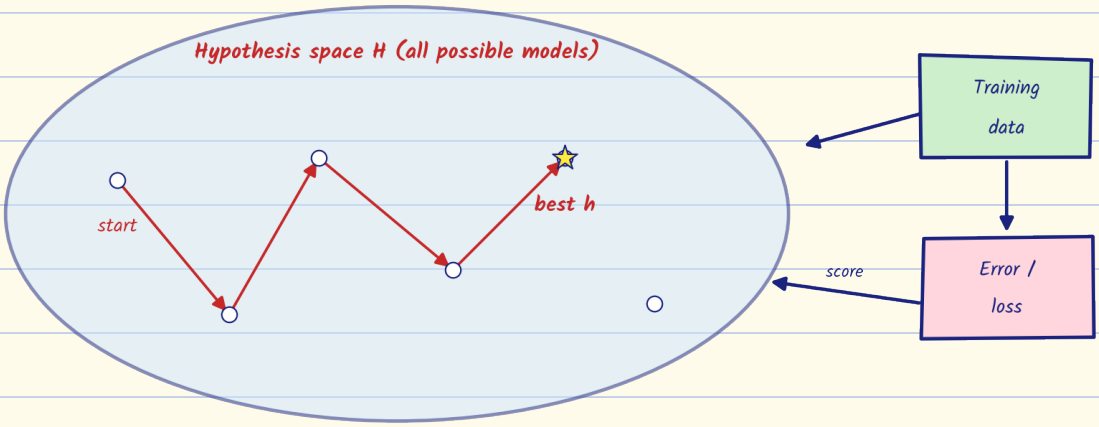
Model prediction (inference) = using the final trained model to give outputs for **new, real-world data**.

- The model is fixed now - the new input gets the **same pre-processing and features** as training data.
- Output: a class / probability (classification), a number (regression), a cluster id, or an action (RL).
- **Deployment:** app, web API, embedded device. Then **monitor** for **data drift** and **retrain** when accuracy falls.
- Example: trained spam model receives a new email -> features -> **P(spam) = 0.93** -> moved to spam folder.

Part 4 : Search and Learning, Data Sets

1.16 Search and Learning

Learning can be seen as a **search problem**: look through a huge **hypothesis space H** (all models we could build) to find the hypothesis **h** that best fits the training data.



learning = searching H for the h with the least error

Fig 1.15 : Learning as search in the hypothesis space

- **Hypothesis space H** - the set of all functions the learner can represent (e.g. all straight lines, all trees of depth 3).
- **Search strategy** : how we move in H - **gradient descent** (continuous), greedy tree building, **genetic algorithms**, hill climbing, exhaustive search (too costly).
- **Evaluation function** guides the search: error / loss on training data (or likelihood).
- **Version space** : the set of all hypotheses consistent with the examples; shrinks as examples increase (Find-S, Candidate-Elimination).
- **Inductive bias** limits H or orders it, so search is possible and generalises.
- Too big an H -> over-fitting and heavy search; too small -> may not contain a good h.

1.17 Data Sets

A **data set** = a collection of samples (rows) with their features (columns) used to train and test models. Standard **benchmark data sets** let us compare algorithms.

Data set	Type / task	Size and details
Iris	Numeric, classification	150 flowers, 4 features (sepal / petal length, width), 3 classes

Data set	Type / task	Size and details
MNIST	Images, classification	70,000 handwritten digits 28x28, 10 classes (60k train, 10k test)
Boston / California housing	Numeric, regression	House features -> price
Titanic	Mixed, binary classification	Survived or not; age, sex, class
CIFAR-10 / ImageNet	Images, classification	Object photos; ImageNet has 1000+ classes
IMDB reviews	Text, sentiment	Positive / negative movie reviews
Wine, Breast cancer (UCI)	Numeric, classification	Chemical / cell measurements

- Sources : **UCI ML Repository**, **Kaggle**, scikit-learn built-in sets, OpenML.
- A good data set is **large, clean, representative and balanced**.
- Always split into **train / validation / test**. Use **stratified** split if classes are imbalanced.

* **Remember:**
Training data teaches the model, **validation data** tunes it, **test data** judges it. Never let test data leak into training.

Part 5 : Quick Revision and Exam Questions

Quick Revision

- **ML** = learning rules from data to improve on a task (Mitchell : T, E, P). AI > ML > DL.
- **Paradigms** : supervised (labels), unsupervised (no labels), semi-supervised (few labels), reinforcement (reward).
- **Rote** = memorise, no generalisation. **Induction** = examples -> general rule.
- **RL** : agent, environment, state, action, reward, policy; trial and error.
- **Data types** : numeric, categorical, ordinal, binary, text, image, time-series.
- **Matching** = compare new pattern with prototypes using distance / similarity.
- **Stages** : data acquisition -> cleaning -> feature engineering -> model selection -> learning -> evaluation -> prediction.
- **Feature selection** keeps a subset; **extraction** builds new features (PCA).
- **Representation** : feature vector, matrix X (n x d), label vector y, one-hot.
- **Learning** minimises loss; **evaluation** uses unseen test data; over-fit = low train, high test error.

- **Learning = search** in hypothesis space guided by error.

Likely exam questions

Short answer (2–3 marks)

1. Define machine learning (Tom Mitchell). How is it different from traditional programming?
2. Differentiate rote learning and learning by induction.
3. List four types of data with examples.
4. What is feature engineering? Give two techniques.
5. What are over-fitting and under-fitting?
6. Define hypothesis space and version space.

Long answer (8–10 marks)

1. Explain the paradigms of machine learning with suitable diagrams and examples.
2. Explain reinforcement learning with the agent-environment model.
3. Describe the various stages in a machine learning project with a neat block diagram.
4. Explain data acquisition, feature engineering and data representation in detail.
5. Explain model selection, learning, evaluation and prediction. Describe train / validation / test split and metrics.
6. Explain how learning can be viewed as a search problem. Mention matching and common data sets.

MACHINE LEARNING

UNIT – II : Nearest Neighbor-Based Models

Easy digital notes with diagrams for students (JNTU B.Tech, 23A3140IT)

Topics in this unit

1. Introduction to proximity measures and distance measures
2. Non-metric similarity functions
3. Proximity between binary patterns
4. Classification algorithms based on distance measures
5. K-Nearest Neighbor (KNN) classifier and choosing k
6. Radius distance nearest neighbor algorithm
7. KNN regression
8. Performance of classifiers
9. Performance of regression algorithms

> * **Study tip:**
> Do one numeric example of each measure by hand - exam questions are mostly 'calculate the distance / classify the point / find accuracy'. The worked examples here use small tables so you can copy the method.

Syllabus (as given)

Nearest Neighbor-Based Models: Introduction to Proximity Measures, Distance Measures, Non-Metric Similarity Functions, Proximity Between Binary Patterns, Different Classification Algorithms Based on the Distance Measures, K-Nearest Neighbor Classifier, Radius Distance Nearest Neighbor Algorithm, KNN Regression, Performance of Classifiers, Performance of Regression Algorithms.

Part I : Proximity and Distance Measures

2.1 Introduction to Proximity Measures

Proximity tells how **close or alike** two patterns are. It has two faces: **similarity** (large value = more alike) and **dissimilarity / distance** (small value = more alike). Nearest-neighbor models are built on one idea: **similar inputs give similar outputs**.

- **Proximity matrix:** for n patterns, an n × n table whose (i, j) entry is the distance (or similarity) between pattern i and j. It is symmetric with zeros on the diagonal for distances.
- **Instance-based / lazy learning:** KNN does **no training**. It stores all examples and does the work only when a query arrives (memory-based). Compare with **eager** learners (tree, SVM) that build a model first.

- **Why choice of measure matters:** the neighbors found, and hence the answer, change with the measure. Numeric data: Euclidean, Manhattan. Binary data: Hamming, Jaccard. Text: cosine. Probabilities: KL divergence.
- **Conversion:** a similarity s in $[0,1]$ gives a distance $d = 1 - s$; a distance gives similarity $s = 1 / (1 + d)$.

2.2 Distance Measures

A function $d(x, y)$ is a **metric (true distance)** if it satisfies four rules for all patterns x, y, z:

Property	Rule	Meaning
Non-negativity	$d(x, y) \geq 0$	distance is never negative
Identity	$d(x, y) = 0 \Leftrightarrow x = y$	zero only for the same point
Symmetry	$d(x, y) = d(y, x)$	A to B equals B to A
Triangle inequality	$d(x, z) \leq d(x, y) + d(y, z)$	a detour is never shorter

For d-dimensional vectors $x = (x1..xd)$, $y = (y1..yd)$ the common distances are:

Measure	Formula	Remark
Euclidean (L2)	$d = \text{sqrt}(\Sigma (xi - yi)^2)$	straight-line; most used; sensitive to scale
Manhattan (L1)	$d = \Sigma \lvert xi - yi \rvert$	city-block; less affected by outliers
Chebyshev (L∞)	$d = \max \lvert xi - yi \rvert$	largest single difference
Minkowski (Lp)	$d = (\Sigma \lvert xi - yi \rvert ^ p)^{(1/p)}$	general form; p = 1, 2, ∞ give the three above
Mahalanobis	$d = \text{sqrt}((x - y)^T \Sigma^{-1} (x - y))$	uses covariance Σ; removes scale and correlation effects

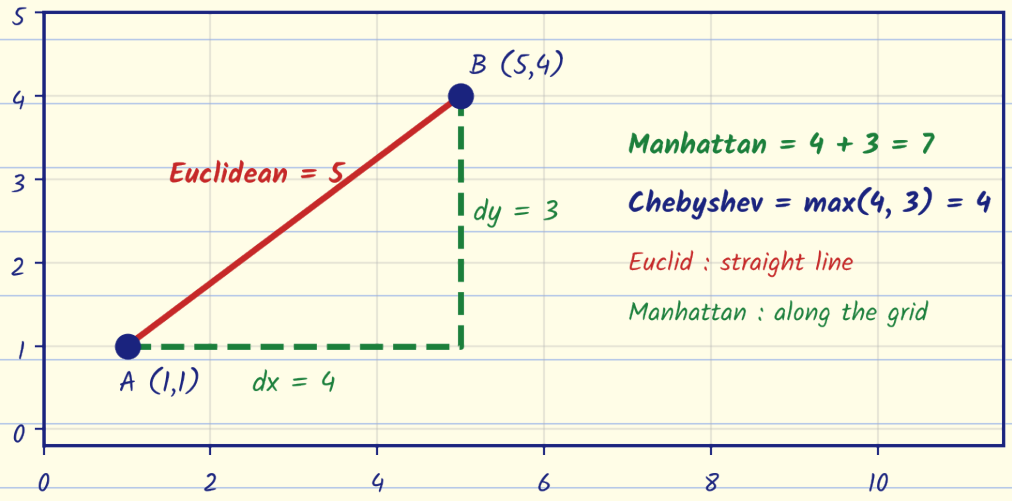


Fig 2.1 : Euclidean, Manhattan and Chebyshev paths between two points

- **Worked example:** $x = (1,1)$, $y = (5,4)$. Euclidean = $\text{sqrt}(4^2 + 3^2) = \mathbf{5}$. Manhattan = $4 + 3 = \mathbf{7}$. Chebyshev = $\max(4,3) = \mathbf{4}$. Minkowski $p = 3$ = $(4^3 + 3^3)^{(1/3)} = (91)^{(1/3)} = \mathbf{4.50}$.
- Always **Chebyshev ≤ Euclidean ≤ Manhattan**. Larger p gives more weight to the biggest difference.

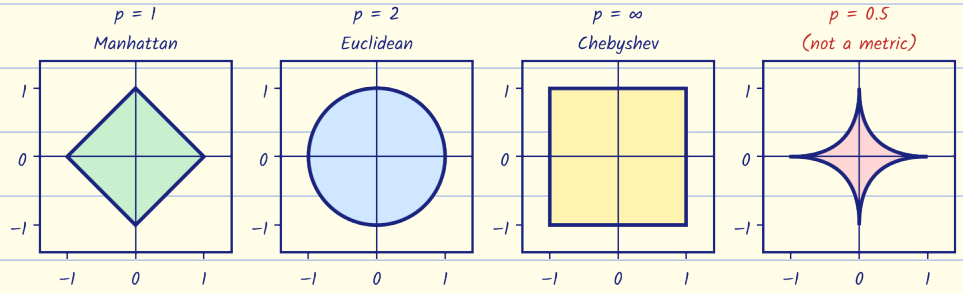


Fig 2.2 : Unit balls (points at distance 1 from origin) for different p

- **Mahalanobis** example: mean 0, Σ = [[4,0],[0,1]]. For x = (2,0): $d^2 = 2^2/4 = 1$, so d = 1. For y = (0,2): $d^2 = 2^2/1 = 4$, so d = 2. Both have Euclidean distance 2, but y is further from the data cloud because feature 2 varies less. If Σ = I, Mahalanobis = Euclidean.

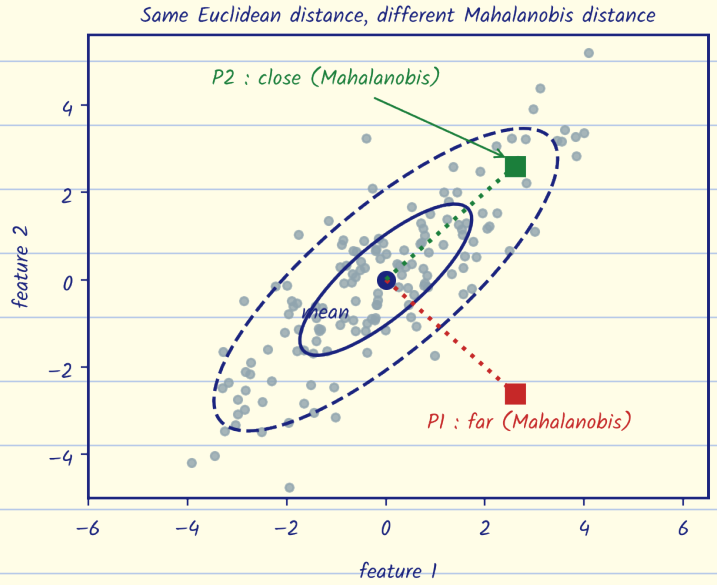


Fig 2.3 : Mahalanobis distance follows the shape of the data

> * **Scaling matters:**
> A feature with a large range (salary) can dominate Euclidean distance. **Always scale** features first: min-max $x' = (x - min)/(max - min)$ or z-score $z = (x - \mu)/\sigma$.

Person	Age	Salary (Rs)
q	30	50,000
A	60	50,100
B	31	60,000

Unscaled: d(q,A) = 104.4

d(q,B) = 10,000.0

Salary swamps age: A looks 'nearest' although 30 yrs older

After min-max scaling (0 to 1):

range age 30-60, salary 50k-60k

q = (0, 0)

A = (1, 0.01) d = 1.00

B = (0.03, 1) d = 1.00

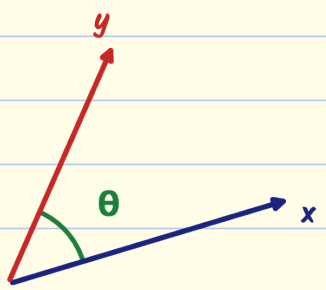
Now both features count equally, **so scale BEFORE using k-NN.**

Fig 2.4 : Without scaling the large-range feature decides the neighbor

2.3 Non-Metric Similarity Functions

Many useful measures **break one or more metric rules** (usually the triangle inequality or the identity rule). They are called **non-metric**. They still work well for matching, but tricks that need a true metric (e.g. some fast search trees) may not apply.

Function	Formula	Why it is non-metric
Cosine similarity	cos θ = x·y / (|x| |y|)	a similarity; 1 - cos θ fails the triangle inequality; x and 2x give 1 (identity fails)
KL divergence	KL(P||Q) = Σ P(i) ln(P(i)/Q(i))	not symmetric: KL(P||Q) ≠ KL(Q||P)
Squared Euclidean	Σ (xi - yi)²	triangle inequality fails (see figure)
Pearson correlation	r = cov(x,y) / (σx σy)	similarity in [-1, 1]; 1 - r not a metric
Fractional Lp, p < 1	(Σ |xi - yi|^p)^(1/p)	unit ball is not convex; triangle fails
Dynamic time warping	best alignment cost of two series	triangle inequality can fail



cos θ = (x·y) / (|x| |y|)

θ small → cos θ ≈ 1

θ = 90° → cos θ = 0

Only direction matters, not length.

Fig 2.5 : Cosine similarity measures the angle, not the length

- **Cosine example:** x = (1,2,0), y = (2,1,1). x·y = 2 + 2 + 0 = 4; |x| = sqrt5 = 2.236; |y| = sqrt6 = 2.449. cos θ = 4 / 5.477 = **0.730**. Cosine distance = 1 - 0.730 = **0.270**. Used for **text documents** (word-count vectors), where length is irrelevant.
- **KL example:** P = (0.5, 0.3, 0.2), Q = (0.8, 0.1, 0.1). KL(P||Q) = **0.233** but KL(Q||P) = **0.197**. Different, so it is **not symmetric**. Symmetric fix: Jensen-Shannon divergence.

4 > 1 + 1 : triangle inequality FAILS

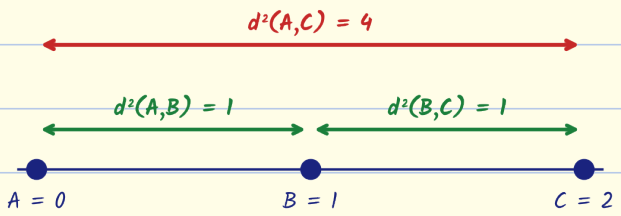


Fig 2.6 : Squared Euclidean distance violates the triangle inequality

2.4 Proximity Between Binary Patterns

For patterns of 0/1 features (present / absent) we count matches and mismatches. For two binary vectors X, Y of length n form the **2 x 2 contingency table**:

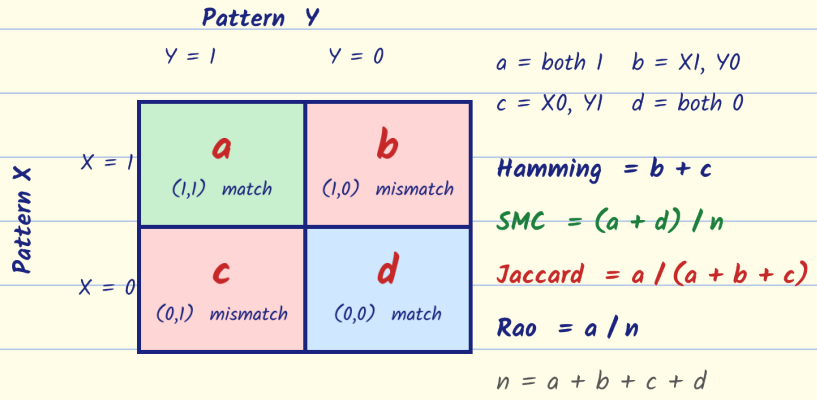


Fig 2.7 : Contingency table for binary patterns X and Y

- **a** = positions where both are 1, **b** = X is 1, Y is 0, **c** = X is 0, Y is 1, **d** = both 0. n = a + b + c + d.

Measure	Formula	Use
Hamming distance	b + c	number of differing bits
Simple Matching Coefficient (SMC)	(a + d) / n	symmetric binary data (0 and 1 equally important)
Jaccard coefficient	a / (a + b + c)	asymmetric data: 0-0 matches ignored (e.g. items bought)
Rao coefficient	a / n	counts only common 1s, relative to all positions
Jaccard distance	1 - a/(a + b + c)	a metric

- **Worked example:** X = 1 1 0 1 0 0 1 0 and Y = 1 0 0 1 1 0 0 0. Counting: a = 2, b = 2, c = 1, d = 3, n = 8.
- Hamming = b + c = **3**. SMC = (a + d)/n = 5/8 = **0.625**. Jaccard = a/(a+b+c) = 2/5 = **0.400**. Rao = a/n = 2/8 = **0.250**.
- **SMC vs Jaccard:** with many zeros (sparse data) SMC is large just because of 0-0 matches; Jaccard avoids this.

Part II : Classification Using Distance

2.5 Classification Algorithms Based on Distance Measures

These classifiers label a new pattern using its **distance to training patterns**. No equation of a boundary is learned.

Algorithm	Idea	Note
Nearest Neighbor (NN, 1-NN)	give q the class of the single closest training pattern	simple; sensitive to noise; error ≤ 2 × Bayes error (Cover-Hart)
K-Nearest Neighbor (KNN)	majority vote of the k closest	k reduces noise effect
Weighted KNN	closer neighbors get larger vote, $w = 1/d^2$	fewer ties; smoother
Radius Distance NN	vote of all points within radius r	number of neighbors varies
Nearest Centroid (NMC)	class of the nearest class mean	very fast; works for compact classes
Prototype / condensed NN	keep only a few representative patterns	reduces storage and time

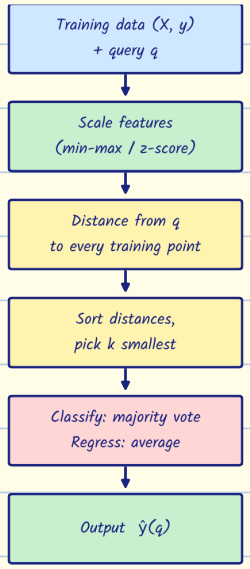


Fig 2.8 : Steps of the KNN algorithm

2.6 K-Nearest Neighbor (KNN) Classifier

Algorithm:

1. Choose k (odd for two classes) and a distance measure; scale the features.
2. For query q compute d(q, xi) to every training pattern xi.
3. Sort the distances and take the k smallest: the k nearest neighbors.
4. **Classify**: assign the **majority class** among them (ties: use smaller k or weights).

Worked example: classify q = (4, 3) using Euclidean distance.

Pattern	Features	Class	d to q = (4,3)	Rank
P1	(1, 1)	A	sqrt(13) = 3.61	6
P2	(2, 2)	A	sqrt(5) = 2.24	2
P3	(2, 5)	A	sqrt(8) = 2.83	3
P4	(5, 3)	B	sqrt(1) = 1.00	1
P5	(7, 3)	B	sqrt(9) = 3.00	4
P6	(5, 0)	B	sqrt(10) = 3.16	5

- **k = 1:** nearest is P4 → class **B**.
- **k = 3:** P4 (B), P2 (A), P3 (A) → A = 2, B = 1 → class **A**.
- **k = 5:** P4 (B), P2 (A), P3 (A), P5 (B), P6 (B) → B = 3, A = 2 → class **B**.
- **Lesson:** the answer **changes with k**, so k is a hyper-parameter that must be tuned.

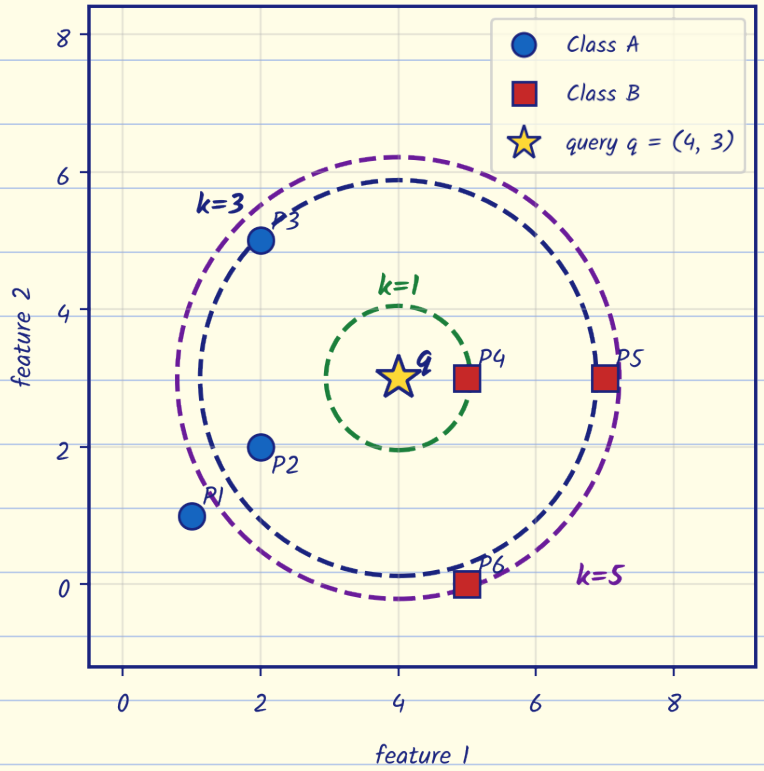


Fig 2.9 : Worked example: query q and its k = 1, 3, 5 neighborhoods

Choosing k

- **Small k** (k = 1): flexible, jagged boundary, follows noise → **over-fitting** (high variance). **Large k:** smooth boundary → **under-fitting** (high bias). k = n always predicts the majority class.
- **Method:** try k = 1, 3, 5, … and pick the k with the lowest **validation / cross-validation error**. Use odd k for 2 classes. Rule of thumb k ≈ sqrt(n).
- Training error at k = 1 is zero (each point is its own neighbor), so it must **never** be used to choose k.

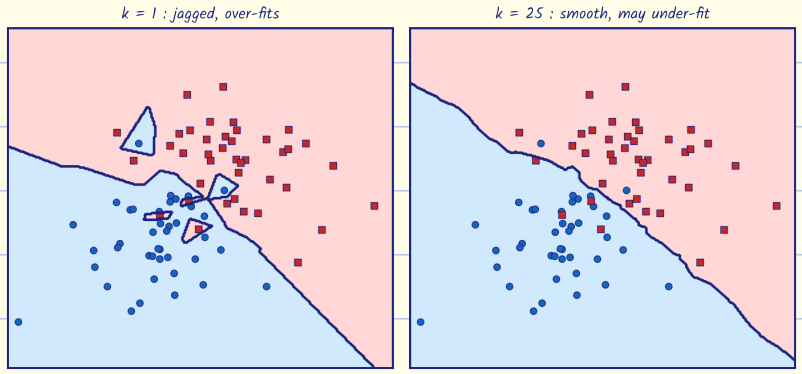


Fig 2.10 : Decision boundary for small and large k

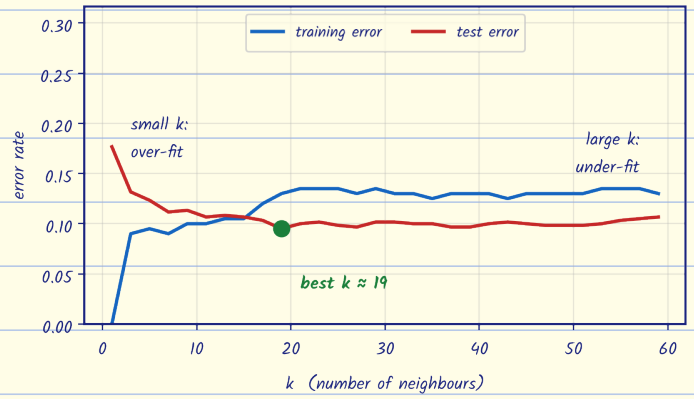


Fig 2.11 : Training and test error against k

Weighted KNN

Give each neighbor a **weight** that falls with distance, $w = 1/d^2$. For q and k = 3: P4 (B): $w = 1/1^2 = 1.000$; P2 (A): $w = 1/2.24^2 = 0.200$; P3 (A): $w = 1/2.83^2 = 0.125$. Votes: B = 1.000, A = 0.325 → class **B**, although plain 3-NN said A.

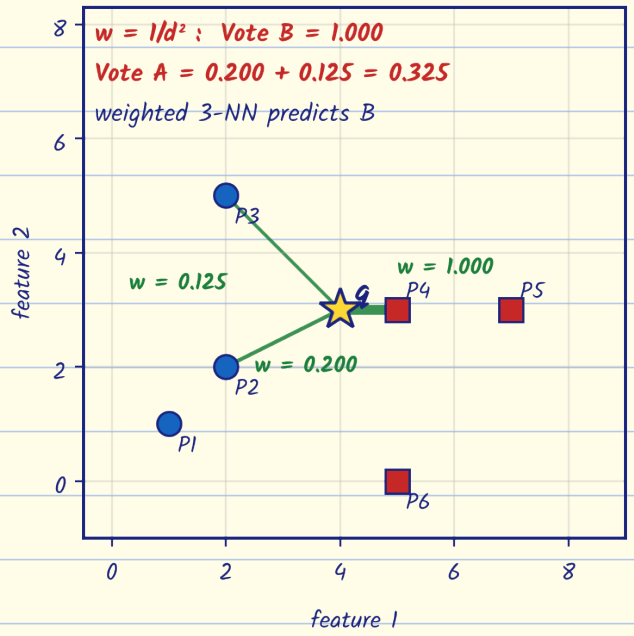


Fig 2.12 : Distance-weighted 3-NN vote

Curse of dimensionality, cost, pros and cons

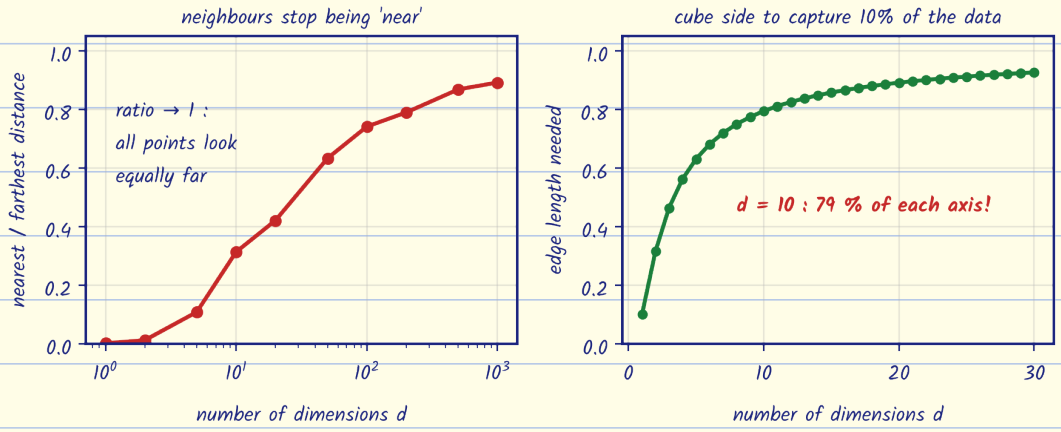


Fig 2.13 : In high dimensions neighbors are no longer 'near'

- **Curse of dimensionality:** as d grows, all points become almost equally far (nearest/farthest → 1) and one needs exponentially more data. Fix: **feature selection, PCA**, more data.
- **Cost:** prediction needs n distance computations of d numbers each → O(n·d) per query. Speed-ups: KD-tree, ball tree, condensed sets, approximate search.

Advantages	Disadvantages
Very simple; no training phase	Slow prediction and large memory (stores all data)
Naturally multi-class	Needs feature scaling; sensitive to irrelevant features
Non-linear boundaries, few assumptions	Poor in high dimensions; sensitive to noise for small k
Works for both classification and regression	Choice of k and distance measure needed

2.7 Radius Distance Nearest Neighbor Algorithm

Instead of a fixed count k, **Radius NN** (fixed-radius NN) uses **all training points within distance r** of the query, and takes the majority class among them.

1. Choose radius r (and distance measure).
2. Find $N(q) = \{ x_i : d(q, x_i) \le r \}$.
3. If N(q) is non-empty: predict the majority (or weighted) class in N(q).
4. If N(q) is empty: output 'outlier / no decision', or use the nearest neighbor, or default to the majority class.

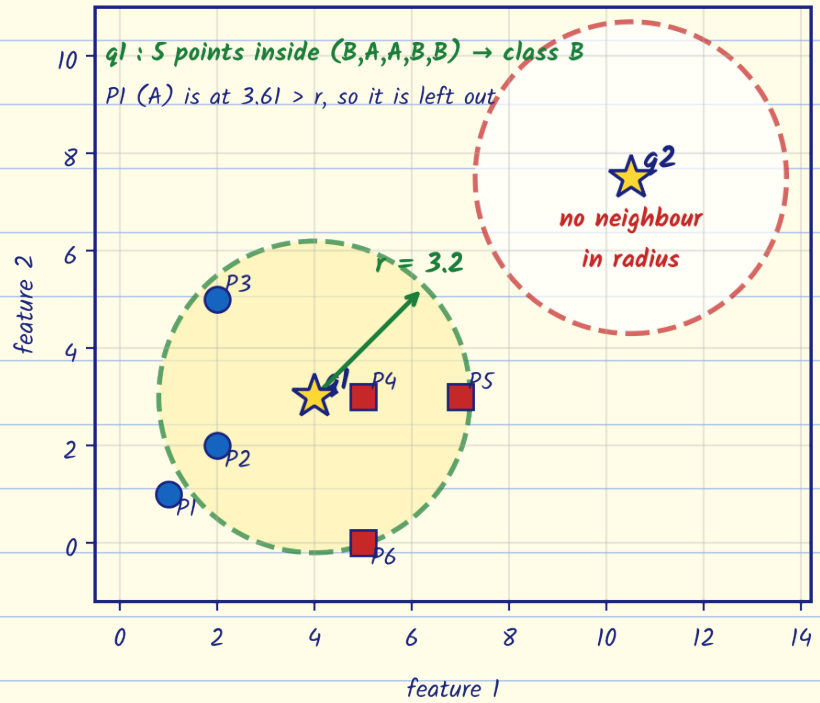


Fig 2.14 : Radius NN with r = 3.2: dense region and empty region

- **Example:** q1 = (4,3), r = 3.2. Distances: P4 1.00, P2 2.24, P3 2.83, P5 3.00, P6 3.16 are inside; P1 (3.61) is outside. Classes B, A, A, B, B → B = 3, A = 2 → **class B**. For q2 far away no point is inside, so it is flagged as an **outlier**.

- **KNN vs Radius NN:** KNN fixes the **number** of neighbors, radius NN fixes the **region**. In dense areas radius NN uses many points, in sparse areas few or none.
- **Choosing r:** too small → many empty neighborhoods; too large → includes other classes (like very large k). Tune r by cross-validation after scaling features.
- **Uses:** outlier / novelty detection, density-based decisions, range queries.

Part III : KNN Regression

2.8 KNN Regression

For a numeric target, the neighbors are found the same way but the prediction is the **average of the neighbors target values** instead of a vote:

Formulas
Plain: $\hat{y}(q) = (1/k) \Sigma\, y_i$ over the k nearest neighbors.
Weighted: $\hat{y}(q) = \Sigma\, w_i \cdot y_i \,/\, \Sigma\, w_i$, $w_i = 1 / d(q, x_i)^2$.

Worked example: training data below, predict y for $q = 4.4$ with $k = 3$.

x	1	2	3	4	5	6	7	8
y	2	3	5	6	6.5	9	10	12

- Distances $|x - 4.4|$: x=4 → 0.4, x=5 → 0.6, x=3 → 1.4 (next is x=6 → 1.6). Neighbors y = 6, 6.5, 5.
- **Plain average:** $(6 + 6.5 + 5)/3 = 17.5/3 = \mathbf{5.83}$.
- **Weighted:** weights $1/0.4^2 = 6.25$, $1/0.6^2 = 2.78$, $1/1.4^2 = 0.51$ → $\hat{y} = (6.25 \cdot 6 + 2.78 \cdot 6.5 + 0.51 \cdot 5) / 9.54 = \mathbf{6.09}$.

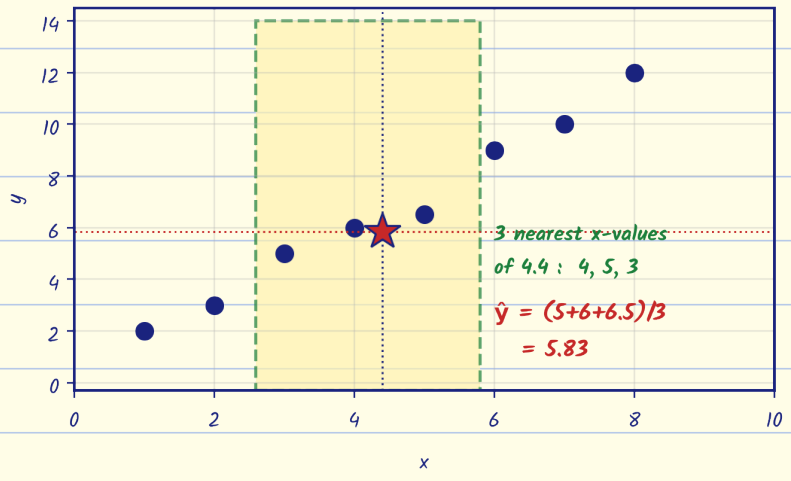


Fig 2.15 : KNN regression with k = 3

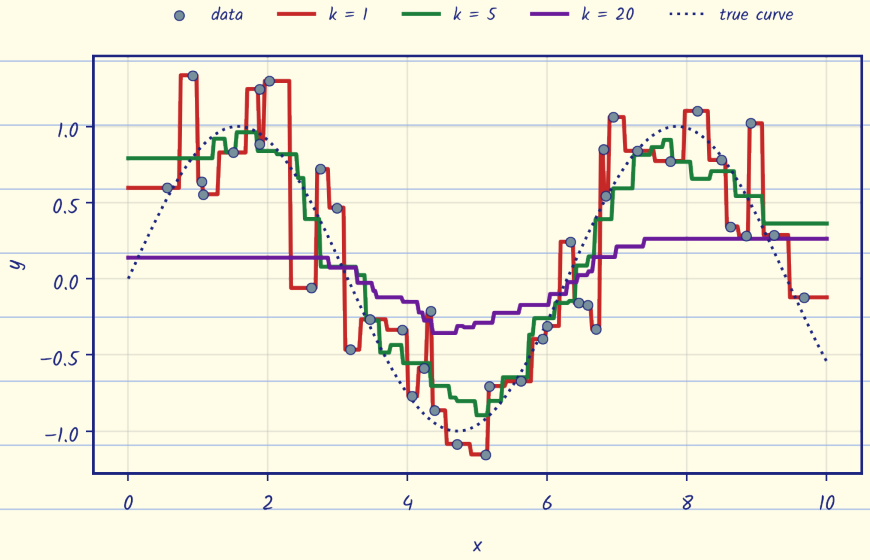


Fig 2.16 : Effect of k on the KNN regression curve

- **Small k**: wiggly curve through the noise (over-fit). **Large k**: flat, over-smooth curve (under-fit). Choose k by cross-validation on RMSE.
- KNN regression is **piecewise constant**, cannot extrapolate beyond the training range (it predicts the average of edge points). Scaling and the curse of dimensionality apply as before.
- Variants: **radius-based** regression (average within r), median instead of mean for outlier robustness.

Part IV : Performance Evaluation

2.9 Performance of Classifiers

Evaluate on **unseen test data**. For a two-class problem the **confusion matrix** counts the four outcomes: TP (positive predicted positive), TN, FP (false alarm), FN (missed positive).

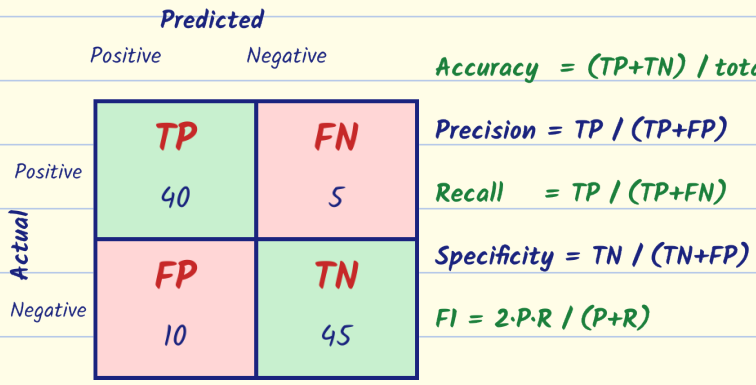


Fig 2.17 : Confusion matrix with TP = 40, FN = 5, FP = 10, TN = 45

Measure	Formula	Value here
Accuracy	(TP + TN) / (TP + TN + FP + FN)	85 / 100 = 0.85
Error rate	1 - accuracy = (FP + FN) / total	0.15
Precision	TP / (TP + FP)	40 / 50 = 0.80
Recall (sensitivity, TPR)	TP / (TP + FN)	40 / 45 = 0.889
Specificity (TNR)	TN / (TN + FP)	45 / 55 = 0.818
F1-score	2·P·R / (P + R) (harmonic mean)	0.842

- **Precision** = of all predicted positives, how many are right. **Recall** = of all real positives, how many were found. Spam filter: high precision. Cancer screening: high **recall**.
- **Imbalanced data:** 99 % negatives → 'always negative' gets 99 % accuracy but recall 0. Use precision, recall, F1.
- **Multi-class:** one confusion matrix of size C × C; per-class precision/recall, then **macro** average (mean of classes) or **micro** average (pool counts).

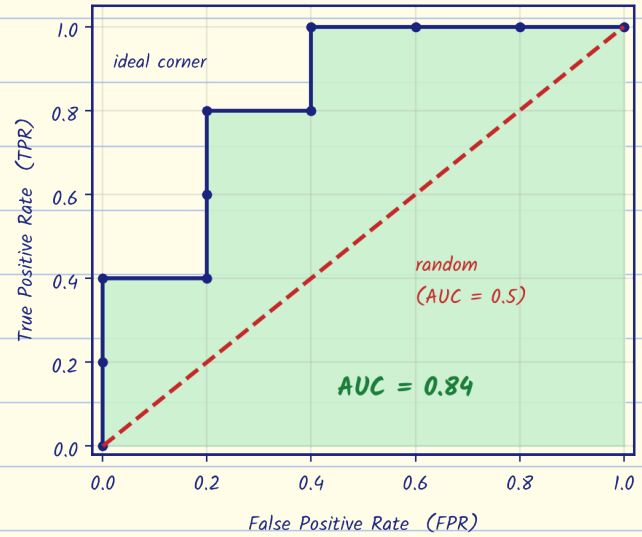


Fig 2.18 : ROC curve: TPR against FPR (AUC = 0.84 for the example)

- **ROC curve:** vary the decision threshold and plot TPR (y) against FPR = FP/(FP + TN) (x). **AUC** = area under it: 1 is perfect, 0.5 is random guessing. (KNN gives a score = fraction of the k neighbors in the class.)
- **Estimating performance:** **hold-out** (e.g. 70 / 30 split), **k-fold cross-validation** (below), **leave-one-out** (k = n), **stratified** splits keep class proportions.

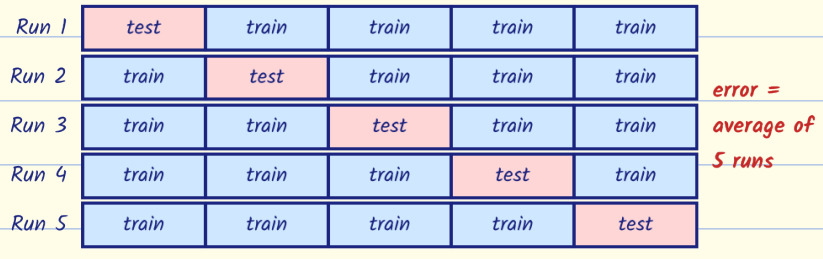


Fig 2.19 : 5-fold cross-validation

2.10 Performance of Regression Algorithms

Compare the predicted ŷ with the actual y. Error e = y - ŷ. Example: actual y = (3, 5, 2.5, 7), predicted ŷ = (2.5, 5, 4, 8) → errors e = (0.5, 0, -1.5, -1).

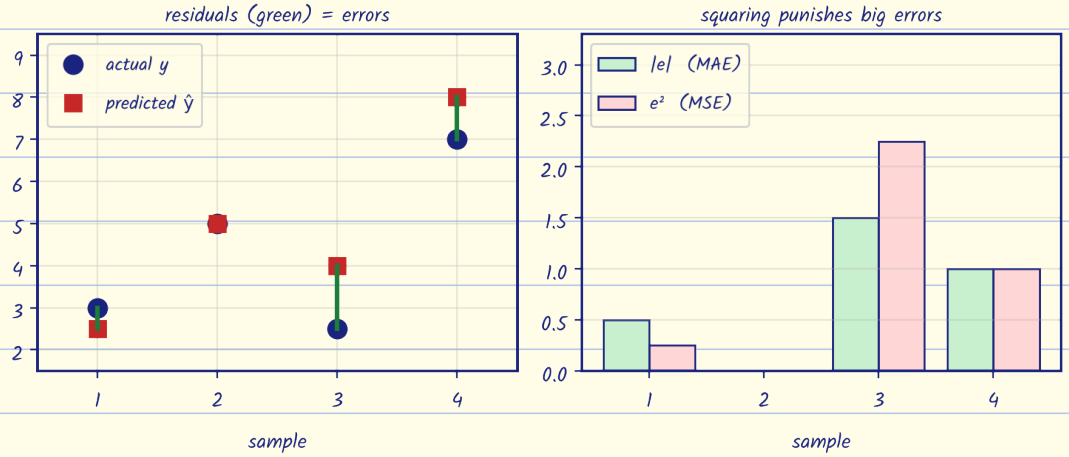


Fig 2.20 : Residuals, absolute error and squared error

Measure	Formula	Value here
MAE	(1/n) Σ |yi - ŷi|	3 / 4 = 0.75
MSE	(1/n) Σ (yi - ŷi)²	3.5 / 4 = 0.875
RMSE	sqrt(MSE) (same unit as y)	0.935
R² (coefficient of determination)	1 - SS_res / SS_tot	1 - 3.5/12.6875 = 0.724

- SS_res = Σ (y - ŷ)² = 0.25 + 0 + 2.25 + 1 = 3.5; mean of y = 4.375; SS_tot = Σ (y - ȳ)² = 12.6875.
- **MAE** treats all errors equally; **MSE/RMSE** punish large errors more (squares) and are more sensitive to outliers.
- **R²**: 1 = perfect, 0 = no better than always predicting the mean, can be negative for a bad model. Here the model explains about 72 % of the variance.

R² = 1 - SS_res / SS_tot (small green gaps vs big red gaps → R² near 1)

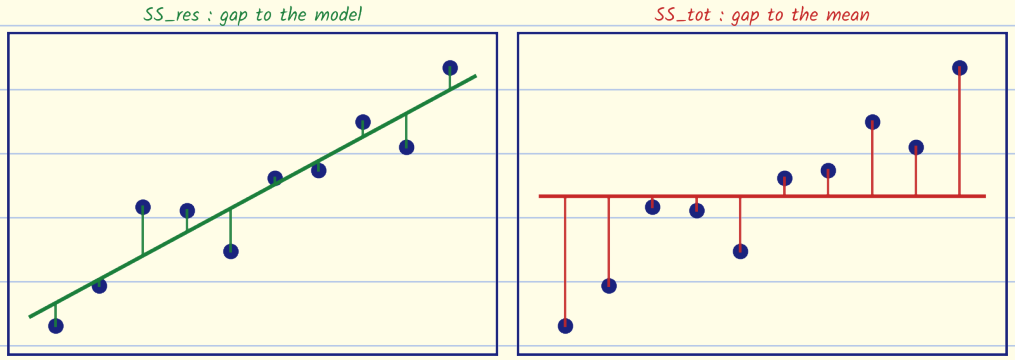


Fig 2.21 : R² compares the model error with the error of the mean

> * **Remember:**
> For a KNN model report errors on a TEST set (or cross-validation) only. On training data the k = 1 model has zero error and looks perfect.

Quick Revision - Unit II

- **Proximity** = similarity (high = alike) or distance (low = alike). KNN is **lazy / instance-based**: no training, all work at query time.
- **Metric**: non-negative, identity, symmetry, triangle inequality.
- **Distances:** Euclidean $\sqrt{\Sigma d^2}$, Manhattan $\Sigma|d|$, Chebyshev $\max|d|$, Minkowski $(\Sigma|d|^p)^{1/p}$, Mahalanobis uses covariance. Order: Cheb ≤ Euc ≤ Man.
- **Non-metric:** cosine, KL (asymmetric), squared Euclidean, fractional Lp, DTW.
- **Binary:** Hamming = b + c; SMC = (a + d)/n; Jaccard = a/(a + b + c); Rao = a/n.
- **KNN:** compute distances → k smallest → majority vote. Small k over-fits, large k under-fits; tune by cross-validation; scale features.
- **Weighted KNN:** $w = 1/d^2$. **Radius NN:** vote of all points within r; empty → outlier.
- **KNN regression:** mean (or weighted mean) of k neighbor targets.
- **Curse of dimensionality:** distances lose meaning in high d; reduce features.
- **Classifier metrics:** accuracy, precision TP/(TP+FP), recall TP/(TP+FN), F1, ROC-AUC, cross-validation.
- **Regression metrics:** MAE, MSE, RMSE, $R^2 = 1 - SS_{res}/SS_{tot}$.

Likely exam questions

1. Define proximity. Explain the properties of a metric and the common distance measures with an example.
2. Compare Euclidean, Manhattan, Minkowski, Chebyshev and Mahalanobis distances.
3. What are non-metric similarity functions? Explain cosine similarity and KL divergence with examples.
4. For binary patterns X = 11010010 and Y = 10011000 compute Hamming, SMC, Jaccard and Rao measures.
5. Explain the K-Nearest Neighbor algorithm. How does the value of k affect the result? How is k chosen?
6. Classify the query point using KNN (k = 1, 3, 5) for the given training table. Show the distance calculation.
7. Explain the radius distance nearest neighbor algorithm and differentiate it from KNN. Describe weighted KNN and KNN regression.
8. Define confusion matrix, accuracy, precision, recall and F1. List the performance measures for regression (MAE, MSE, RMSE, R^2) with an example.

MACHINE LEARNING

UNIT – III : Decision Trees and the Bayes Classifier

Easy digital notes with diagrams and worked examples for students (JNTU B.Tech, 23A31401T)

Topics in this unit

1. Decision trees for classification and impurity measures (entropy, Gini, error)
2. Properties of decision trees, overfitting and pruning
3. Regression based on decision trees
4. Bias-variance trade-off
5. Random forests for classification and regression
6. Bayes classifier: Bayes' rule, inference and optimality
7. Multi-class classification
8. Class conditional independence and the Naive Bayes classifier (NBC)

> * **Study tip:**
> Both halves of this unit use the same toy data - the 14-day Play-Tennis table. Do the ID3 and Naive Bayes calculations once by hand and every formula becomes easy.

Syllabus (as given)

Models Based on Decision Trees: Decision Trees for Classification, Impurity Measures, Properties, Regression Based on Decision Trees, Bias-Variance Trade-off, Random Forests for Classification and Regression. The Bayes Classifier: Introduction to the Bayes Classifier, Bayes' Rule and Inference, The Bayes Classifier and its Optimality, Multi-Class Classification, Class Conditional Independence and Naive Bayes Classifier (NBC).

Part I : Models Based on Decision Trees

3.1 Decision Trees for Classification

A **decision tree** is a flow-chart-like model that classifies an input by asking a **series of questions** about its features. Each question sends the input down one branch until it reaches a final answer.

- **Root node**: first test. **Internal node**: a test on one feature. **Branch**: outcome of the test. **Leaf node**: class label (the answer).
- **Prediction:** start at the root, follow the branch that matches the input, repeat, and output the label of the leaf reached.
- **Geometry:** every test like $x_j < t$ cuts the space with a line parallel to an axis, so the tree divides the feature space into **rectangular regions**, one class per region (Fig 3.1).

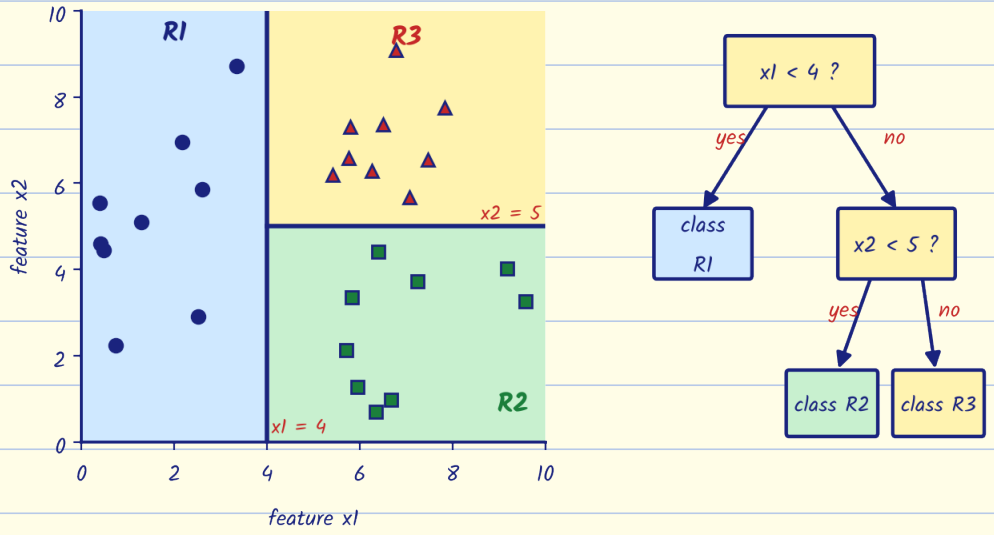


Fig 3.1 : A decision tree partitions the feature space into axis-parallel regions

Learning a tree: recursive partitioning (greedy, top-down)

1. Start with all training data at the root.
2. If the node is **pure** (one class) or a stopping rule fires, make it a **leaf** labelled with the **majority class**.
3. Otherwise try every feature (and threshold) and pick the split with the **largest impurity decrease** (best gain).
4. Split the data into child nodes and **repeat** the same steps on each child.

Algorithm	Split criterion	Features / output	Remarks
ID3 (Quinlan, 1986)	Information gain (entropy)	Categorical only; classification	Multi-way split; no pruning
C4.5	Gain ratio	Categorical + continuous; handles missing values	Post-pruning; successor of ID3
CART (Breiman, 1984)	Gini index (variance for regression)	Any features; classification + regression	Always **binary splits**; cost-complexity pruning

3.2 Impurity Measures

The best split is the one that makes the child nodes **as pure as possible**. An **impurity measure** $I(t)$ is 0 for a pure node and largest when classes are mixed equally. Let p_k = fraction of class k in node t.

Measure	Formula	Pure node	Max (2 classes)
Entropy	$H(t) = -\sum p_k \log_2 p_k$	0	1 ($\log_2 K$ for K classes)
Gini index	$Gini(t) = 1 - \sum p_k^2$	0	0.5 ($1 - 1/K$ for K classes)
Misclassification error	$E(t) = 1 - \max_k p_k$	0	0.5

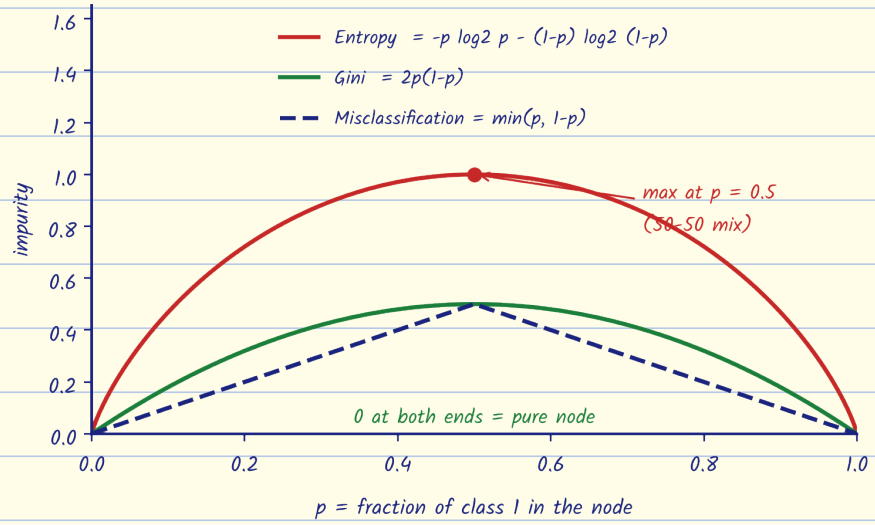


Fig 3.2 : Entropy, Gini and misclassification error for two classes

Node [Yes, No]	Entropy	Gini	Error
[7, 0]	0.000	0.000	0.000
[6, 1]	0.592	0.245	0.143
[9, 5]	0.940	0.459	0.357
[4, 3]	0.985	0.490	0.429
[5, 5]	1.000	0.500	0.500

- **Information gain** of splitting node S on feature A: $Gain(S, A) = H(S) - \Sigma_v (|S_v| / |S|) \cdot H(S_v)$ i.e. parent impurity minus **weighted** child impurity. Using Gini gives the **Gini decrease**.
- **Gain ratio** (C4.5): GainRatio = Gain / SplitInfo, $SplitInfo = - \Sigma_v (|S_v|/|S|) \log_2 (|S_v|/|S|)$. It punishes features with **many values** (e.g. an ID column has huge gain but is useless).
- **Misclassification error** is not smooth, so it is a poor choice for **growing** a tree, but it is common for **pruning**. Entropy and Gini are strictly concave, so they reward any purer child.

> * **Study tip:**
> Entropy and Gini give almost the same trees in practice. Gini is cheaper (no logarithm), entropy is slightly more sensitive to class mix.

3.3 Worked Example : Building a Tree with ID3

Decide **PlayTennis** from 4 features. Data set: **14 days, 9 Yes and 5 No.**

Day	Outlook	Temp	Humid.	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes

Day	Outlook	Temp	Humid.	Wind	Play
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Step 1 : entropy of the whole data set

$H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) =$ **0.940** bits. $Gini(S) = 1 - (9/14)^2 - (5/14)^2 =$ **0.459**. Error $= 5/14 = 0.357$.

Step 2 : gain of each feature at the root

Take **Outlook**: Sunny [2 Yes, 3 No], Overcast [4 Yes, 0 No], Rain [3 Yes, 2 No]. $H(Sunny) = H(Rain) = 0.971$, $H(Overcast) = 0$.

$Gain(S, Outlook) = 0.940 - (5/14)(0.971) - (4/14)(0) - (5/14)(0.971) = 0.940 - 0.694 =$ **0.247**

Feature	Gain (entropy)	Gini decrease	SplitInfo	Gain ratio
Outlook	0.247	0.116	1.577	0.156
Temp	0.029	0.019	1.557	0.019
Humidity	0.152	0.092	1.000	0.152
Wind	0.048	0.031	0.985	0.049

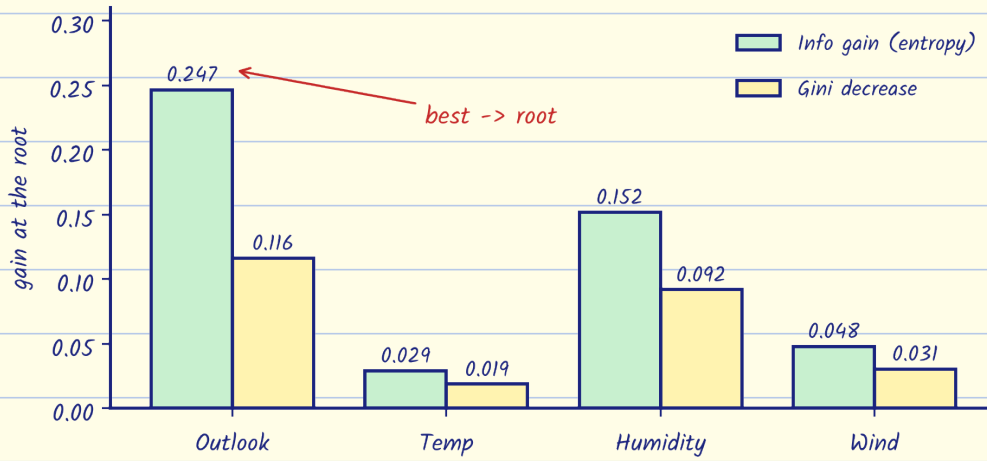


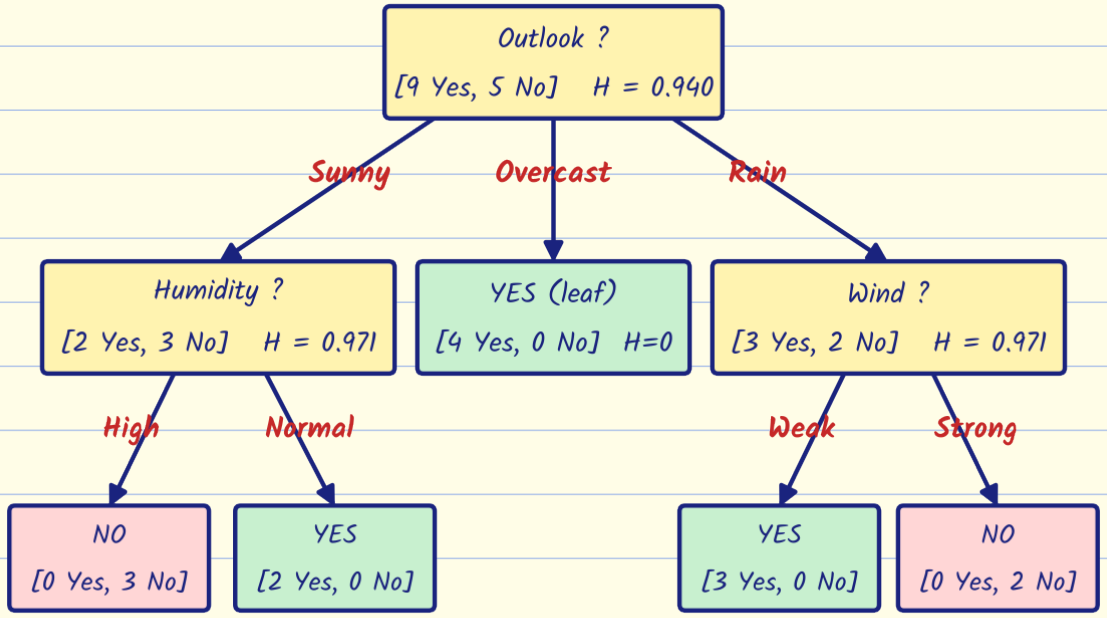
Fig 3.3 : Information gain and Gini decrease at the root: Outlook wins

Outlook has the highest gain (0.247) so it becomes the **root**. Overcast is already pure, so it becomes a leaf (Yes).

Step 3 : repeat on the impure branches

- **Sunny** [2 Yes, 3 No]: gains Temp = 0.571, Humidity = **0.971**, Wind = 0.020. Humidity separates perfectly (High -> No, Normal -> Yes).

- **Rain** [3 Yes, 2 No]: gains Temp = 0.020, Humidity = 0.020, Wind = **0.971**. Wind separates perfectly (Weak -> Yes, Strong -> No).
- All leaves are pure, so we **stop**. Temperature is never used - the tree has **selected features** automatically.



root = top node | internal node = a test | leaf = final answer (class)

Fig 3.4 : Final ID3 tree for Play-Tennis (5 pure leaves)

- **Rules read from the tree:** IF Overcast THEN Yes; IF Sunny AND Humidity = High THEN No; IF Sunny AND Humidity = Normal THEN Yes; IF Rain AND Wind = Weak THEN Yes; IF Rain AND Wind = Strong THEN No.
- **Predict** (Sunny, Cool, High, Strong): Outlook = Sunny -> Humidity = High -> **No**. Training error = 0.

3.4 Properties of Decision Trees

Property	Meaning
Non-parametric, interpretable	No fixed form assumed; the tree can be drawn and read as IF-THEN rules
Mixed data, little preparation	Handles numeric and categorical features; **no scaling needed**; invariant to monotonic transforms
Greedy search	Best split at each node only, so the tree is **not globally optimal** (finding the optimal tree is NP-complete)
Axis-parallel boundaries	Tilted boundaries need many small steps (staircase)
Automatic feature selection	Irrelevant features are simply never chosen for a split
High variance (unstable)	A small change in data can give a completely different tree
Overfitting	A fully grown tree reaches 100 % training accuracy by memorising noise
Bias to many-valued features	Information gain prefers features with many values - use **gain ratio**

Continuous features, missing values

- **Continuous feature:** sort the values, test thresholds at the **mid-points** between neighbouring values, and pick the best threshold t for the binary test $x < t$.
- **Missing values:** send the example down all branches with fractional weights (C4.5), or use surrogate splits (CART), or impute first.

Overfitting and pruning

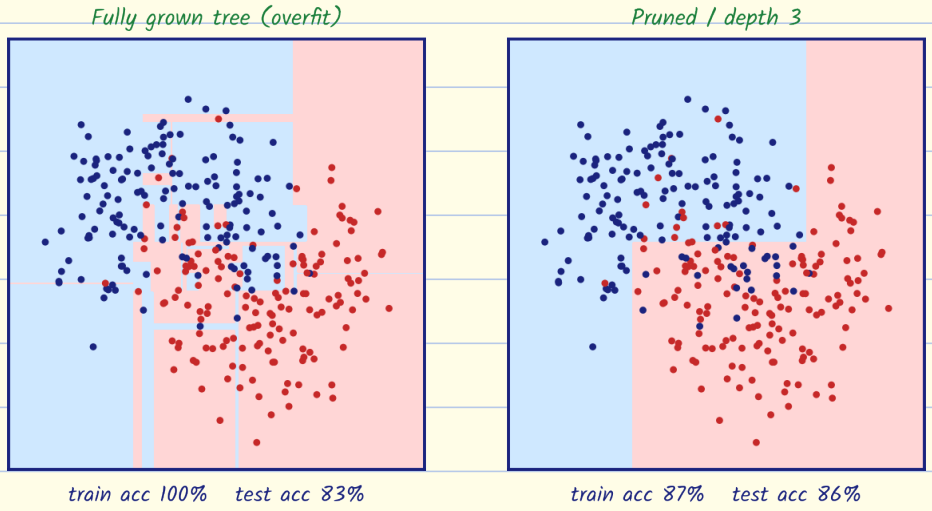


Fig 3.5 : Fully grown tree overfits; a shallower tree generalises better

- **Pre-pruning (early stopping):** stop growing when depth reaches a limit, a node has fewer than a minimum number of samples, or the best gain is below a threshold.
- **Post-pruning:** grow the full tree, then cut back subtrees that do not help on validation data. **Reduced-error pruning** replaces a subtree by a leaf if validation error does not rise.
- **Cost-complexity pruning (CART):** minimise $R_\alpha(T) = R(T) + \alpha\,|T|$, where $R(T)$ = training error and $|T|$ = number of leaves. Larger α gives a smaller tree; α is chosen by cross-validation.
- **Hyper-parameters:** max_depth, min_samples_leaf, min_impurity_decrease, ccp_alpha.

3.5 Regression Based on Decision Trees

When the target is a **real number**, the tree is a **regression tree** (CART). The leaf prediction is the **mean** of the training targets in that leaf, so the fitted function is a **step function**.

- **Impurity:** node variance, i.e. the **sum of squared errors** $SSE(t) = \Sigma\,(y_i - \bar{y}_t)^2$ with $\bar{y}_t$ = mean of y in t.
- **Best split** (feature j, threshold s) minimises SSE(left) + SSE(right), i.e. gives the largest **variance reduction**: $\Delta SSE = SSE(parent) - SSE(left) - SSE(right)$.
- **Prediction** for x falling in leaf R_m: $\hat{y}(x) = \bar{y}_{R_m} = (1/N_m)\,\Sigma_{x_i \in R_m}\, y_i$.
- **Stopping and pruning** are the same as for classification trees (depth, minimum samples, α-pruning with squared error).

Worked example : 8 points, one feature

x	1	2	3	4	5	6	7	8
y	4	5	4	6	15	16	14	15

SSE(parent) = 214.88 (mean = 9.875). Try thresholds: $x \le 1.5$: 175.43, $x \le 2.5$: 137.83, $x \le 3.5$: 67.47, $x \le 4.5$: 4.75, $x \le 5.5$: 88.80, $x \le 6.5$: 157.83, $x \le 7.5$: 184.86. Best is **$x \le 4.5$** with SSE = 2.75 + 2.00 = 4.75, so variance reduction = 214.88 - 4.75 = **210.12**.

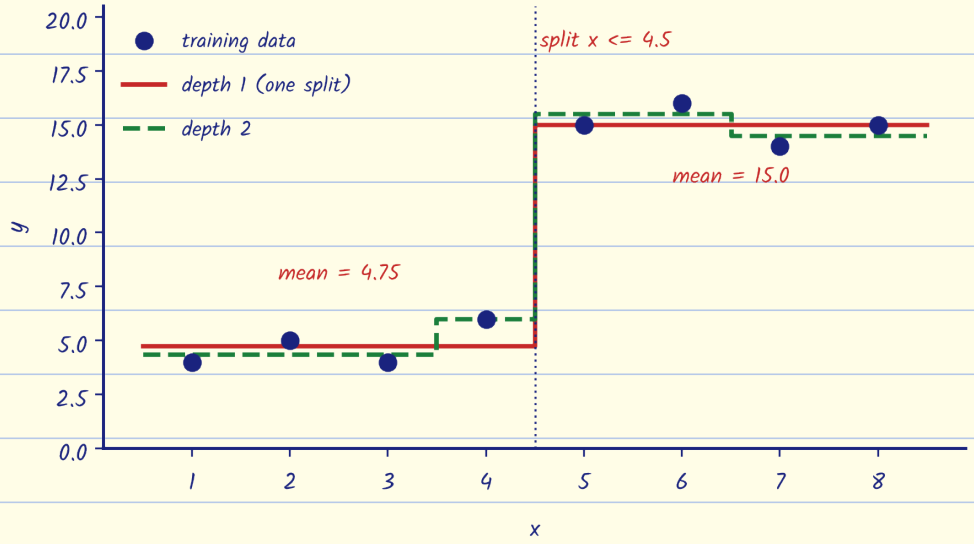


Fig 3.6 : Regression tree = step function; leaves predict means (4.75 and 15.0)

- The tree predicts **4.75** for $x \le 4.5$ and **15.0** for $x > 4.5$. Depth 2 adds more steps and follows the data more closely (lower bias, higher variance).
- **Limitations:** predictions are piecewise constant, cannot extrapolate beyond the training range, and the tree is unstable.

3.6 Bias-Variance Trade-off

Suppose $y = f(x) + \varepsilon$, with noise variance σ^2. A learner trained on a random training set gives a prediction $\hat{f}(x)$. Over many training sets, the expected squared error at x splits into three parts:

Bias-variance decomposition
$E[(y - \hat{f}(x))^2] =$ **Bias²** + **Variance** + σ^2
Bias = $E[\hat{f}(x)] - f(x)$ (systematic error: how far the average model is from the truth)
Variance = $E[(\hat{f}(x) - E[\hat{f}(x)])^2]$ (how much the model changes with the training set)
σ^2 = irreducible noise (no model can remove it).

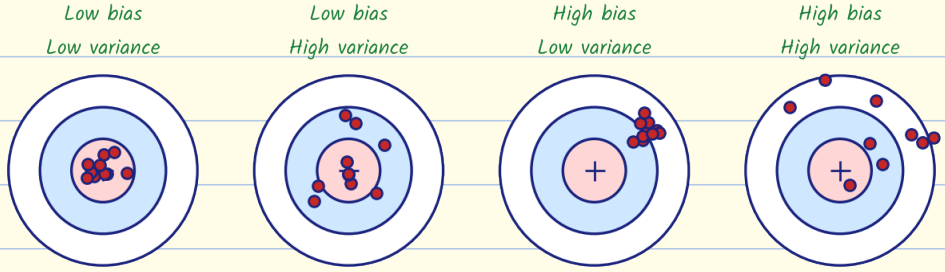


Fig 3.7 : Dartboard view: bias = distance of the cluster from the centre, variance = spread

	High bias (underfit)	High variance (overfit)
Model	Too simple (shallow tree, stump)	Too complex (deep tree)
Training error	High	Very low
Test error	High, close to training error	High; big gap from training error
Fix	Deeper tree, more features, more complex model	Prune, more data, **bagging / random forest**, regularisation

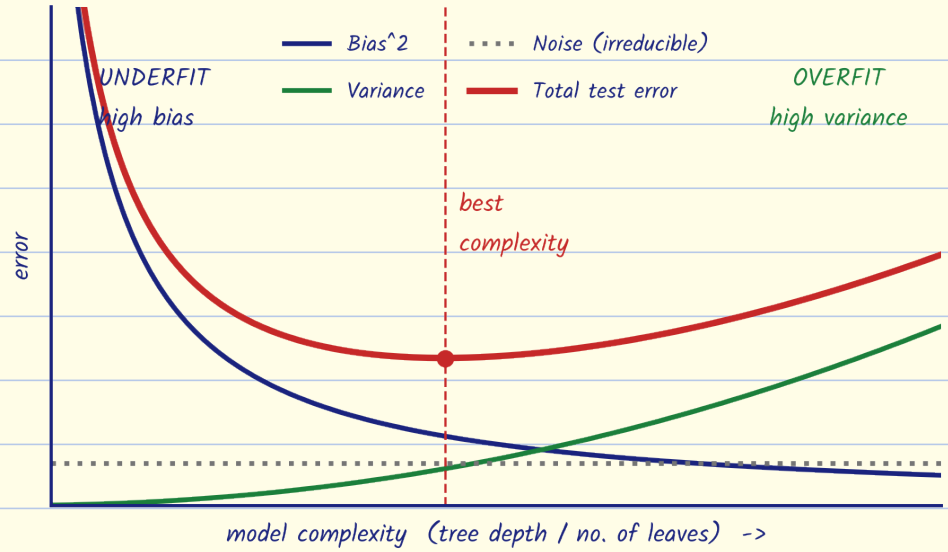


Fig 3.8 : Error against complexity: total error is lowest at the best depth

- **Trade-off:** a deeper tree **lowers bias** but **raises variance**; test error is U-shaped. The best depth is chosen on a validation set or by cross-validation.
- **Single decision tree** = low bias, **high variance**. Averaging many trees (random forest) cuts variance without raising bias much - this is the idea of the next section.

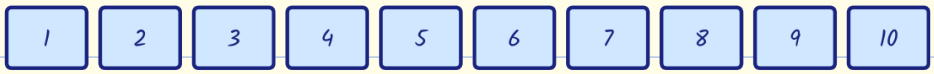
3.7 Random Forests for Classification and Regression

A **random forest** (Breiman, 2001) is an **ensemble** of many decision trees, each trained on a slightly different data set and a random choice of features. The trees' outputs are combined, which **reduces variance**.

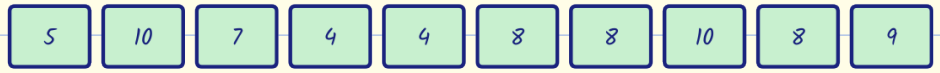
Step 1 : Bagging (bootstrap aggregating)

- Draw B **bootstrap samples**: each has N rows picked from the N training rows **with replacement**. A row is missed with probability $(1 - 1/N)^N \approx e^{-1} = 0.368$, so each sample holds about **63.2 %** of the distinct rows.
- Grow one tree on each sample - **deep and unpruned** (low bias, high variance).

Original data (10 rows):



One bootstrap sample (draw 10 WITH replacement):



Left out (out-of-bag, OOB) rows: 1, 2, 3, 6 -> used as a free test set

Fig 3.9 : A bootstrap sample and its out-of-bag (OOB) rows

Step 2 : Random feature subsets

- At **every split**, consider only m randomly chosen features out of d, and take the best of those. Typical: $m = \sqrt{d}$ for classification, $m = d/3$ for regression.
- **Why?** Without it, one strong feature sits at the top of every tree and the trees look alike (high correlation). Random subsets **de-correlate** the trees.

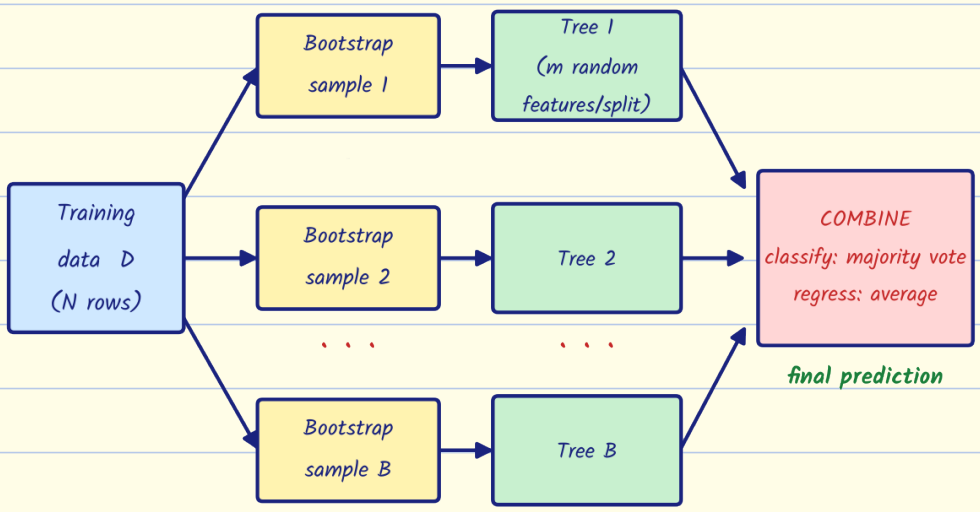


Fig 3.10 : Random forest: bootstrap, random-feature trees, combine

Step 3 : Combine

	Classification	Regression
Each tree outputs	A class label	A number
Forest output	Majority vote: $\hat{y} = \text{mode}\{ h_1(x), \ldots, h_B(x) \}$	Average: $\hat{y} = (1/B) \Sigma h_b(x)$
Tree split rule	Gini / entropy	Variance reduction (SSE)
Default m	$\sqrt{d}$	$d/3$

- **Why it works:** if the B trees have variance σ^2 and pairwise correlation ρ, the average has variance $\rho\sigma^2 + (1 - \rho)\sigma^2/B$. More trees kill the second term; random features lower ρ. Bias stays about the same as one deep tree.
- **Out-of-bag (OOB) error:** each tree never saw about 37 % of the rows. Predict each row using only the trees that left it out; the error over all rows is an **honest test-error estimate** - no separate validation set needed.

- **Feature importance:** (a) **mean decrease in impurity** - total impurity drop caused by splits on the feature, averaged over trees; (b) **permutation importance** - shuffle one feature and see how much OOB accuracy falls.

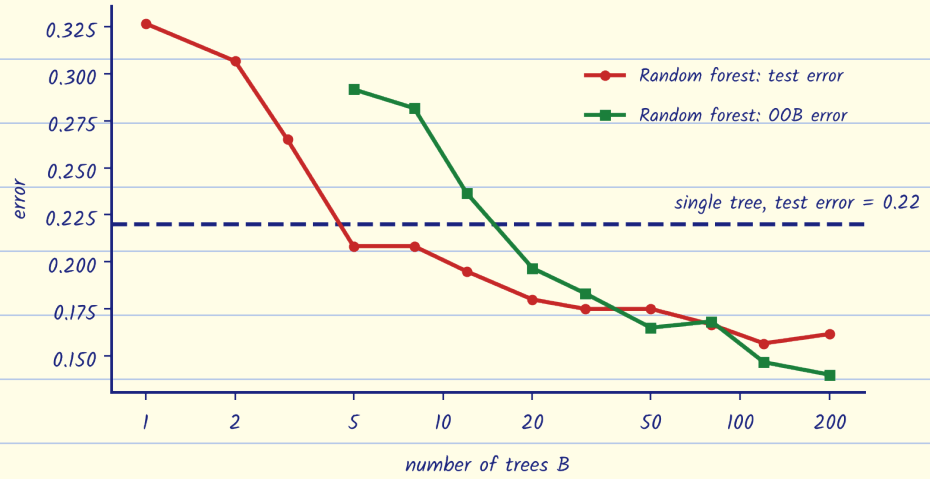


Fig 3.11 : Test and OOB error fall as trees are added and beat a single tree

- **More trees never cause overfitting** (error flattens); the main hyper-parameters are B, m, max_depth and min_samples_leaf.
- **Pros:** high accuracy, robust to noise and outliers, works on large data, gives importance. **Cons:** less interpretable than one tree, slower, large memory.

> * **Remember:**
> Tree: low bias, high variance. Random forest = many trees + bootstrap + random features + average/vote -> low bias, LOW variance.

Part II : The Bayes Classifier

3.8 Introduction to the Bayes Classifier

The Bayes classifier is a **probabilistic** classifier. For an input x it computes the **posterior probability** $P(C_k \mid x)$ of every class and picks the most probable class. Everything rests on a few probability rules.

Term	Meaning
Prior P(C)	Belief in class C before seeing any features (class proportion in the data)
Likelihood P(x | C)	How probable the features x are when the class is C
Evidence P(x)	$P(x) = \Sigma_k P(x \mid C_k) P(C_k)$ (total probability); same for every class
Posterior P(C | x)	Probability of class C after seeing x - this is what we want
Product rule	$P(A, B) = P(A \mid B) P(B) = P(B \mid A) P(A)$

3.9 Bayes' Rule and Inference

Bayes' rule
$P(C_k \mid x) = P(x \mid C_k) \cdot P(C_k) \,/\, P(x)$ posterior = likelihood × prior / evidence

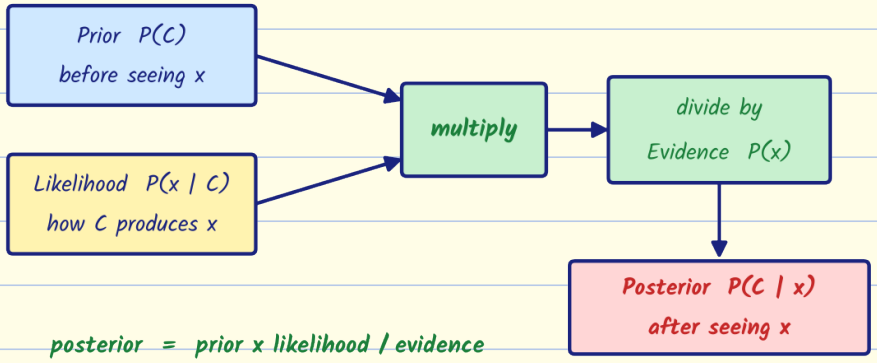


Fig 3.12 : Bayes' rule: prior and likelihood combine into the posterior

- **Inference** = updating belief: start with the prior, observe x, get the posterior (which becomes the new prior if more evidence arrives).
- **MAP (maximum a posteriori)** decision: $\hat{C} = \text{argmax}_k P(C_k \mid x) = \text{argmax}_k P(x \mid C_k) P(C_k)$. The evidence $P(x)$ is the same for all k, so it is **dropped** when only the winner is needed.
- **ML (maximum likelihood)** decision: $\text{argmax}_k P(x \mid C_k)$. It equals MAP when all priors are equal.

Worked example : medical test

Disease prevalence $P(D) = 0.01$. Test: $P(+ \mid D) = 0.95$, false-positive rate $P(+ \mid \text{no } D) = 0.05$. A patient tests positive. What is $P(D \mid +)$?

$P(+) = 0.95 \cdot 0.01 + 0.05 \cdot 0.99 = 0.0095 + 0.0495 = 0.0590$. $P(D \mid +) = 0.0095 \,/\, 0.0590 =$ **0.161**

* **Remember:**
Only about 16 % even after a positive test, because the disease is rare (low prior). Priors matter - do not ignore them.

3.10 The Bayes Classifier and its Optimality

The **Bayes classifier** assigns x to the class with the **largest posterior**:

* **Key idea:**
$f^*(x) = \text{argmax}_k P(C_k \mid x) = \text{argmax}_k P(C_k)\, p(x \mid C_k)$ (for continuous x use density $p(x \mid C_k)$)

- **Decision boundary:** the points where two posteriors are equal, e.g. $P(C_1)\, p(x|C_1) = P(C_2)\, p(x|C_2)$. In Fig 3.13 (equal priors, equal variance) it is the midpoint $x^* = 0$.
- **Error of any classifier h** at x: $P(\text{error} \mid x) = 1 - P(h(x) \mid x)$. This is smallest when $h(x)$ is the class with the **maximum** posterior. Hence for the Bayes classifier $P(\text{error} \mid x) =$ **$1 - \max_k P(C_k \mid x)$**.

Optimality theorem
Under 0-1 loss, no classifier (tree, neural network, anything) can have a smaller expected error than the Bayes classifier: $P(\text{error of } h) \geq P(\text{error of } f^*) = E_x[\, 1 - \max_k P(C_k \mid x)\,]$ = **Bayes error rate**.

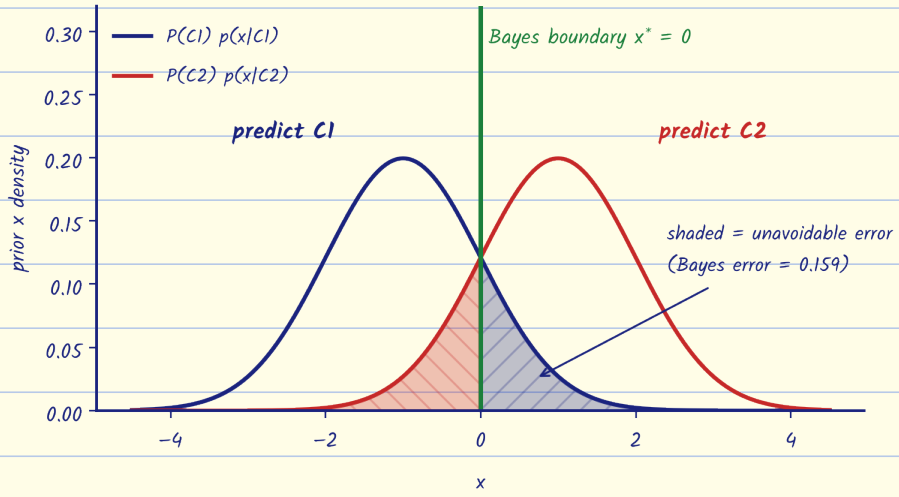


Fig 3.13 : Two classes with equal priors: Bayes boundary and the unavoidable (Bayes) error

- **Proof idea:** for fixed x, $P(\text{correct} \mid x) = P(h(x) \mid x) \le \max_k P(C_k \mid x)$, with equality when h picks the arg-max. Averaging over x keeps the inequality.
- **Bayes error rate** = the **irreducible** error caused by class overlap. Example in Fig 3.13: N(-1,1) vs N(+1,1), equal priors: error = $\Phi(-1)$ = **0.159** (the shaded area).
- **General loss:** with loss $\lambda(a_i \mid C_k)$ choose the action with the least **expected risk** $R(a_i \mid x) = \Sigma_k \lambda(a_i \mid C_k) P(C_k \mid x)$; 0-1 loss gives the MAP rule. (Useful when a missed cancer costs more than a false alarm.)
- **Practical catch:** the true $P(x \mid C_k)$ is unknown, so the Bayes classifier is a **theoretical benchmark**. Real methods estimate the probabilities - Naive Bayes does it with an independence assumption.

3.11 Multi-Class Classification

With K classes $C_1 \dots C_K$ the Bayes rule needs **no change**: compute K scores $g_k(x) = P(C_k)\, p(x \mid C_k)$ and choose the largest. The posteriors are the scores **normalised to sum to 1**: $P(C_k \mid x) = g_k(x) / \Sigma_j g_j(x)$.

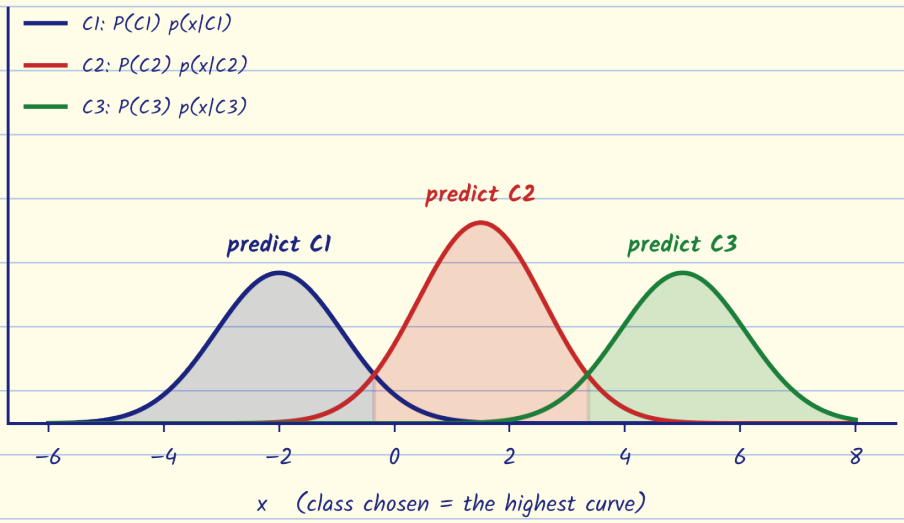


Fig 3.14 : Three classes: the class whose weighted density is highest wins; the line-up gives the decision regions

- **Decision regions** $R_k = \{x : g_k(x) \ge g_j(x) \text{ for all } j\}$; boundaries lie where the top two scores tie.
- **Error** = $1 - E[\max_k P(C_k \mid x)]$. It is easier to compute through the **correct** probability than by summing $K(K-1)$ misclassification terms.

- **Tree vs Bayes:** decision trees and random forests also handle K classes directly (leaf = majority class; Gini/entropy use all K proportions). Binary methods such as SVM need one-vs-rest or one-vs-one; the Bayes classifier does not.
- **Evaluation:** use a K × K confusion matrix; accuracy = diagonal sum / total.

3.12 Class Conditional Independence and the Naive Bayes Classifier

For d features, estimating the full likelihood $P(x_1, ..., x_d \mid C)$ needs a table of size $2^d - 1$ per class (binary features) - impossible for large d. **Naive Bayes** makes one simple assumption:

Class conditional independence
Given the class, the features are independent: $P(x_1, x_2, ..., x_d \mid C) = \Pi_{j=1..d} P(x_j \mid C)$

General (features depend on each other) Naive Bayes (conditionally independent)

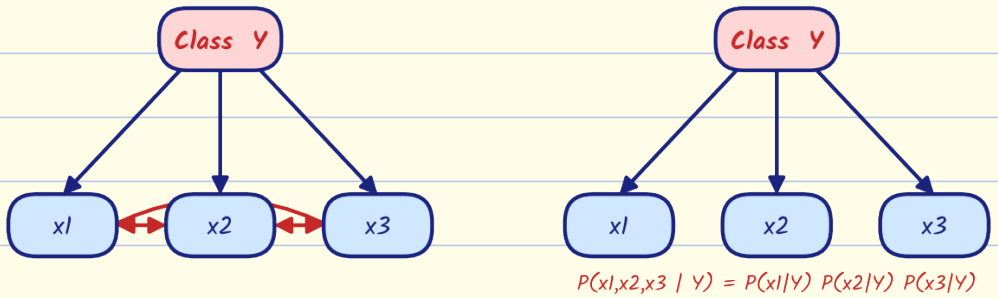


Fig 3.15 : Naive Bayes: the class alone explains each feature; no arrows between features

Naive Bayes classifier (NBC)
$\hat{C} = \text{argmax}_k \, P(C_k) \cdot \Pi_{j=1..d} P(x_j \mid C_k)$

- **Training** = just **counting**: $P(C_k) = N_k/N$ and $P(x_j = v \mid C_k) = \text{count}(x_j = v, C_k) / N_k$. One pass over the data; parameters grow as d per class, not 2^d.
- **Why "naive"?** Features are rarely truly independent (humidity and temperature), yet the **arg-max is often still right** because only the ranking of classes matters, not exact probabilities.
- **Use** small products as **log-sums**: $\log P(C_k) + \Sigma \log P(x_j \mid C_k)$ to avoid numerical underflow.

Worked example 1 : Play-Tennis, x = (Sunny, Cool, High, Strong)

Quantity	Class = Yes (9 rows)	Class = No (5 rows)
Prior P(C)	9/14	5/14
P(Sunny | C)	2/9	3/5
P(Cool | C)	3/9	1/5
P(High | C)	3/9	4/5
P(Strong | C)	3/9	3/5
Score = product	0.00529	0.02057
Posterior (normalised)	0.205	**0.795**

Yes: (9/14)(2/9)(3/9)(3/9)(3/9) = 0.00529. No: (5/14)(3/5)(1/5)(4/5)(3/5) = 0.02057. The larger is **No**, so the prediction is **do not play** (posterior 0.80); same answer as the decision tree.

The zero-frequency problem and Laplace smoothing

New day x = (**Overcast**, Cool, High, Strong). In the data, Overcast never occurs with No, so P(Overcast | No) = 0/5 = 0 and the **whole product** for No becomes 0, however strong the other evidence is.

Laplace (add-one) smoothing: $P(x_j = v \mid C) = (\text{count}(v, C) + \alpha) / (N_C + \alpha \cdot V_j)$, $\alpha = 1$, V_j = number of values of feature j (Outlook 3, Temp 3, Humidity 2, Wind 2). Prior is left unsmoothed here.

Factor	Without smoothing	With Laplace (α = 1)
Yes: P(Over|Yes), P(Cool|Yes), P(High|Yes), P(Strong|Yes)	4/9, 3/9, 3/9, 3/9	5/12, 1/3, 4/11, 4/11
No: P(Over|No), P(Cool|No), P(High|No), P(Strong|No)	**0/5**, 1/5, 4/5, 3/5	1/8, 1/4, 5/7, 4/7
Score Yes	0.0106	0.0118
Score No	**0**	0.0046
Posterior Yes / No	1.00 / 0.00	0.722 / 0.278

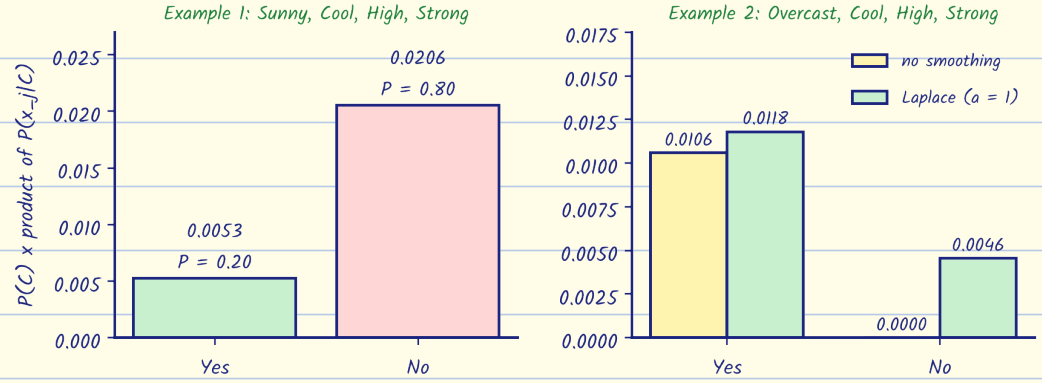


Fig 3.16 : NBC scores: Example 1 (left) and the zero-frequency case with and without smoothing (right)

- Without smoothing the model says Yes with **100 % certainty** just because one count is 0 - over-confident. With smoothing the zero disappears and the posterior is a sensible **0.72** for Yes; the decision (Yes) is the same.
- Larger α pulls every estimate toward uniform; $\alpha \to 0$ gives the raw frequencies. α is a hyper-parameter.

Naive Bayes for continuous features : Gaussian NB

For a numeric feature, assume $P(x_j \mid C_k)$ is a **Gaussian** whose mean μ_{jk} and variance σ_{jk}^2 are estimated from the class-k training values: $p(x_j \mid C_k) = (1 / \sqrt{(2\pi\ \sigma^2)}) \cdot \exp(- (x_j - \mu)^2 / (2\sigma^2))$.

Example: temperature of Yes days = 70, 72, 74 ($\mu = 72$, $\sigma^2 = 2.67$); No days = 80, 85, 90 ($\mu = 85$, $\sigma^2 = 16.67$). For x = 75: p(75 | Yes) = 0.0452, p(75 | No) = 0.0049. With equal priors the ratio is 9.3 : 1, so predict **Yes**.

Feature type	Model for $P(x_j \mid C)$	Variant
Categorical / binary	Counts with Laplace smoothing	Multinomial / Bernoulli NB (text, spam)
Continuous	Gaussian $N(\mu, \sigma^2)$ per class	Gaussian NB

Advantages of NBC	Limitations of NBC
Very fast training and prediction; one pass of counting	Independence assumption is unrealistic; correlated features are double-counted
Works with small data and many features (text, spam filtering)	Probability outputs are poorly calibrated (over-confident); use only the ranking
Handles missing features (skip the factor); easy to update	Zero-frequency needs smoothing; cannot learn feature interactions

	Decision tree / forest	Naive Bayes
Type	Discriminative: learns the boundary directly	Generative: models $P(x \mid C)$ and $P(C)$, then applies Bayes' rule
Feature interactions	Captured by successive splits	Ignored (independence)
Data needed	Moderate to large	Small is enough
Interpretability	Rules (tree); low for forest	Per-feature probabilities

Quick Revision – Unit III

- **Decision tree:** root, internal nodes (tests), branches, leaves (class). Built greedily by recursive partitioning; axis-parallel regions.
- **Impurity:** Entropy $= -\Sigma\, p \log_2 p$; Gini $= 1 - \Sigma\, p^2$; Error $= 1 - \max p$. All 0 for a pure node, maximum at an even mix.
- **Gain** = H(parent) - weighted H(children); **gain ratio** = Gain / SplitInfo. ID3 (entropy), C4.5 (gain ratio), CART (Gini, binary).
- **Play-Tennis:** $H(S) = 0.940$; Gain(Outlook) = 0.247 (root), Humidity = 0.152, Wind = 0.048, Temp = 0.029.
- **Properties:** interpretable, no scaling, greedy, unstable, overfits. **Pruning:** pre (depth, min samples), post (reduced-error, cost-complexity $R(T) + \alpha|T|$).
- **Regression tree:** leaf = mean; split minimises SSE (variance reduction); piecewise-constant, no extrapolation.
- **Bias-variance:** $E[\text{error}] = \text{Bias}^2 + \text{Variance} + \sigma^2$. Complexity up -> bias down, variance up; test error is U-shaped.
- **Random forest:** bagging (bootstrap, 63.2 % rows) + random m features per split ($\sqrt{d}$ / $d/3$); vote or average; OOB error; feature importance; $\text{Var} = \rho\sigma^2 + (1 - \rho)\sigma^2/B$.
- **Bayes rule:** posterior = likelihood × prior / evidence. **MAP:** argmax $P(x|C)P(C)$.
- **Bayes classifier:** $f^*(x) = \text{argmax}\, P(C_k \mid x)$; **optimal** under 0-1 loss; Bayes error $= E[1 - \max P(C_k \mid x)]$ (irreducible).

- **Multi-class:** same rule with K scores, normalised to give posteriors.
- **Naive Bayes:** class-conditional independence, $P(x|C) = \prod P(x_j|C)$; train by counting; **Laplace** (count + 1)/(N_C + V) fixes zero frequency; **Gaussian NB** for numeric features.

Likely exam questions

1. What is a decision tree? Explain how a decision tree is constructed for classification. Compare ID3, C4.5 and CART.
2. Define entropy, Gini index and misclassification error. Compute the information gain of Outlook for the Play-Tennis data.
3. Construct a decision tree for a given small data set using information gain (show every gain calculation).
4. List the properties of decision trees. What is overfitting? Explain pre-pruning and post-pruning.
5. Explain regression trees. How is the best split chosen? Work out a split for a small numeric data set.
6. Explain the bias-variance trade-off and the decomposition of expected error with a diagram.
7. Explain random forests: bagging, random feature selection, OOB error and feature importance. Why does a forest have lower variance than a tree?
8. State Bayes' rule. Show that the Bayes classifier is optimal and define the Bayes error rate. Explain Naive Bayes with Laplace smoothing and classify a given sample.

MACHINE LEARNING (23A31401T)

UNIT – IV : Linear Discriminants, SVM & MLP

Easy digital notes with diagrams for students

Topics in this unit

1. Introduction to Linear Discriminants (hyperplane, w.x + b)
2. Linear Discriminants for Classification
3. Perceptron Classifier
4. Perceptron Learning Algorithm (AND-gate example)
5. Support Vector Machines (maximum margin)
6. Linearly Non-Separable Case (soft margin)
7. Non-linear SVM
8. Kernel Trick (polynomial, RBF)
9. Logistic Regression
10. Linear Regression
11. Multi-Layer Perceptrons (MLPs)
12. Back propagation for training an MLP

> * **Study tip:**
> Every topic has a small diagram. Learn the RED words first, then draw the diagram once from memory - that is the fastest way to remember it for exams. Practise the three numeric examples (AND perceptron, regression line, one backprop step) twice.

Syllabus (as given)

Linear Discriminants for Machine Learning: **Introduction to Linear Discriminants, Linear Discriminants for Classification, Perceptron Classifier, Perceptron Learning Algorithm, Support Vector Machines, Linearly Non-Separable Case, Non-linear SVM, Kernel Trick, Logistic Regression, Linear Regression, Multi-Layer Perceptrons (MLPs), Back propagation for Training an MLP.**

Part I : Linear Discriminants & the Perceptron

4.1 Introduction to Linear Discriminants

A **linear discriminant** is a function that separates classes using a **straight boundary**. It is the simplest and fastest way to classify.

- Score: **$g(x) = w.x + b$** (w = weight vector, b = bias).
- In 2-D the boundary **$w.x + b = 0$** is a line; in 3-D a plane; in n-D a **hyperplane**.
- The weight vector **w is perpendicular (normal)** to the hyperplane.
- Sign of $g(x)$ tells the side: positive -> class +1, negative -> class -1.
- **b** shifts the line away from the origin; distance of the plane from origin = $|b| / ||w||$.

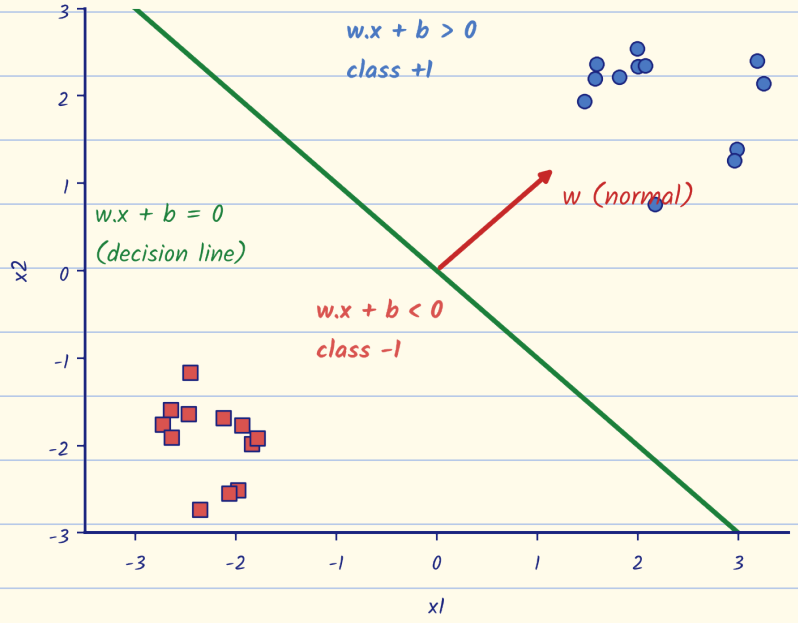


Fig 4.1 : Linear separator, its normal vector w and the two sides

> * **Remember:**
> Linear discriminant = **$w.x + b$**. Hyperplane = set of points where it equals 0. Distance of a point x from it = **$|w.x + b| / ||w||$**.

4.2 Linear Discriminants for Classification

- **Two classes:** predict +1 if $w.x + b > 0$, else -1. One hyperplane is enough.
- **K classes (one-vs-rest):** train K discriminants $g_k(x) = w_k.x + b_k$ and choose the class with the **largest score**.

- **One-vs-one:** train a classifier for every pair of classes (K(K-1)/2) and take a majority vote.
- Data that can be split by a hyperplane is called **linearly separable**.
- Ways to learn w and b: perceptron rule, SVM (max margin), logistic regression, least squares.

Method	Idea
Perceptron	Any separating line; updates only on mistakes
SVM	Best line = **widest margin**
Logistic regression	Gives a **probability** using sigmoid
Least squares	Minimise squared error of w.x + b

4.3 Perceptron Classifier

The **perceptron** (Rosenblatt, 1958) is the simplest artificial neuron: a weighted sum followed by a **step activation**.

- Net input: **net = w1x1 + w2x2 + ... + wnxn + b = w.x + b**.
- Output: **y = 1 if net > 0, else 0** (or +1 / -1).
- It is a **linear classifier** - it draws one hyperplane.
- **Limitation:** cannot solve **XOR** because XOR is not linearly separable. Fix: add hidden layers (MLP).

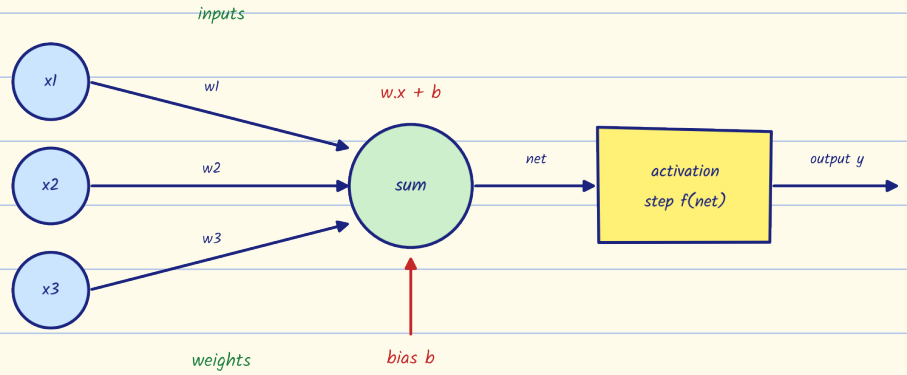


Fig 4.2 : Perceptron as a neuron (inputs, weights, sum, activation)

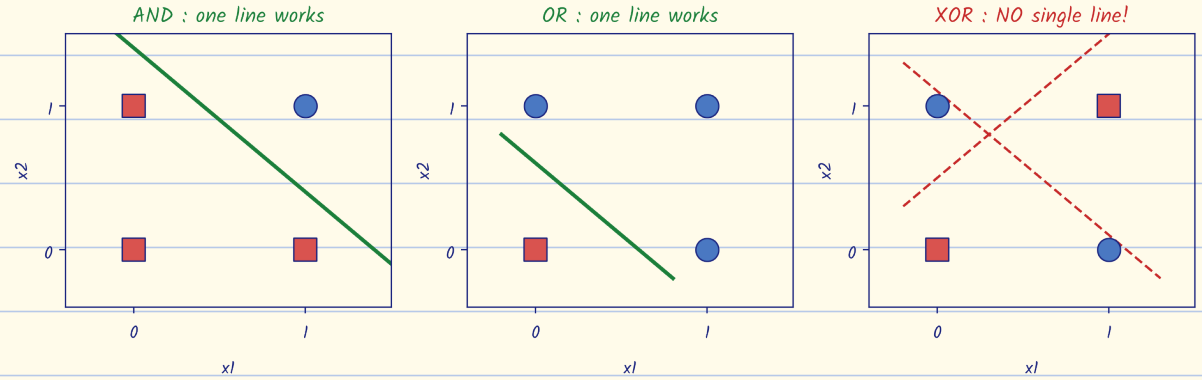


Fig 4.3 : AND and OR are separable by one line, XOR is not

4.4 Perceptron Learning Algorithm (PLA)

Start with small weights and **correct them only when a sample is misclassified**. Repeat over the data (each pass = **epoch**) till no errors.

1. Initialise w = 0, b = 0; choose learning rate **eta** (here 1).
2. For each sample (x, t): compute y = step(w.x + b).
3. Update: **w = w + eta (t - y) x** and **b = b + eta (t - y)**.
4. If y = t nothing changes. Stop after an epoch with **zero errors**.

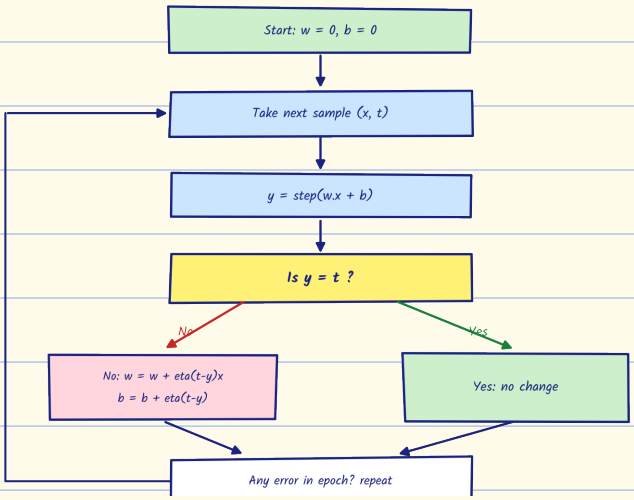


Fig 4.4 : Flowchart of the perceptron learning loop

* **Convergence theorem:**
If the data is **linearly separable**, PLA reaches a perfect separator in a **finite number of steps**. If not (like XOR), it never settles.

Worked example : learning the AND gate

Inputs (x1,x2), target t = x1 AND x2, eta = 1, start w1 = w2 = b = 0, rule: y = 1 if net > 0.

Epoch	(x1,x2)	t	net	y	t-y	new w1, w2, b
1	(0,0)	0	0	0	0	0, 0, 0
1	(0,1)	0	0	0	0	0, 0, 0
1	(1,0)	0	0	0	0	0, 0, 0
1	(1,1)	1	0	0	+1	1, 1, 1
2	(0,0)	0	1	1	-1	1, 1, 0
2	(0,1)	0	1	1	-1	1, 0, -1
2	(1,0)	0	0	0	0	1, 0, -1

Epoch	(x1,x2)	t	net	y	t-y	new w1, w2, b
2	(1,1)	1	0	0	+1	2, 1, 0
3	all 4 samples	-	-	-	3 errors	2, 1, -1
4	all 4 samples	-	-	-	2 errors	2, 2, -1
5	all 4 samples	-	-	-	1 error	2, 1, -2
6	all 4 samples	-	-	-	0 errors	2, 1, -2

Epoch 6 has **no errors**, so we stop: **w1 = 2, w2 = 1, b = -2**. Decision line: **2x1 + x2 - 2 = 0**. Check (1,1): net = 1 > 0 -> 1. Check (0,1): net = -1 -> 0.

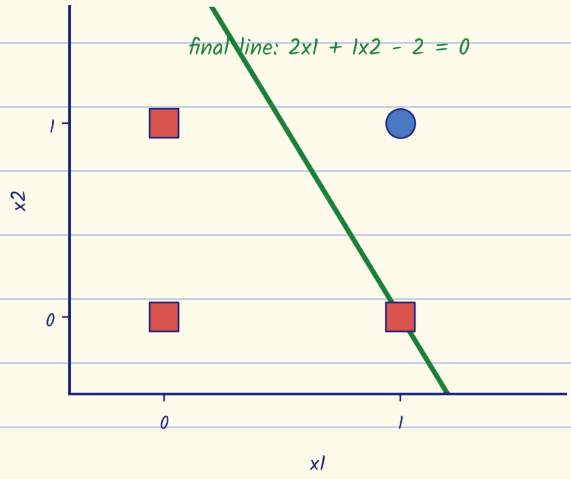


Fig 4.5 : Final AND decision line found by PLA

Part 2 : Support Vector Machines

4.5 Support Vector Machines (SVM)

Many lines can separate two classes. The **SVM chooses the best one - the line with the maximum margin** (largest gap to the nearest points). A wide gap means better generalisation.

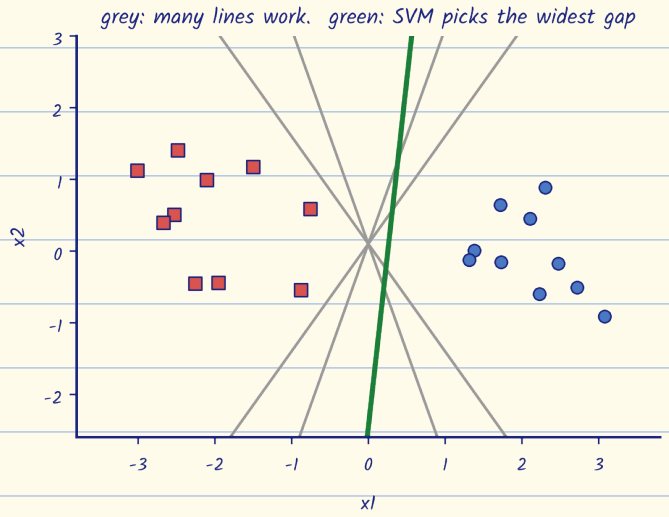


Fig 4.6 : Many separators exist, SVM picks the widest-gap one

- **Support vectors** = the training points **closest** to the boundary (on the margin lines). Only they decide the hyperplane; other points can be removed freely.
- Margin lines: **w.x + b = +1** and **w.x + b = -1**; boundary: w.x + b = 0.
- Constraint for every point: **t_i (w.x_i + b) >= 1** (t_i = +1 or -1).
- Width of the gap: **margin = 2 / ||w||**. So **maximise margin = minimise ||w||**.

> * **Optimisation problem (hard margin):**
> minimise **(1/2) ||w||^2** subject to **t_i (w.x_i + b) >= 1** for all i. It is a convex problem (quadratic programming) with a single global solution.

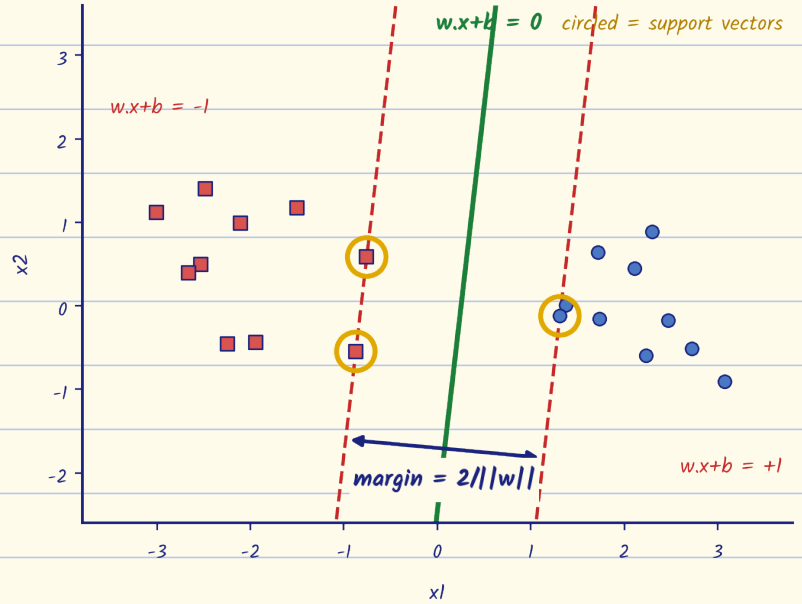


Fig 4.7 : Maximum margin, margin lines and support vectors (circled)

* **Example:**
If w = (3, 4) then ||w|| = sqrt(9 + 16) = 5, so **margin = 2/5 = 0.4**. A larger ||w|| means a thinner margin.

4.6 Linearly Non-Separable Case (Soft Margin)

Real data has noise and overlap, so a perfect line may not exist. The **soft-margin SVM** allows some points to break the rules, but charges a penalty.

- **Slack variable xi_i >= 0** = how much point i violates its margin (0 = fine, between 0 and 1 = inside margin, > 1 = misclassified).
- Constraint: **t_i (w.x_i + b) >= 1 - xi_i**.
- Objective: minimise **(1/2)||w||^2 + C * sum of xi_i**.
- **C** = trade-off knob between a wide margin and few errors.

C value	Effect
Large C	Few violations allowed, narrow margin, may **overfit**
Small C	More violations allowed, wide margin, may **underfit**

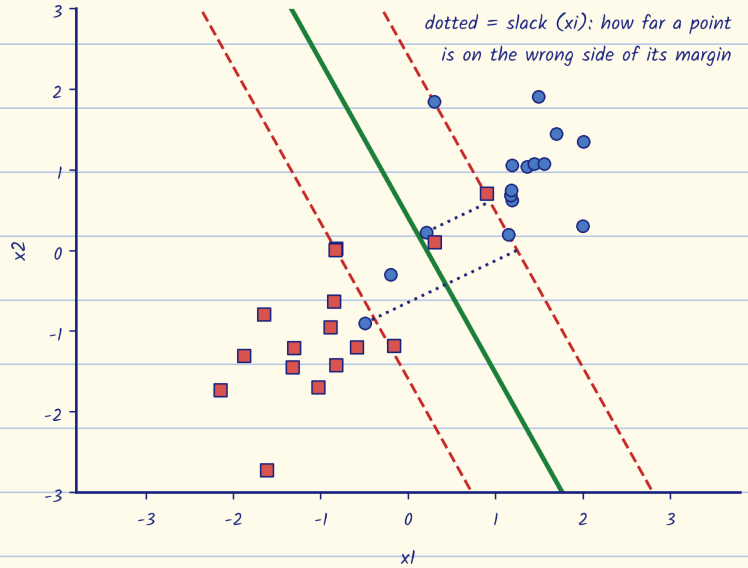


Fig 4.8 : Soft margin - slack (dotted) for points on the wrong side

4.7 Non-linear SVM

Some data (e.g. a ring of points around a blob) cannot be split by any line, even with slack. Idea: **map the data to a higher-dimensional space** where it becomes linearly separable, then use a linear SVM there.

- Feature map: **x -> phi(x)**. A linear boundary in phi-space is a **curved boundary** in the original space.
- Example: 2-D points (x1, x2) -> 3-D with extra feature **z = x1^2 + x2^2** (squared distance from centre).
- Inner circle has small z, outer ring has large z, so a **flat plane z = constant** separates them.

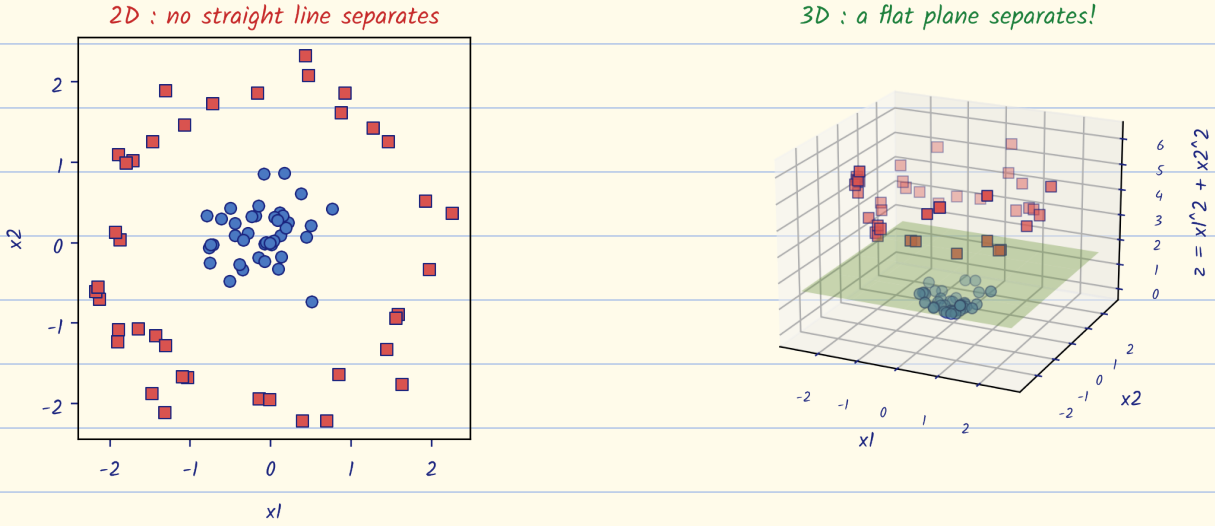


Fig 4.9 : 2-D data not separable, lifted to 3-D it is separable by a plane

4.8 Kernel Trick

Computing phi(x) explicitly can be huge (even infinite). The SVM only needs **dot products** phi(x).phi(z). A **kernel** gives that number directly from the original vectors.

> * **Kernel trick:**
> **K(x, z) = phi(x) . phi(z)**. Replace every dot product x.z in the SVM by K(x, z) - we get a non-linear classifier at the cost of a linear one. We never build phi(x).

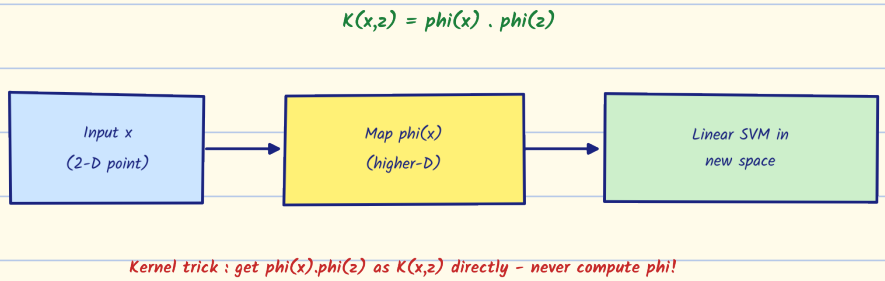


Fig 4.10 : Idea of the kernel trick

Kernel	Formula	Use
Linear	K = x.z	Linearly separable data

Kernel	Formula	Use
Polynomial	K = (x.z + c)^d	Curved, degree-d boundaries
RBF (Gaussian)	K = exp(-gamma ||x - z||^2)	Default choice; very flexible, local
Sigmoid	K = tanh(a x.z + c)	Neural-net like

- In RBF, **gamma** controls width: large gamma -> wiggly boundary (overfit), small gamma -> smooth.
- RBF kernel = 1 when x = z and falls towards 0 as points move apart (a similarity measure).

Worked example : kernel = dot product in a bigger space

Take x = (1, 2), z = (3, 1) and the degree-2 map **phi(v) = (v1^2, sqrt(2) v1 v2, v2^2)**.

Step	Calculation	Result
phi(x)	(1, sqrt2 * 1 * 2, 4)	(1.00, 2.83, 4.00)
phi(z)	(9, sqrt2 * 3 * 1, 1)	(9.00, 4.24, 1.00)
phi(x).phi(z)	9 + (2.83 x 4.24) + 4	**25.0**
Kernel (x.z)^2	x.z = 3 + 2 = 5, so 5^2	**25.0**
RBF, gamma = 0.5	||x-z||^2 = 4 + 1 = 5; exp(-0.5 x 5)	0.082

Both routes give **25** - but the kernel never left 2-D. That is the trick.

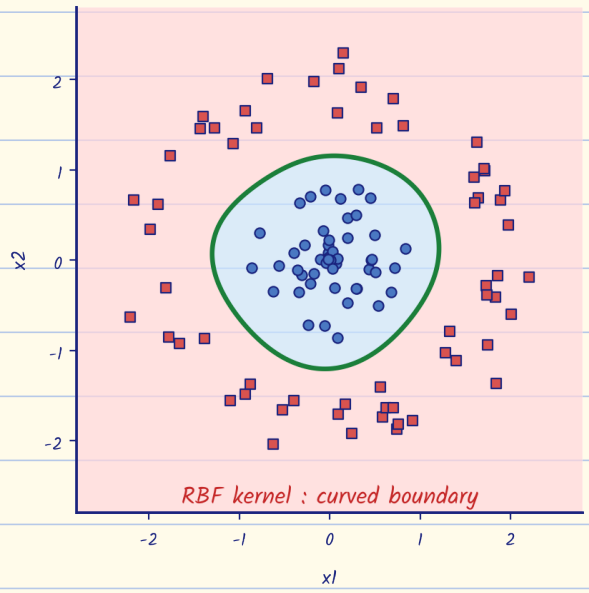


Fig 4.11 : RBF-kernel SVM draws a curved boundary around the inner class

Part 3 : Logistic & Linear Regression

4.9 Logistic Regression

Despite its name it is a **classification** method. It passes the linear score z = w.x + b through the **sigmoid** to get a probability between 0 and 1.

- Sigmoid: **p = 1 / (1 + e^(-z))**, where z = w.x + b.
- p = P(class 1 | x). Predict class 1 if **p >= 0.5** (i.e. z >= 0).
- Decision boundary is still **linear** (z = 0).
- Example: z = 0.8 gives p = **0.69** -> class 1.

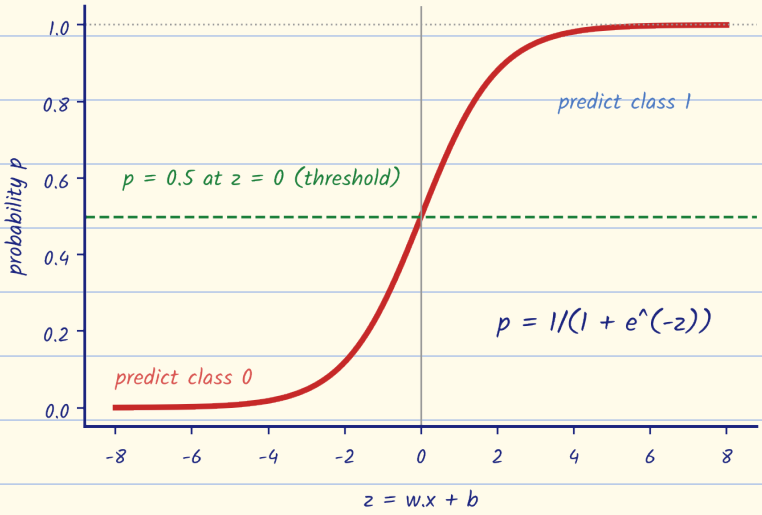


Fig 4.12 : Sigmoid squashes any score into a probability

Loss (log-loss / binary cross-entropy): for m samples with label y (0 or 1):

```
J(w,b) = -(1/m) * sum[ y*ln(p) + (1-y)*ln(1-p) ]
```

- If y = 1 and p is near 0, loss is huge; if p is near 1, loss is near 0 - it punishes confident wrong answers.
- Squared error would be non-convex here; log-loss is **convex**, so no bad local minima.
- **Gradient descent update:** w = w - alpha * (1/m) sum (p - y) x and b = b - alpha * (1/m) sum (p - y).
- Multi-class: **softmax** regression.

> * **Remember:**
> Logistic = linear score + sigmoid + log-loss + gradient descent. The gradient looks the same as in linear regression, only p replaces the prediction.

4.10 Linear Regression

Predicts a **continuous value** by fitting a straight line (hyperplane) to the data.

- Model: **y = w x + b** (or y = w.x + b for many inputs).
- **Residual** = actual y - predicted y.
- Least squares: choose w, b to minimise **SSE = sum (y - y_hat)^2** (or MSE = SSE / m).

Method	Formula / step
Simple formula	slope w = sum[(x - xbar)(y - ybar)] / sum[(x - xbar)^2]; b = ybar - w xbar
Normal equation	theta = (X^T X)^-1 X^T y (X has a column of 1s for b). Exact, but slow when features are many.
Gradient descent	w = w - alpha * (2/m) sum (y_hat - y) x. Repeat till loss stops falling. Works for huge data; needs a good alpha.

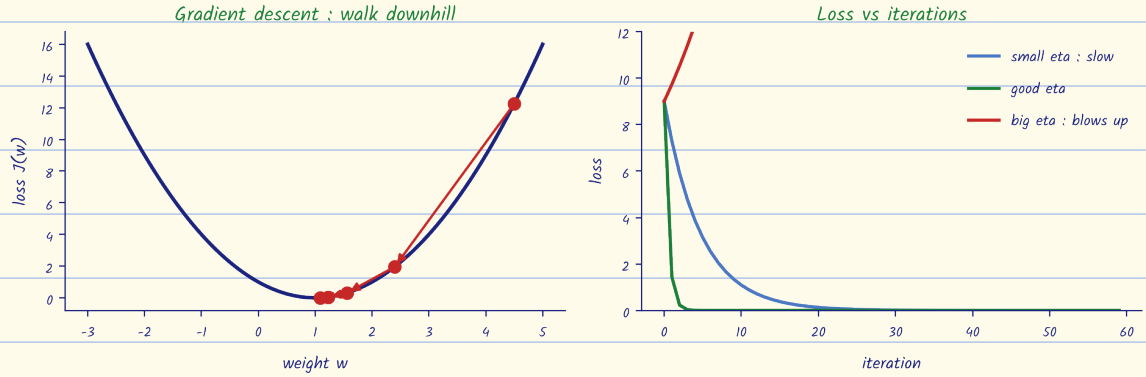


Fig 4.13 : Gradient descent walks downhill; learning rate decides the speed

Worked example : fit a line

Data (x, y): (1,2) (2,4) (3,5) (4,4) (5,5). xbar = 3, ybar = 4.

- sum (x-xbar)(y-ybar) = 4 + 0 + 0 + 0 + 4... computed exactly = **6**; sum (x-xbar)^2 = **10**.
- Slope w = 6/10 = **0.6**; intercept b = 4 - 0.6 x 3 = **2.2**. Line: **y = 0.6 x + 2.2**.
- Normal equation check: X^T X = [[5, 15],[15, 55]], X^T y = [20, 66]; theta = (2.2, 0.6). Same answer.
- Predictions: 2.8, 3.4, 4.0, 4.6, 5.2; residuals: -0.8, +0.6, +1.0, -0.6, -0.2; **SSE = 2.4**.
- Predict for x = 6: y = 0.6 x 6 + 2.2 = **5.8**.

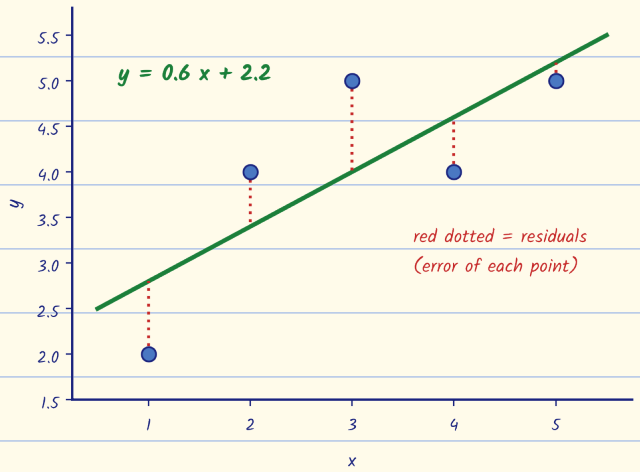


Fig 4.14 : Least-squares line and residuals

Part 4 : Multi-Layer Perceptrons & Backpropagation

4.11 Multi-Layer Perceptrons (MLPs)

A single perceptron cannot learn XOR. Stack neurons in layers and the network can learn **non-linear** functions. This is the **MLP**, a feed-forward neural network.

- **Input layer:** just holds the features (no computation).
- **Hidden layer(s):** neurons compute **h = f(W.x + b)**. Hidden layers learn new features.
- **Output layer:** gives the answer (sigmoid for binary, softmax for many classes, linear for regression).
- **Fully connected** and **feed-forward**: signals move only forward; every neuron links to all neurons of the next layer.
- Without a non-linear activation, many layers collapse into **one linear layer** - so activations are essential.
- **Universal approximation:** one hidden layer with enough neurons can approximate any continuous function.

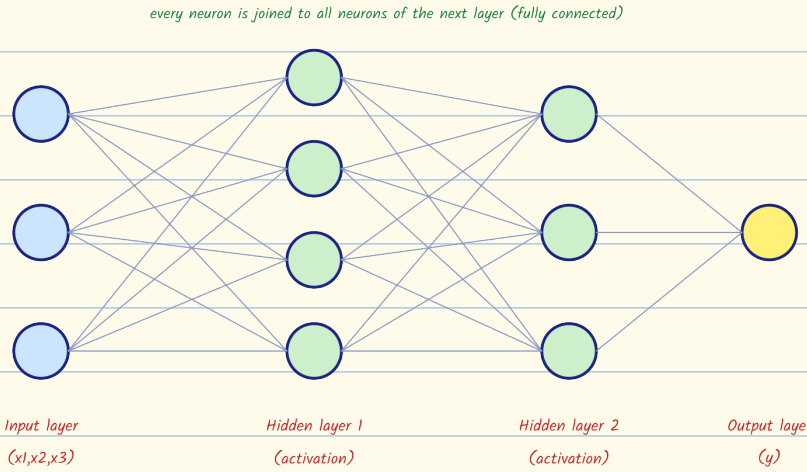


Fig 4.15 : A multi-layer perceptron (input, two hidden, output layers)

Activation	Formula	Output range / use
Step	1 if z > 0 else 0	0 or 1; perceptron, not differentiable
Sigmoid	1 / (1 + e^-z)	0 to 1; output of binary classifier
Tanh	(e^z - e^-z) / (e^z + e^-z)	-1 to 1; zero-centred
ReLU	max(0, z)	0 to inf; fast, most used in hidden layers

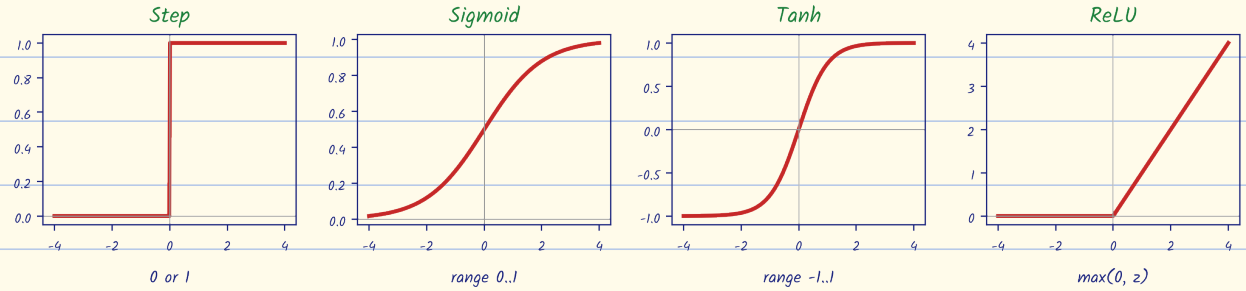


Fig 4.16 : Common activation functions

4.12 Back propagation for Training an MLP

Backpropagation is the algorithm that computes how much each weight contributed to the error, using the **chain rule**, and then updates the weights by **gradient descent**.

1. **Forward pass:** feed x through the layers to get output o.
2. **Error:** E = 1/2 (t - o)^2 (t = target).
3. **Backward pass:** use the chain rule to get dE/dw for every weight, from output layer back to the input.
4. **Update:** w = w - eta * dE/dw. Repeat for all samples / epochs.

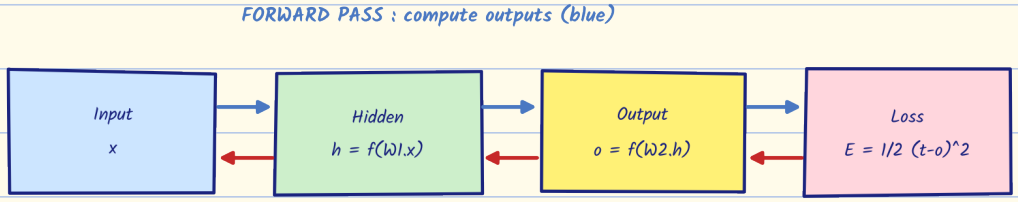


Fig 4.17 : Forward pass (blue) and backward pass (red)

For sigmoid units the chain rule gives (sigmoid derivative = o(1 - o)):

- Output delta: **d_o = (o - t) * o * (1 - o)**; gradient of weight h_j -> o is d_o * h_j.
- Hidden delta: **d_j = (w_jo * d_o) * h_j * (1 - h_j)**; gradient of weight x_i -> h_j is d_j * x_i.
- Bias gradient = the delta of that neuron.

Worked example : one step on a 2-2-1 network

Sigmoid everywhere, eta = 0.5, input x = (0.5, 1.0), target t = 1.

- Weights: x1->h1 = 0.1, x2->h1 = 0.3, x1->h2 = 0.2, x2->h2 = 0.4; biases of h1, h2 = 0.1; h1->o = 0.5, h2->o = 0.6; bias of o = 0.1.

Step	Calculation	Value
Forward: hidden	z_h1 = 0.1(0.5)+0.3(1.0)+0.1 = 0.45; z_h2 = 0.2(0.5)+0.4(1.0)+0.1 = 0.60	h1 = 0.6106, h2 = 0.6457
Forward: output	z_o = 0.5(0.6106)+0.6(0.6457)+0.1 = 0.7927	o = 0.6884
Error	E = 1/2 (1 - 0.6884)^2	E = 0.0485
Output delta	(o - t) o (1 - o) = (-0.3116)(0.6884)(0.3116)	d_o = -0.0668
Hidden deltas	d_1 = 0.5 d_o h1(1-h1); d_2 = 0.6 d_o h2(1-h2)	d_1 = -0.0079, d_2 = -0.0092
Update h->o	0.5 - 0.5(d_o h1); 0.6 - 0.5(d_o h2)	0.5204, 0.6216
Update x->h	w(x1->h1) = 0.1 - 0.5(d_1)(0.5); w(x2->h1) = 0.3 - 0.5(d_1)(1.0)	0.1020, 0.3040
New forward pass	same input, new weights	o = 0.7016, E = 0.0445

Output moved from **0.6884 to 0.7016** (towards t = 1) and error fell from **0.0485 to 0.0445** after a single update. Repeating this for many epochs trains the network.

* **Remember:**
Forward = compute. Backward = blame. Update = fix. Use a **small learning rate**; too big overshoots, too small is slow. Sigmoid/tanh can cause **vanishing gradients** in deep nets - ReLU helps.

Part 5 : Quick Revision & Likely Exam Questions

Quick Revision

Topic	One-line summary
Linear discriminant	g(x) = w.x + b; hyperplane g = 0; w is normal to it
Perceptron	Weighted sum + step; w = w + eta(t-y)x; fails on XOR
SVM	Maximum margin = 2/||w||; support vectors fix the boundary
Soft margin	Slack xi, penalty C: large C = narrow margin, small C = wide margin
Kernel trick	K(x,z) = phi(x).phi(z); poly (x.z + c)^d, RBF exp(-gamma||x-z||^2)
Logistic reg.	Sigmoid probability, log-loss, gradient descent, linear boundary
Linear reg.	Least squares: normal equation or gradient descent
MLP	Layers + non-linear activations; solves XOR
Backprop	Forward pass, error, chain-rule deltas backwards, w = w - eta dE/dw

Likely exam questions

1. What is a linear discriminant? Explain hyperplane and the role of w and b with a diagram.
2. Explain the perceptron and the perceptron learning algorithm. Train it for the AND gate (show epochs).
3. Why can a single perceptron not solve XOR? How does an MLP solve it?
4. Explain SVM. Derive / state margin = 2/||w|| and define support vectors.
5. What is the soft-margin SVM? Explain slack variables and the parameter C.
6. What is the kernel trick? Describe polynomial and RBF kernels with an example.
7. Explain logistic regression: sigmoid, log-loss and gradient descent. How is it different from linear regression?
8. Explain the backpropagation algorithm with a neat diagram and a numeric example for a 2-2-1 network.

MACHINE LEARNING

UNIT – V : Clustering

Easy digital notes with diagrams and solved examples (JNTU B.Tech, 23A31401T)

Topics in this unit

1. Introduction to clustering, clustering of patterns, partitioning of data
2. Matrix factorization
3. Hierarchical: divisive and agglomerative clustering
4. Partitional clustering and K-Means
5. Soft partitioning, soft clustering, Fuzzy C-Means
6. Rough clustering and Rough K-Means
7. Expectation Maximization (EM) based clustering
8. Spectral clustering

> * **Study tip:**
> All numbers in the worked examples were computed by a program, so you can copy them into an exam answer. Learn the RED words, then redo each small example once on paper.

Syllabus (as given)

Clustering: Introduction to Clustering, Partitioning of Data, Matrix Factorization, Clustering of Patterns, Divisive Clustering, Agglomerative Clustering, Partitional Clustering, K-Means Clustering, Soft Partitioning, Soft Clustering, Fuzzy C-Means Clustering, Rough Clustering, Rough K-Means Clustering Algorithm, Expectation Maximization-Based Clustering, Spectral Clustering.

Part I : Foundations of Clustering

5.1 Introduction to Clustering

Clustering groups a set of objects so that objects in the **same group (cluster)** are **similar** to each other and **different** from objects in other groups. It is **unsupervised learning**: there are **no class labels**, the structure is found from the data alone.

- **Goal:** high **intra-cluster similarity** (compact clusters) and low **inter-cluster similarity** (well separated clusters).
- **Applications:** customer segmentation, document / news grouping, image segmentation, gene-expression analysis, anomaly detection, data compression (vector quantization).
- **Clustering vs classification:** classification learns from labelled examples and predicts a known class; clustering **discovers** the groups itself and the number of groups may be unknown.

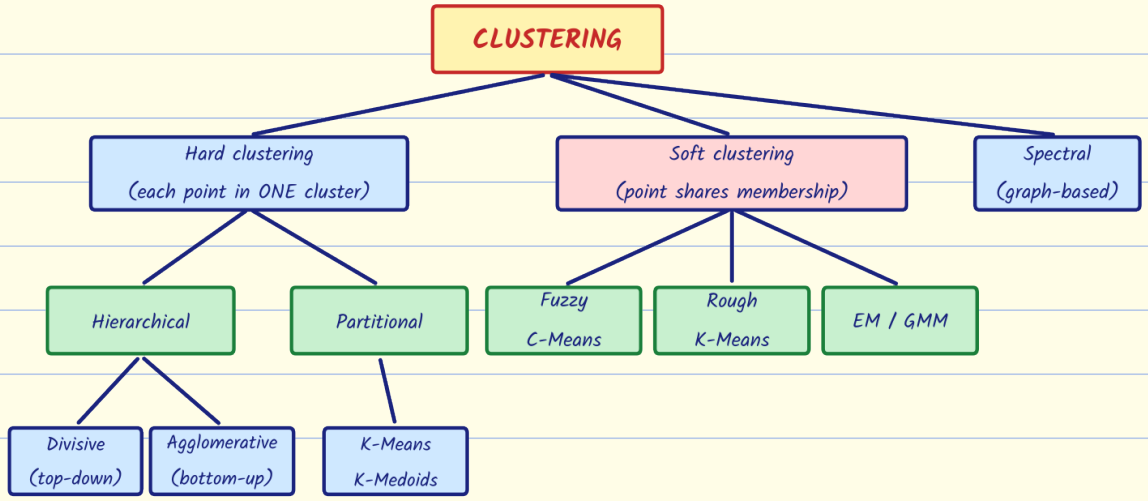


Fig 5.1 : Types of clustering covered in this unit

> * **Remember:**
> Clustering has no single 'correct' answer. The result depends on the distance measure, the algorithm and the number of clusters chosen.

5.2 Clustering of Patterns

A **pattern** is one object described by **d features**, written as a vector $x = (x_1, \ldots, x_d)$. A data set of n patterns is the **n × d pattern matrix**. Clustering of patterns needs a way to measure how close two patterns are.

Measure	Formula	Use
Euclidean	$d(x,y) = \sqrt{\Sigma (x_i - y_i)^2}$	Default for numeric data; K-Means
Manhattan	$d(x,y) = \Sigma \lvert x_i - y_i \rvert$	Robust to outliers, grid-like data
Minkowski	$d = (\Sigma \lvert x_i - y_i \rvert^p)^{1/p}$	$p = 1$ Manhattan, $p = 2$ Euclidean
Cosine similarity	$\cos \theta = (x \cdot y) / (\lvert x \rvert \lvert y \rvert)$	Text / document vectors
Hamming / Jaccard	Count of differing bits / $\lvert A \cap B \rvert$ / $\lvert A \cup B \rvert$	Binary and set data

Steps of a clustering task

1. **Represent** patterns as feature vectors (scale features so that no feature dominates).
2. Choose a **proximity measure** (distance or similarity).
3. Choose the **clustering algorithm** and the number of clusters K (if needed).
4. **Validate** the result: **internal** measures (SSE, silhouette) need no labels; **external** measures (purity, Rand index) compare with known labels.
5. **Interpret** the clusters (give them meaning, e.g. 'high-spending young customers').

> * **Study tip:**
> ALWAYS standardise features first. If income is in lakhs and age in years, income will dominate the Euclidean distance.

5.3 Partitioning of Data

A **partition** divides data $X = \{x_1, ..., x_n\}$ into K clusters $C_1, ..., C_K$. A **hard (crisp) partition** must satisfy three conditions:

- Every cluster is **non-empty**: $C_i \neq \emptyset$
- Clusters **cover** all data: $C_1 \cup C_2 \cup ... \cup C_K = X$
- Clusters are **disjoint**: $C_i \cap C_j = \emptyset$ for $i \neq j$ (each object in exactly ONE cluster).

In a **soft partition** the third condition is relaxed. Each object has a **membership degree** $u_{ij} \in [0,1]$ in every cluster, with $\Sigma_j u_{ij} = 1$. A hard partition is the special case where every u_{ij} is 0 or 1.

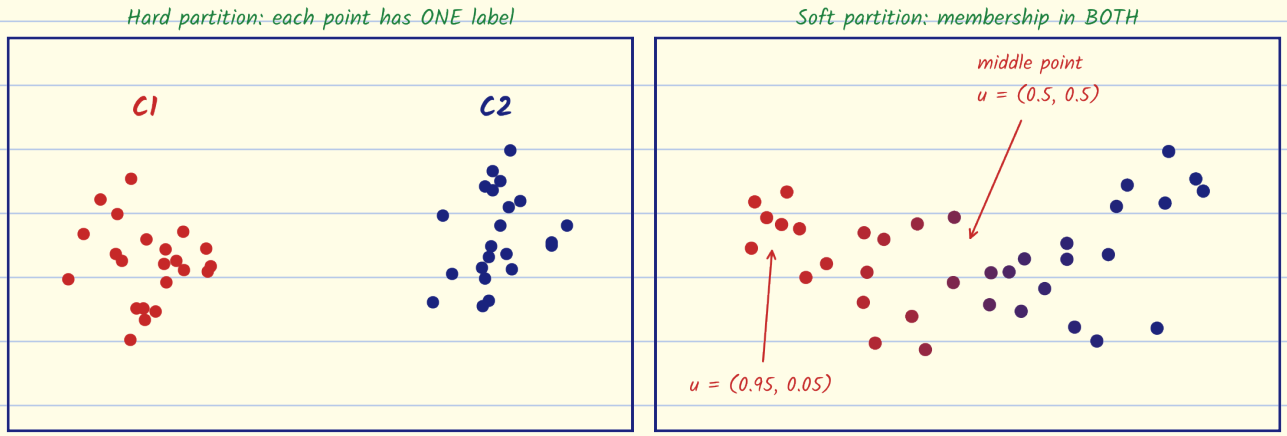
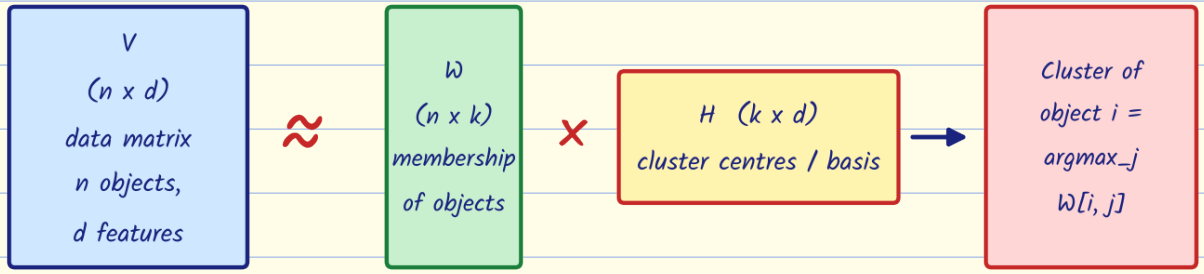


Fig 5.2 : Hard partition and soft partition of the same kind of data

Why we need smart algorithms: the number of ways to partition n objects into K non-empty clusters is the **Stirling number of the second kind** $S(n,K) \approx K^n / K!$. Check: $S(6,2) = 31$; $S(10,3) = 9,330$; $S(100,5) \approx 6.57e+67$. Trying every partition is impossible, so we use **heuristics** (K-Means) that find a good, not always the best, partition.

5.4 Matrix Factorization

Matrix factorization approximates the data matrix V ($n \times d$) by the product of two smaller matrices: $V \approx W \cdot H$ with W ($n \times k$) and H ($k \times d$), k much smaller than n and d. It gives a **low-rank** view of the data and can be read as clustering.



k << n, d : each row of W says which cluster, each row of H is one centre

Fig 5.3 : Matrix factorization view of clustering

Method	Constraint	Clustering meaning
K-Means	W is binary, exactly one 1 per row	H = cluster centres; W = hard assignment. Minimises $\|\|V - WH\|\|^2$ (same as SSE)
NMF	V, W, H all ≥ 0	Parts-based; W gives soft membership; cluster = argmax of row of W
SVD / PCA	$V = U\Sigma V^T$, orthogonal factors	Keep top k singular vectors, then cluster rows of $U\Sigma$ (denoise + reduce dimension)
Spectral	Eigenvectors of Laplacian	Factorise the graph matrix, cluster the eigenvector rows (see 5.14)

NMF objective: minimise $\|\|V - WH\|\|_F^2$ subject to W, H ≥ 0. Lee-Seung multiplicative updates: $H \leftarrow H \odot (W^TV) / (W^TWH)$, $W \leftarrow W \odot (VH^T) / (WHH^T)$.

Tiny example: 4 documents × 4 words (counts). NMF with k = 2 gives the memberships below: D1, D2 load on factor 2 (words w1, w2) and D3, D4 on factor 1 (words w3, w4), so the documents form **2 clusters**.

Doc	w1	w2	w3	w4	W (k=2)	Cluster
D1	3	2	0	0	(0.00, 1.60)	B
D2	2	3	0	1	(0.14, 1.62)	B
D3	0	0	4	3	(1.84, 0.00)	A
D4	1	0	3	4	(1.83, 0.26)	A

Part II : Hierarchical Clustering

5.5 Divisive Clustering (top-down)

Hierarchical clustering builds a **tree of clusters** (dendrogram) and needs no K in advance. **Divisive** clustering starts with **all objects in one cluster** and **splits** it again and again until every object is alone (or K clusters remain). **Agglomerative** does the opposite.

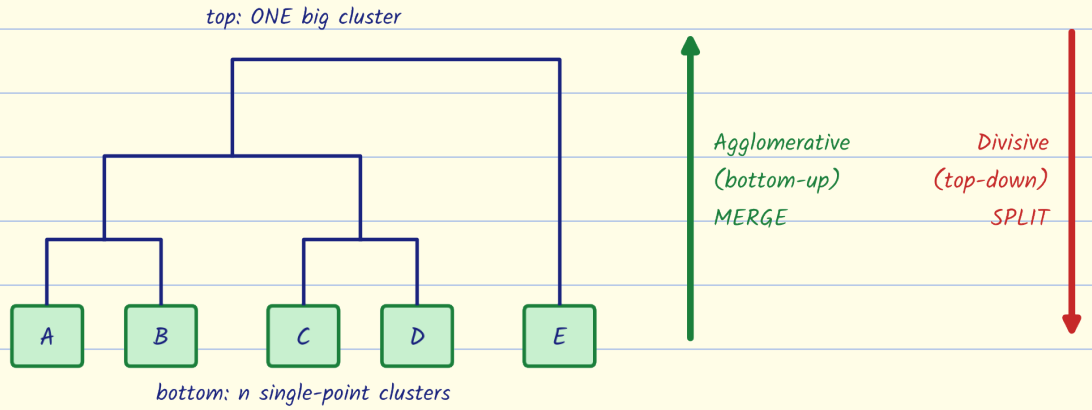


Fig 5.4 : Divisive (top-down) and agglomerative (bottom-up) directions

DIANA algorithm (Divisive Analysis)

1. Put all objects in one cluster. Pick the cluster with the **largest diameter** to split.
2. Find the object with the **largest average dissimilarity** to the others - it starts the **splinter group**.
3. For each remaining object compute avg distance to main group - avg distance to splinter group. If it is **positive**, move the object to the splinter group. Repeat until no object moves.
4. Repeat from step 1 until clusters are single objects or K clusters are reached.

Example (1-D points A=1, B=2, C=6, D=8, E=15). Average dissimilarity of each point to the rest: A 6.75, B 6.00, C 5.00, D 5.50, E 10.75. E is largest, so splinter = {E}; the other points are closer to their own group, so nobody moves: split 1 = {A,B,C,D} | {E}. Largest-diameter cluster is {A,B,C,D} (diameter 7): averages A 4.33, B 3.67, C 3.67, D 5.00, so splinter starts with D; C is closer to D (avg 2) than to A,B (avg 4.5), so it moves. Split 2 = {A,B} | {C,D}. Final tree: same as the agglomerative tree of 5.6.

- **Bisecting K-Means:** a popular practical divisive method - repeatedly split the cluster with the largest SSE using K-Means with K = 2.
- **Cost:** there are $2^{n-1} - 1$ ways to split n objects, so exact divisive clustering is very costly; heuristics (DIANA) are used. Good at finding the **few large clusters** first.

5.6 Agglomerative Clustering (bottom-up)

Agglomerative (AGNES) starts with **n clusters of one object each** and **merges** the two **closest** clusters in every step until a single cluster remains. The history of merges is drawn as a **dendrogram**; cutting it at a height gives the clusters.

1. Compute the n x n distance matrix; every object is its own cluster
2. Find the two closest clusters Ci, Cj
3. Merge them; update the distances to the new cluster (linkage rule)
4. Repeat 2-3 until one cluster is left (or K clusters remain)

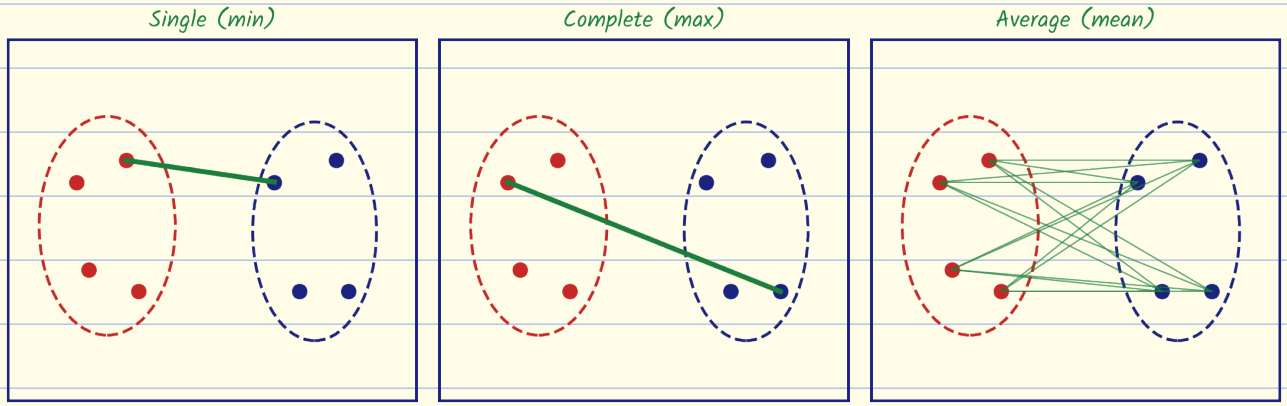


Fig 5.5 : Single, complete and average linkage

Linkage	Distance between clusters A and B	Behaviour
Single	min d(a,b) (closest pair)	Finds long, chain-like shapes; suffers **chaining**
Complete	max d(a,b) (farthest pair)	Compact, equal-sized clusters; sensitive to outliers
Average	mean of all d(a,b) (UPGMA)	Compromise between single and complete
Centroid / Ward	distance of centres / increase in SSE	Ward gives compact, balanced clusters

Worked example: points A=1, B=2, C=6, D=8, E=15 (1-D, so d = |x - y|).

d	A	B	C	D	E
A	0	1	5	7	14
B	1	0	4	6	13
C	5	4	0	2	9
D	7	6	2	0	7
E	14	13	9	7	0

Step	Merge (single linkage)	Distance	Clusters left
1	A + B	1	{AB}, C, D, E
2	C + D	2	{AB}, {CD}, E
3	{AB} + {CD}	min(|2-6|) = 4	{ABCD}, E
4	{ABCD} + E	min(|8-15|) = 7	{ABCDE}

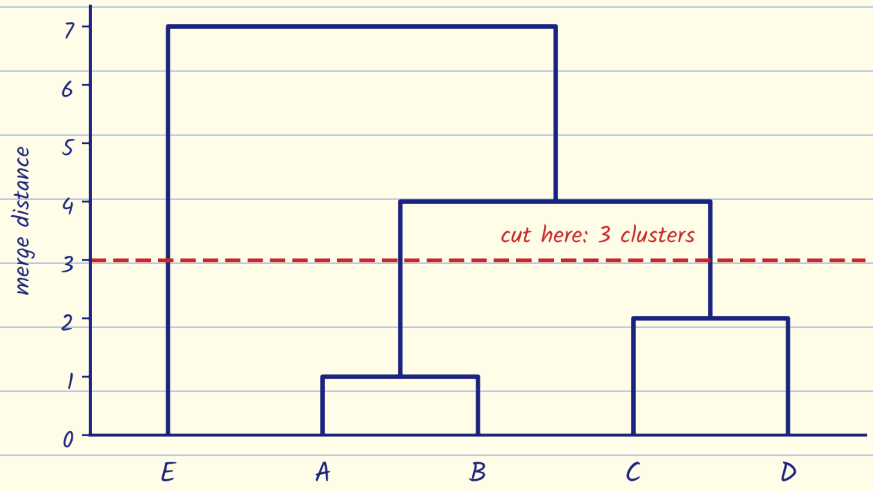


Fig 5.6 : Dendrogram of the single-linkage example; cutting at height 3 gives {A,B}, {C,D}, {E}

Same data, complete linkage: steps 1, 2 are the same (1, 2); step 3 merges {AB} and {CD} at max distance 7 (not 4); step 4 adds E at 14. Average linkage gives 5.5 and 10.75. So the **linkage choice changes the heights**, though here the tree shape stays the same.

- **Choosing K:** cut the dendrogram where there is the **longest vertical gap** (a big jump in merge distance) - here the jump 2 → 4 → 7.
- **Complexity:** $O(n^2)$ memory and $O(n^3)$ time ($O(n^2 \log n)$ with a heap). Hence **not suitable for very large n**.
- **Pros:** no K needed in advance, dendrogram is informative, any distance works. **Cons:** a merge can never be undone (greedy), slow, sensitive to noise.

Part III : Partitional Clustering and K-Means

5.7 Partitional Clustering

Partitional clustering directly divides the data into **K flat (non-overlapping) clusters** in one go, then **improves** the partition step by step by optimising a criterion (usually minimise the squared error). K must be given. Examples: K-Means, K-Medoids (PAM), K-Modes, CLARA.

Point	Hierarchical	Partitional
Output	Tree (dendrogram), nested clusters	One flat partition into K clusters
K needed?	No (cut the tree later)	Yes (input)
Time	$O(n^2)$ - $O(n^3)$	$O(n \cdot K \cdot t)$ - fast
Undo a step?	No (greedy merges / splits)	Yes - points can move between clusters
Best for	Small data, taxonomy	Large data, round clusters

5.8 K-Means Clustering

K-Means (MacQueen, Lloyd) represents each cluster by its **centroid (mean)**. It alternates between **assigning** points to the nearest centroid and **moving** each centroid to the mean of its points.

Objective (SSE / WCSS): $J = \Sigma_{j=1..K} \Sigma_{x \in C_j} || x - \mu_j ||^2$

Assign: $x \rightarrow C_j$ with smallest $||x - \mu_j||$ **Update:** $\mu_j = (1/|C_j|) \Sigma_{x \in C_j} x$

1. Choose K and initial centroids mu_1 ... mu_K (random points / k-means++)
2. ASSIGN each point to its nearest centroid
3. UPDATE each centroid = mean of the points assigned to it
4. Repeat 2-3 until assignments stop changing (or J stops decreasing)

Worked example (K = 2). Points P1(1,1), P2(2,1), P3(1,2), P4(5,4), P5(6,5), P6(5,5). Take **initial centroids = P1 and P2**.

Iteration 1: c1 = (1.00, 1.00), c2 = (2.00, 1.00). Assign: C1 = {P1, P3}, C2 = {P2, P4, P5, P6}. **New centroids:** c1 = (1.00, 1.50), c2 = (4.50, 3.75); SSE = 20.25.

Iteration 2: c1 = (1.00, 1.50), c2 = (4.50, 3.75). Assign: C1 = {P1, P2, P3}, C2 = {P4, P5, P6}. **New centroids:** c1 = (1.33, 1.33), c2 = (5.33, 4.67); SSE = 2.67.

Point	It.1: d(c1)	d(c2)	Cluster	It.2: d(c1)	d(c2)	Cluster
P1	0.00	1.00	C1	0.50	4.45	C1
P2	1.00	0.00	C2	1.12	3.72	C1
P3	1.00	1.41	C1	0.50	3.91	C1
P4	5.00	4.24	C2	4.72	0.56	C2
P5	6.40	5.66	C2	6.10	1.95	C2
P6	5.66	5.00	C2	5.32	1.35	C2

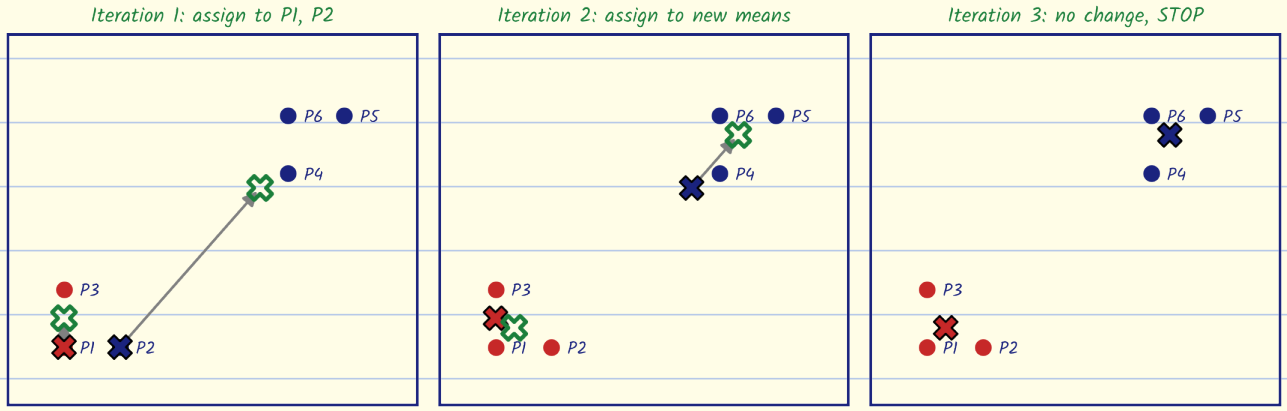


Fig 5.7 : K-Means on the six points: assign, move the centroids, stop when nothing changes

Iteration 3 (check): with c1 = (1.33, 1.33) and c2 = (5.33, 4.67) the assignment is the same, so the algorithm **has converged**. Final SSE = 2.67 (it fell from 20.25 to 2.67 to 2.67; J never increases). Silhouette score of this clustering = 0.78.

Choosing K, initialisation and variants

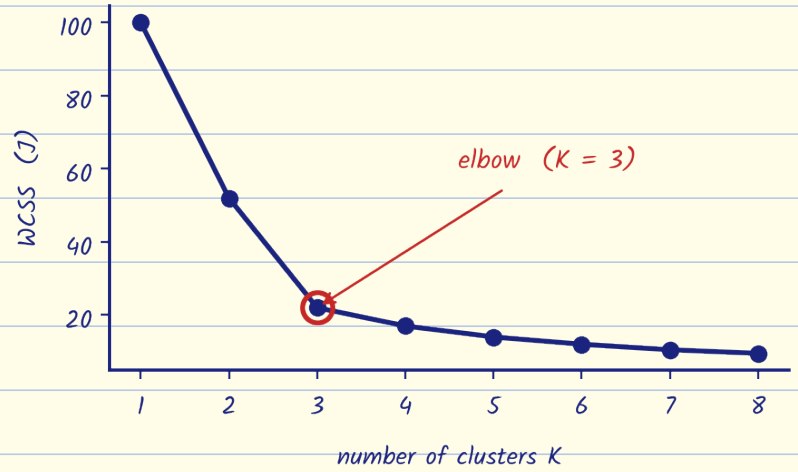


Fig 5.8 : Elbow method (illustrative values): WCSS falls sharply until K = 3, then slowly

- **Elbow method:** plot J against K; pick the K at the bend. **Silhouette:** $s(i) = (b - a) / \max(a, b)$, a = mean distance to own cluster, b = mean distance to nearest other cluster; s near +1 is good; choose K with highest average s.
- **K-Means++:** choose the first centroid at random, each next one with probability $\propto D(x)^2$ (distance to nearest chosen centroid) - spread-out start, better results. Run several times, keep the lowest J.
- **K-Medoids (PAM):** centre is an actual data point (medoid) - robust to outliers, works with any distance. **Mini-batch K-Means:** update with small random batches for big data.
- **Complexity:** $O(n \cdot K \cdot d \cdot t)$ for t iterations - linear in n. **Always converges** to a **local** minimum (not necessarily the global one).

Pros	Cons
Simple, fast, scales to large data; easy to implement	K must be chosen beforehand
Compact clusters; works well for round, equal-sized blobs	Depends on initial centres - local minimum
Output is easy to interpret (centroids)	Fails on non-convex shapes, unequal sizes / densities
Many variants (K-Medoids, mini-batch)	Sensitive to outliers and to feature scaling; hard assignment only

Part IV : Soft Clustering Methods

5.9 Soft Partitioning and Soft Clustering

Soft clustering (overlapping clustering) lets one object belong to **several clusters** with a **degree**. The result is a **membership matrix U** ($n \times K$) instead of one label per point. This is natural for borderline objects: a document on 'sports finance', a pixel on an object edge, a gene with two functions.

Point	Hard clustering	Soft clustering
Membership	$u_{ij} \in \{0, 1\}$	$u_{ij} \in [0, 1]$ (fuzzy / probability) or lower / upper sets (rough)
Row sum	$\Sigma_j u_{ij} = 1$ (single 1)	$\Sigma_j u_{ij} = 1$ (shared) [rough: sets]
Boundary points	Forced into one cluster	Shown as ambiguous - extra information
Examples	K-Means, Agglomerative	Fuzzy C-Means, Rough K-Means, EM / GMM
Cost	Cheaper, simpler	More computation; need a rule to get a final hard label (take the maximum u)

- Three ways to be soft: **fuzzy** (degrees in [0,1], FCM), **probabilistic** (posterior probabilities, EM/GMM), **rough** (lower and upper approximation sets).
- **Fuzzy vs probabilistic:** both rows sum to 1, but a probabilistic membership is a **probability under a model**, a fuzzy membership is just a **degree of belonging**.

5.10 Fuzzy C-Means (FCM) Clustering

Fuzzy C-Means (Dunn 1973, Bezdek 1981) is the **soft version of K-Means**. There are C clusters with centres v_j; each point x_i has a membership u_{ij} in each cluster, with $\Sigma_j u_{ij} = 1$.

Objective: $J_m = \Sigma_{i=1..n} \Sigma_{j=1..C} (u_{ij})^m \cdot || x_i - v_j ||^2$ $m > 1$ = fuzzifier
Membership: $u_{ij} = 1 / \Sigma_{k=1..C} (d_{ij} / d_{ik})^{2/(m-1)}$ $d_{ij} = ||x_i - v_j||$
Centre: $v_j = \Sigma_i (u_{ij})^m x_i / \Sigma_i (u_{ij})^m$

1. Fix C, fuzzifier m (usually 2), tolerance eps; initialise U randomly (rows sum to 1)
2. Compute centres v_j from U using the centre formula
3. Update memberships u_ij from the distances
4. Stop if max change in U < eps, else go to 2

- **Fuzzifier m:** $m \to 1$ gives hard K-Means ($u \to 0$ or 1); large m makes all memberships $\to 1/C$ (very fuzzy). $m = 2$ is the common choice.
- A point very close to a centre gets $u \approx 1$ for it; a point midway gets $u = 0.5, 0.5$ (for $C = 2$). Unlike probabilities, memberships do not use a density model.

Worked example (1-D, C = 2, m = 2). Data $x = 1, 2, 5, 8$; current centres $v_1 = 1.5$, $v_2 = 6.5$. For $x = 1$: $u_1 = 1 / (1 + (0.5/5.5)^2) = 0.9918$. All memberships:

x	d1	d2	u1	u2	u1²	u2²
1	0.5	5.5	0.9918	0.0082	0.9837	0.0001
2	0.5	4.5	0.9878	0.0122	0.9758	0.0001
5	3.5	1.5	0.1552	0.8448	0.0241	0.7137
8	6.5	1.5	0.0506	0.9494	0.0026	0.9014

New centres: $v_1 = \Sigma u_1^2 x / \Sigma u_1^2 = 1.5488$; $v_2 = 6.6736$. Objective J fell from 4.532 to 4.456. After more iterations the centres settle near 1.5 and 6.7. Point $x = 5$ stays mostly in cluster 2 ($u_2 = 0.81$) but keeps $u_1 = 0.19$ in cluster 1 - the soft information.

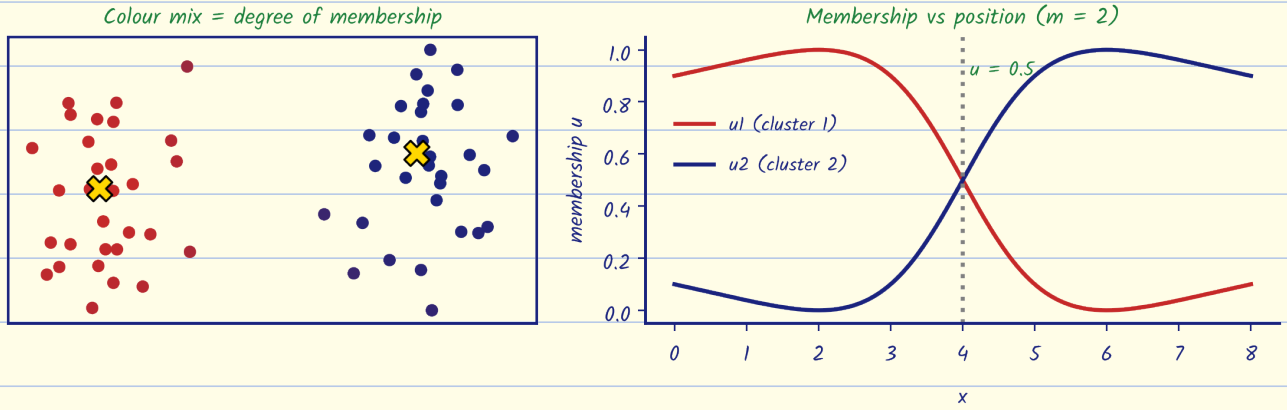


Fig 5.9 : FCM memberships: colour mix on the left, membership curves for two centres on the right

- **Pros:** gives degrees of belonging, handles overlapping clusters, usually more robust than K-Means near boundaries.
- **Cons:** C and m must be chosen; sensitive to initialisation and noise (memberships sum to 1, so outliers still get high membership); slower than K-Means; assumes round clusters.

5.11 Rough Clustering

Rough set theory (Pawlak, 1982) describes a set that cannot be defined exactly using **two crisp sets**: the **lower approximation** and the **upper approximation**. For a cluster C:

- **Lower approximation** $\underline{C}$: objects that **surely** belong to C (and to no other cluster).
- **Upper approximation** $\overline{C}$: objects that **possibly** belong to C (surely + doubtful ones).
- **Boundary region** = $\overline{C} - \underline{C}$: doubtful objects, shared with other clusters. Properties: $\underline{C} \subseteq C \subseteq \overline{C}$; if the boundary is empty the cluster is an ordinary crisp set.
- If an object is in the lower approximation of C_i it is in the lower approximation of **no other** cluster, and also in the upper approx. of C_j; an object in no lower approximation belongs to the upper approximation of **at least two** clusters.

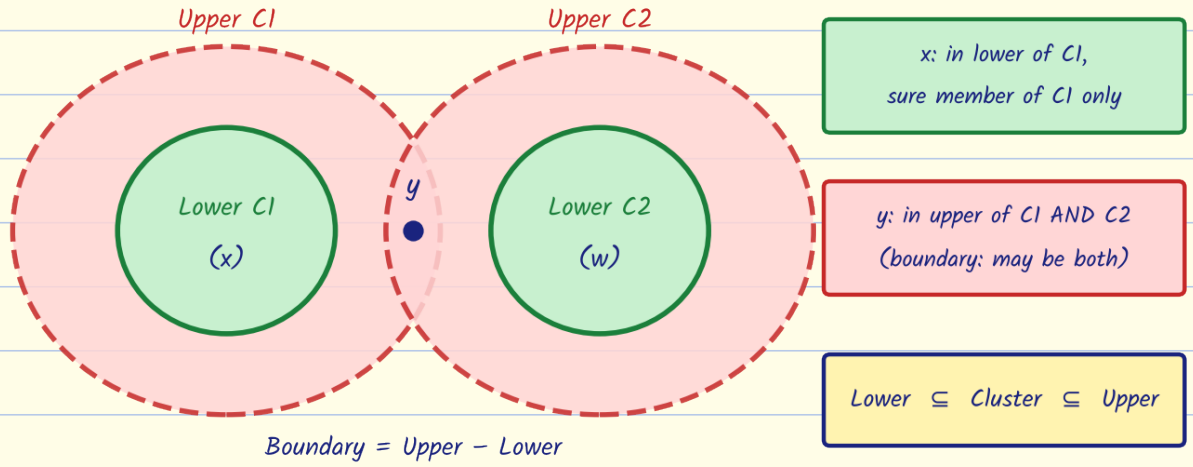


Fig 5.10 : Rough clustering: lower approximation (sure), upper approximation (possible) and boundary

Idea: unlike fuzzy sets (a number for every point), rough clustering gives a **sure core** plus a **doubtful fringe** - useful when we only want to say 'definitely in', 'maybe in'.

5.12 Rough K-Means Clustering Algorithm (Lingras and West)

Rough K-Means keeps the K-Means loop but every cluster has a **lower** and an **upper** approximation, and each centre is a **weighted mean** of the two parts.

1. Choose K, threshold eps, weights w_l + w_u = 1 (w_l > w_u), initial centres
2. For each x: find nearest centre m (d_min). T = { j : d(x,c_j) - d_min <= eps }
3. If |T| = 1 : x goes to LOWER and UPPER approximation of that cluster
   If |T| > 1 : x goes to the UPPER approximation of every cluster in T (boundary)
4. New centre: c_j = w_l * mean(lower_j) + w_u * mean(boundary_j)
   (if boundary_j is empty, c_j = mean(lower_j))
5. Repeat 2-4 until the centres stop changing

Centre update: $c_j = w_l \cdot (1/|\underline{C}_j|) \sum_{x \in \underline{C}_j} x + w_u \cdot (1/|\overline{C}_j - \underline{C}_j|) \sum_{x \in \overline{C}_j - \underline{C}_j} x$

Worked example: x = 1, 2, 5, 8, 9; c_1 = 1.5, c_2 = 8.5; ε = 1; w_l = 0.7, w_u = 0.3. Distances (to c_1, c_2): 1→(0.5, 7.5), 2→(0.5, 6.5), 5→(3.5, 3.5), 8→(6.5, 0.5), 9→(7.5, 0.5). Point 5 has difference 0 ≤ ε, so T = {1,2}: it is a boundary object. All others have a difference > 1, so they are sure members.

Cluster	Lower approx.	Boundary	Upper approx.	New centre
1	{1, 2} mean 1.5	{5}	{1, 2, 5}	0.7×1.5 + 0.3×5 = 2.55
2	{8, 9} mean 8.5	{5}	{5, 8, 9}	0.7×8.5 + 0.3×5 = 7.45

- **Effect:** the boundary point pulls each centre only with the small weight w_u, so uncertain points do not distort the clusters. With ε = 0 and w_u = 0 it behaves like K-Means.
- **Parameters:** ε controls how big the boundary is, w_l / w_u control the influence. **Pros:** handles vague boundaries and outliers better, no fuzzy-membership computation. **Cons:** three parameters (K, ε, w) to tune.

5.13 Expectation Maximization (EM) Based Clustering

EM clustering assumes the data come from a **mixture of K Gaussians** (GMM): $p(x) = \Sigma_k \pi_k N(x \mid \mu_k, \Sigma_k)$, with **mixing weights** π_k ($\Sigma\pi_k = 1$), means μ_k and covariances Σ_k. The cluster of a point is the hidden (latent) component that generated it. EM finds the parameters $\theta = (\pi, \mu, \Sigma)$ that **maximise the log-likelihood** $\Sigma_i \log p(x_i)$.

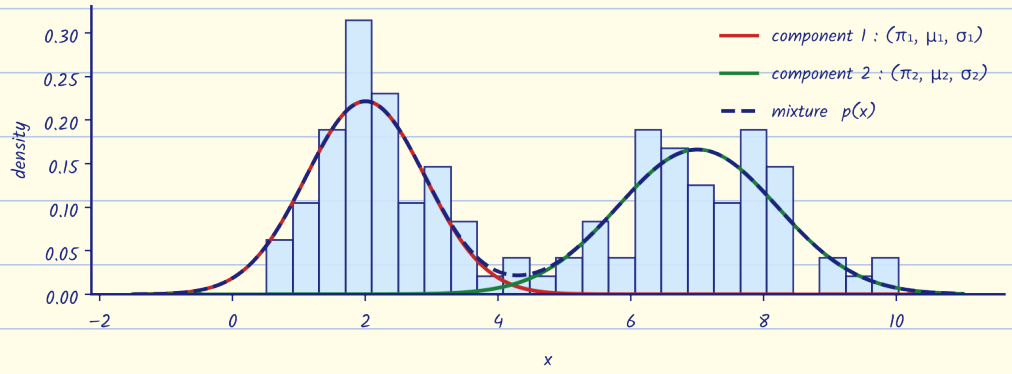


Fig 5.11 : A two-component Gaussian mixture fitted to 1-D data

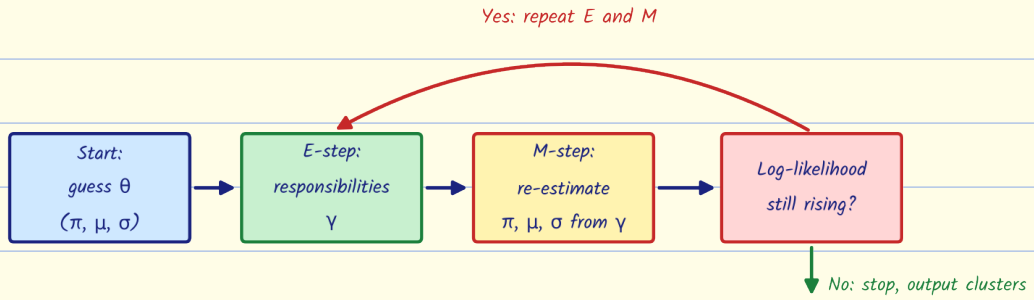


Fig 5.12 : The EM loop: E-step and M-step repeated until convergence

E-step (responsibility): $\gamma_{ik} = \pi_k \, N(x_i \mid \mu_k, \Sigma_k) \; / \; \Sigma_j \, \pi_j \, N(x_i \mid \mu_j, \Sigma_j)$

M-step: $N_k = \Sigma_i \, \gamma_{ik} \quad \mu_k = (1/N_k) \, \Sigma_i \, \gamma_{ik} \, x_i \quad \Sigma_k = (1/N_k) \, \Sigma_i \, \gamma_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^T \quad \pi_k = N_k \; / \; n$

- **Algorithm:** initialise π, μ, Σ (often from K-Means) → E-step → M-step → repeat until the log-likelihood changes by less than a tolerance. Each iteration **never decreases** the likelihood.
- **Output:** soft clusters γ_{ik} (probabilities); hard label = $\text{argmax}_k \, \gamma_{ik}$. **K-Means is a special case:** equal weights, equal spherical covariances, and γ forced to 0/1.

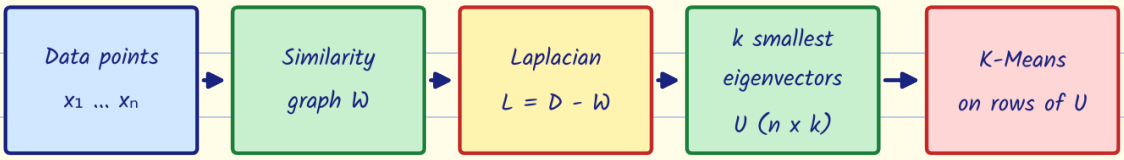
Worked example (one EM iteration). Data x = 1, 2, 5, 8, 9; K = 2; start μ = (2, 8), σ = (2, 2), π = (0.5, 0.5). E-step responsibilities γ_{i1} (cluster 1): x=1: 0.998, x=2: 0.989, x=5: 0.500, x=8: 0.011, x=9: 0.002. The middle point x = 5 is exactly 0.5 / 0.5.

Parameter	Cluster 1	Cluster 2	How
$N_k = \Sigma\gamma$	2.50	2.50	sum of responsibilities
$\pi_k = N_k/5$	0.50	0.50	share of data
μ_k	2.234	7.766	weighted mean (moved from 2 and 8)
σ_k	1.533	1.533	weighted std. deviation (from 2)
log-likelihood	-12.181 → -11.801		increased after one EM step

Point	K-Means	EM / GMM
Assignment	Hard	Soft (probabilities)
Cluster shape	Spherical, equal size	Elliptical (full covariance), different sizes
Model	Distance only	Probabilistic generative model
Model selection	Elbow, silhouette	BIC / AIC, likelihood
Weakness	Needs round clusters	Local optimum, singular covariance, slower

5.14 Spectral Clustering

Spectral clustering uses the **eigenvectors** of a graph matrix to turn the data into a space where clusters become easy, then runs K-Means there. It finds clusters of **any shape** (connected / non-convex) that K-Means cannot.



graph -> matrix -> new coordinates -> ordinary clustering

Fig 5.13 : Spectral clustering pipeline

1. **Similarity graph:** each point is a node; edge weight $W_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma^2)$ (Gaussian), or connect only the k nearest neighbours (k-NN graph) or points within ε.
2. **Degree matrix** $D = \mathrm{diag}(d_i)$, $d_i = \sum_j W_{ij}$. **Laplacian** $L = D - W$ (normalised: $L_{sym} = I - D^{-1/2} W D^{-1/2}$).
3. Compute the **k smallest eigenvectors** of L and stack them as columns of U (n × k). (Normalised version: normalise each row of U to length 1.)
4. Treat each **row of U** as a new k-dimensional point and run **K-Means** on the rows. Point i gets the cluster of row i.

> **Why it works:** for any vector f, $f^T L f = \frac{1}{2} \sum_{ij} W_{ij} (f_i - f_j)^2 \geq 0$. Small values mean f changes only across weak edges = a **graph cut**.

- The eigenvalue **0** of L appears **once for every connected component**; with k clusters nearly separated, k eigenvalues are close to 0 (the **eigengap** also helps choose k).
- **Spectral clustering approximates the normalised graph cut** (Ncut / RatioCut), which is NP-hard to solve exactly.

Worked example: 6 nodes: triangle {1,2,3}, triangle {4,5,6}, plus one bridge edge 3-4 (all weights 1). Eigenvalues of L = D - W: 0.000, 0.438, 3.000, 3.000, 3.000, 4.562. Only the first is 0 (one connected graph) and the second is small (0.438) - a clear gap, so K = 2. The second eigenvector (Fiedler vector) is (−0.46, −0.46, −0.26, +0.26, +0.46, +0.46): its **sign** separates {1,2,3} from {4,5,6} (for K = 2 no K-Means is needed).

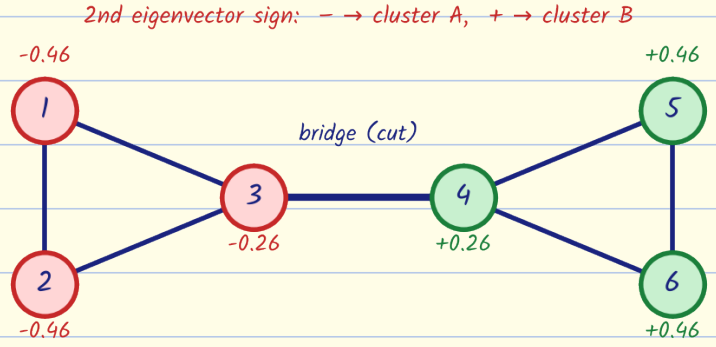


Fig 5.14 : Two triangles joined by a bridge; the 2nd Laplacian eigenvector splits them

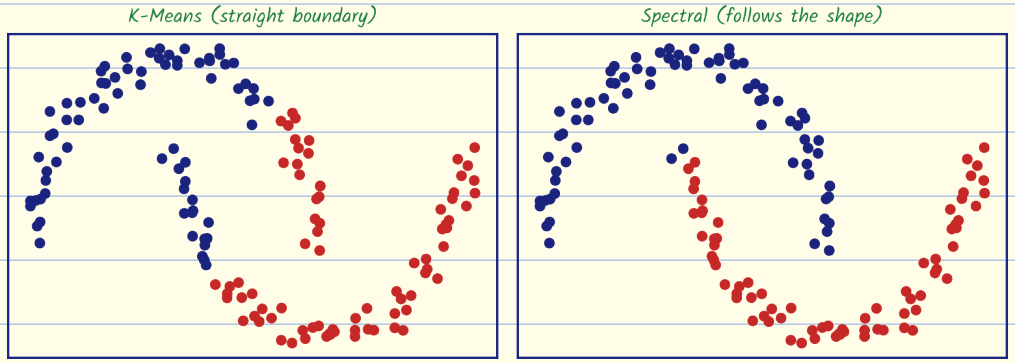


Fig 5.15 : Two half-moons: K-Means cuts straight through them, spectral clustering follows the shape

- **Pros:** arbitrary shapes, uses only a similarity matrix, strong results on images / graphs. **Cons:** $O(n^3)$ eigen-decomposition (use sparse graphs / Lanczos), $n \times n$ matrix, choice of σ or neighbours, K must be known, sensitive to graph construction.

Part V : Comparison and Revision

5.15 Comparison of Clustering Methods

Method	Type	Needs K?	Shape handled	Key idea / cost
Agglomerative	Hierarchical, hard	No	Depends on linkage	Merge closest pair; $O(n^3)$
Divisive (DIANA)	Hierarchical, hard	No	Depends on split	Split by splinter group; costly
K-Means	Partitional, hard	Yes	Round, equal size	Mean + nearest centre; $O(nKdt)$
Fuzzy C-Means	Soft (fuzzy)	Yes	Round	Membership u_{ij} weighted by m
Rough K-Means	Soft (rough)	Yes	Round	Lower / upper sets, weights w_l, w_u
EM / GMM	Soft (probabilistic)	Yes	Elliptical	Maximise likelihood; E and M steps
Spectral	Graph-based	Yes	Any (connected)	Eigenvectors of Laplacian + K-Means

* **Study tip:**
Choosing a method: need a tree or small data → agglomerative. Large data, round clusters → K-Means. Overlapping clusters → FCM / EM. Vague boundaries → Rough K-Means. Strange shapes → spectral.

Quick Revision - Unit V

- **Clustering** = unsupervised grouping; high intra-cluster, low inter-cluster similarity. Features must be scaled; pick a distance first.
- **Hard partition:** non-empty, cover X, disjoint. **Soft partition:** memberships $u_{ij} \in [0,1]$, row sum 1. Number of partitions = Stirling $S(n,K) \approx K^n/K!$.

- **Matrix factorization:** $V \approx W H$; K-Means = binary W; NMF (non-negative) gives soft, parts-based clusters.
- **Divisive** = top-down (DIANA, bisecting K-Means). **Agglomerative** = bottom-up; linkage: single (min), complete (max), average; dendrogram cut gives clusters.
- **K-Means:** $J = \Sigma\Sigma||x - \mu||^2$; assign → update; converges to a local minimum; choose K by elbow / silhouette; k-means++ init; K-Medoids for outliers.
- **FCM:** $u_{ij} = 1 / \Sigma(d_{ij}/d_{ik})^{2/(m-1)}$; $v_j = \Sigma u^m x / \Sigma u^m$. **Rough K-Means:** lower / upper approx., $c = w_l \cdot \text{mean(lower)} + w_u \cdot \text{mean(boundary)}$.
- **EM / GMM:** E-step γ_{ik}; M-step N_k, μ, Σ, π; likelihood never decreases; K-Means is the hard limit.
- **Spectral:** graph $W \rightarrow L = D - W \rightarrow k$ smallest eigenvectors → K-Means on rows; finds non-convex clusters; eigenvalue 0 count = components.

Likely exam questions

1. Define clustering. Explain the different types of clustering with a neat diagram and give applications.
2. What is partitioning of data? Differentiate hard and soft partitions with the conditions each must satisfy.
3. Explain matrix factorization for clustering (K-Means and NMF as $V \approx WH$) with an example.
4. Explain divisive and agglomerative hierarchical clustering. Cluster the points 1, 2, 6, 8, 15 by single linkage and draw the dendrogram.
5. Describe the K-Means algorithm. Apply two iterations on a small data set and discuss how K is chosen, its advantages and limitations.
6. Explain the Fuzzy C-Means algorithm with the membership and centre update equations. How does it differ from K-Means?
7. What are rough sets? Explain the Rough K-Means algorithm with lower and upper approximations.
8. Explain EM-based clustering using Gaussian mixtures (E-step and M-step). Explain spectral clustering and where it is better than K-Means.